Homework Helper's Guide to Software Requirements

August 14, 2026

CONTENTS

Homework Helper's Guide to Software Requirements

- Software Requirements Engineering — Explained So a Kid Could Master It
- Meet Your Guides
- How to Use This Book
- Table of Contents

Chapter 5: Finalizing Requirements Development

- Part 1: Prototyping — Trying Ideas Before You Build
- Purposes of Prototypes
- Paper Prototypes
- Mock-ups
- Wireframe
- Proof of Concept
- Typical Applications of Software Prototypes (Table 15-1)
- Classes of Prototype Attributes
- Ways to Fit Prototyping into the SDLC (Figure 15-1)
- Working with Prototypes (Figure 15-2)
- Dialog Map
- Risks of Prototyping
- Prototyping Success Factors
- Part 2: Prioritizing — What Gets Built First?
- Why Prioritize Requirements?
- Prioritization Pragmatics
- In or Out
- Pairwise Comparison and Rank Ordering
- MoSCoW
- Three-Level Scale (Importance × Urgency)
- Multipass Prioritization

The $100 Game (Resource-Allocation Framing)

Value/Cost/Risk-Based Prioritization Model

Part 3: Validating — Checking Your Work Before It's Too Late

Validation and Verification

Reviewing Requirements

Peer Desk Check

Pass-Around

Walkthrough

Formal Peer Reviews

Inspection

The Inspection Process

Defect Checklist — Completeness

Defect Checklist — Correctness

Defect Checklist — Quality Attributes

Requirements Review Tips

Requirements Review Challenges

Prototyping in Requirements Validation

Testing the Requirements

Acceptance Criteria

Part 4: Reusing — Don't Reinvent the Wheel

Why Reuse Requirements?

Dimensions of Requirements Reuse

Requirement Evolution

Requirement Patterns

Reuse Barriers

Reuse Success Factors

Part 5: Using Requirements — Where All This Work Goes

Use of Requirements

🏆 Chapter Wrap-Up

Chapter 6: Requirements For Specific Project Classes

📦 Packaged Solution Projects (COTS & SaaS)

What Is a Packaged Solution (COTS)?

Software as a Service (SaaS) & Cloud Solutions

Why Requirements Still Matter for Packaged Solutions

Configuring, Integrating & Extending a Package

Requirements for Selecting a Packaged Solution

The Spectrum of Implementation Effort

Common Challenges With Packaged Solutions

🤝 Outsourced Projects

Appropriate Levels of Requirements Detail

Acquirer–Supplier Interactions

Plan Time for Multiple Review Cycles

Hold Requirements Workshops Immediately Followed by Implementation Tasks

Peer Reviews and Prototypes Reveal Supplier Interpretation

Risk: Contractors Lacking Domain Knowledge

🔄 Business Process Automation Projects

Business Process Analysis (BPA)

Business Process Reengineering (BPR)

Business Process Improvement (BPI)

Business Process Management (BPM)

Business Process Model and Notation (BPMN)

📊 Big Data Projects

What Is Big Data?

Managing Big Data

Modeling Related Data With ERDs

Examples of Big-Data-Generating Organizations

Data vs. Information

The Simple Business Analytics Framework

Information Usage by People

Big Data Structure

Data-Based Requirements

Defining Analyses That Transform the Data

🤖 Embedded and Real-Time Systems Projects

System Requirements, Architecture, and Allocation

A System's Architecture Has Three Elements

System Requirements Decomposition & Allocation

Timing Requirements for Real-Time Systems

Additional Timing & Scheduling Issues to Explore

🏆 Chapter Wrap-Up

Chapter 7: Requirements Management

What Is Requirements Management?

Definition of Requirements Management

Requirements Baseline

The Four Pillars: How We Manage Requirements

Major Requirements Management Activities (The Four Pillars)

Version Control

Change Control

Requirements Status Tracking

Requirements Tracing

When Things Need to Change: Handling New Requirements

Four Ways to Accommodate New or Changed Requirements

Keeping Good Records: Attributes, Status, and Issues

Requirement Attributes

Tracking Requirements Status

Resolving Requirements Issues

Why and How We Manage Change

Why Manage Changes

Managing Scope Creep

Setting Up the Rules: Policy, Process, and the CCB

Change Control Policy

Change Control Process Description

The Change Control Board (CCB)

CCB Composition

CCB Decision-Making

Tools and Measurement

Change Control Tools

Measuring Change Activity

Change Impact Analysis: Being a Requirements Detective

Impact Analysis Procedure

Simple Evaluation Procedure

Figure 28-5 — Checklist: Understanding the Implications

Figure 28-6 — Checklist: What Work Products Are Affected

Figure 28-7 — Worksheet: Estimating the Effort

Figure 28-8 — Impact Analysis Template

Managing Change on Agile Teams

Change Management on Agile Projects

🏆 Chapter Wrap-Up

Chapter 8: Requirements Tracing and Tools

Motivations for Tracing Requirements

Finding Missing Requirements

Finding Unnecessary Requirements

Certification and Compliance

Change Impact Analysis

Maintenance

Project Tracking

Reengineering

Reuse

Testing

The Requirements Traceability Matrix (RTM)

What Is a Requirements Traceability Matrix (RTM)?

Table 29-1 Style: One Combined Matrix

Table 29-2 Style: Multiple Separate Matrices

Suspect Links

A Requirements Tracing Procedure

Step 1: Educate the Team and Management

Step 2: Select Which Link Relationships to Define

Step 3: Choose the Matrix Style and Storage Mechanism

Step 4: Identify Which Parts of the Product Need Traceability

Step 5: Identify Who Supplies Each Type of Link, and the Coordinator

Step 6: Modify Development Procedures to Update Links Promptly

Step 7: Define Labeling Conventions

Step 8: Provide Trace Info as Small Units of Work Are Completed

Step 9: Audit the Trace Information Periodically

Limitations of the Document-Based Approach to Tracing

Difficult to Keep Documents Current and Synchronized

Communicating Changes to All Affected Team Members Is a Manual Process

Not Easy to Store Supplementary Attributes About Each Requirement

Hard to Define Links Between Requirements and Other System Elements

Tracking Status of Individual Requirements and the Whole Set Is Cumbersome

Concurrently Managing Requirement Sets Planned for Different Releases Is Tricky

Reusing a Requirement Means Manually Copying Text From One Document Into Another

Difficult for Multiple, Especially Geographically Separated, Participants to Modify Requirements Together

No Convenient Place to Store Rejected/Deleted Requirements

Identifying Missing, Duplicate, and Unnecessary Requirements Is Difficult

Requirements Development Tools

Elicitation Tools

Prototyping Tools

Modeling Tools (e.g., UML)

Requirements Management (RM) Tools and Their Benefits

Manage Versions and Changes

Store Requirements Attributes

Facilitate Impact Analysis

Identify Missing and Extraneous Requirements

Track Requirements Status

Control Access

Communicate With Stakeholders

Reuse Requirements

Track Issue Status

Generate Tailored Subsets

RM Tool Features (Figure 30-1)

RM Tool Feature Families

RM Tool Capabilities and Integration With Other Tools (Figure 30-2)

RM Tools Integrate With Other Kinds of Tools (Figure 30-2)

Setting Up the Tool and Processes

Assign an Experienced BA to Own Tool Setup and Process Adaptation

Think Carefully About the Requirement Types You Define

Use the Tool to Facilitate Communication With Stakeholders in Various Locations

Don’t Try to Capture Requirements Directly in the RM Tool During Early Elicitation Workshops

Use Requirements-Development (RD) Tools During Elicitation Only If Confident They Won't Slow Discovery Down
Don't Define Trace Links Until Requirements Stabilize
Set a Cutover Date
Facilitating User Adoption
Identify a Tool Advocate
Overcome the Belief That "The Tool Won't Add Value"
Train Users Rather Than Expecting Them to Self-Discover the Tool
Don't Expect Instant Results
Reference
Where These Ideas Come From (Wiegers & Beatty)
🏆 Chapter Wrap-Up
Chapter 9: Improving Requirements Processes
Why Bother Improving? (The Big Picture)
Objective of Process Improvement
How Requirements Connect to Everything Else on the Project
How Requirements Relate to Other Project Processes
Project Planning Process
Project Tracking and Control Process
Change Control Process
Acceptance & System Testing Processes
Construction Process
User Documentation Process
Who Cares About Requirements? Meet the Stakeholders
Requirements and Various Stakeholder Groups
Management
Marketing or Product Management
Technical Support
Users

Project Sponsor

Systems Engineering

Procurers

Legal Department

Getting People to Say Yes to Change

Gaining Commitment to Change

Resistance Form 1: “I Don’t Have Time to Improve”

Resistance Form 2: “Change Control Is Just a Barrier”

Resistance Form 3: “Writing This Down Is a Waste of Time”

The Golden Rules of Getting Better

Fundamentals of Software Process Improvement

Principle 1: Evolutionary and Continuous

Principle 2: People Change Only With Incentive

Principle 3: Goal-Oriented Change

Principle 4: Treat Improvement as Mini-Projects

Finding the REAL Problem

Root Cause Analysis

The Circle of Getting Better

The Process Improvement Cycle

Building Blocks: Process Assets

Types of Process Assets

Checklist

Example (as a process asset)

Plan

Policy

Procedure

Template

Requirements Engineering Process Assets

Measuring Whether You’re Actually Getting Better
Performance Indicators (KPIs)
Drawing the Map to a Better Future
Creating a Requirements Process Improvement Road Map
🏆 Chapter Wrap-Up
Chapter 10: Software Requirements & Risk Management
What Is Risk Management?
Risk Management
Assessment
Identification
Analysis
Prioritization
Avoidance
Control
Management Planning
Resolution
Monitoring
Where Do Requirements Risks Hide?
Requirements-Related Risks by Activity
Risks When Gathering Requirements (Elicitation)
Requirements Elicitation Risks
Product vision and project scope
Time spent on requirements development
Customer engagement
Completeness and correctness of requirements specifications
Requirements for innovative products
Defining non-functional requirements
Customer agreement on requirements

Unstated requirements

Existing product used as the requirements reference

Distrust between the business and the development team

Risks When Studying Requirements (Analysis)

Requirements Analysis Risks

Requirements prioritization

Technically difficult features

Unfamiliar technologies, methods, languages, tools, or hardware

Risks When Writing Requirements Down (Specification)

Requirements Specification Risks

Requirements understanding

Time pressure to proceed despite open issues

Ambiguous terminology

Design included in requirements

Risks When Checking Requirements (Validation)

Requirements Validation Risks

Invalidated requirements

Inspection proficiency

Risks When Managing Requirements Over Time (Management)

Requirements Management Risks

Changing requirements

Requirements change process

Unimplemented requirements

Expanding project scope

Our Chapter's Guidebook

Reference: Karl Wiegers and Joy Beatty, Software Requirements

🏆 Chapter Wrap-Up

🧠 Total Recall Cheat Sheet

Chapter 5: Finalizing Requirements Development

Chapter 6: Requirements For Specific Project Classes

Chapter 7: Requirements Management

Chapter 8: Requirements Tracing and Tools

Chapter 9: Improving Requirements Processes

Chapter 10: Software Requirements & Risk Management

Homework Helper’s Guide to Software Requirements

SOFTWARE REQUIREMENTS ENGINEERING — EXPLAINED SO A KID COULD MASTER IT

Covering Chapters 5–10: Finalizing Requirements, Special Project Types, Requirements Management, Tracing & Tools, Improving the Process, and Risk Management

---

Meet Your Guides

Every idea in this book is explained through one continuing story, so nothing ever feels like a random, disconnected fact. Here’s who you’ll be following through all six chapters:

- 👷 **Maya** — the project lead / business analyst. She talks to teachers and students to figure out exactly what the app needs to do, and writes it all down clearly.
- 👩‍💻 **Sam** — the developer. Sam turns Maya’s requirements into real, working code.
- 🔍 **Priya** — the tester. Priya makes sure everything actually works the way it’s supposed to, before anyone else finds out it doesn’t.
- 👨‍🏫 **Mr. Chen** — their teacher and project sponsor. He represents “the customer” — the person who asked for the project and has to be happy with the result.
- 🎓 **The Students** — the people who will actually use the finished app every day.

Together, they’re building **Homework Helper** — a mobile app for their school that helps students track assignments, deadlines, and get study reminders. Every single idea in this book — no matter how technical it sounds — gets explained through something that happens to this team while they build it.

How to Use This Book

Every topic follows the exact same four-part pattern, so you always know what to expect:

> **In Simple Words:** the idea explained using something you already know — LEGO, pizza, recipes, treasure hunts, sports.
>
> **Example — Homework Helper App:** the idea actually happening to Maya, Sam, Priya, or Mr. Chen, with real (made-up) details.
>
> 🧠 **Remember It Like This:** one short, punchy sentence designed to stick in your memory forever.

Whenever a brand-new big word shows up for the first time, you’ll see a box like this:

> 📖 **New Word!** *(term)* means *(simple everyday definition)*.

At the end of every big section, there's a 🎯 **Quick Recap** — a fast bulleted review of everything you just read. And at the end of every chapter, there's a 🏆 **Chapter Wrap-Up** that pulls the whole story together and tells you the handful of things worth remembering above all else.

Real diagrams and tables from the original course material are included too — redrawn as clean tables or step lists, each with a one-line "🖼️ What this picture is telling you" caption so you understand exactly what the original figure was trying to show, in plain language.

Tip for photographic memory: Read a chapter normally first. Then flip to the **Total Recall Cheat Sheet** at the very end of this book — it lists every single topic from every chapter next to just its one-line memory hook. Cover the hooks with your hand and see how many you can recall just from the topic name. That's the fastest way to lock all of this into long-term memory.

Table of Contents

Chapter 5 — Finalizing Requirements Development Part 1: Prototyping · Part 2: Prioritizing · Part 3: Validating · Part 4: Reusing · Part 5: Using Requirements

Chapter 6 — Requirements For Specific Project Classes Packaged Solutions (COTS & SaaS) · Outsourced Projects · Business Process Automation · Big Data Projects · Embedded & Real-Time Systems

Chapter 7 — Requirements Management The Four Pillars · Handling New Requirements · Attributes, Status & Issues · Managing Change · Policy, Process & the CCB · Tools & Measurement · Change Impact Analysis · Agile Change Management

Chapter 8 — Requirements Tracing and Tools Motivations for Tracing · The Traceability Matrix (RTM) · A Tracing Procedure · Limits of Document-Based Tracing · Requirements Development Tools · RM Tools & Benefits · RM Tool Features & Capabilities · Setting Up Tools & Process · User Adoption

Chapter 9 — Improving Requirements Processes Why Bother Improving? · How Requirements Connect to Everything · Meet the Stakeholders · Getting People to Say Yes to Change · The Golden Rules of Getting Better · Finding the Real Problem · The Circle of Getting Better · Process Assets · Measuring Improvement · Drawing the Map to a Better Future

Chapter 10 — Software Requirements & Risk Management What Is Risk Management? · Where Requirements Risks Hide · Risks in Elicitation, Analysis, Specification, Validation, and Management

Total Recall Cheat Sheet Every topic from every chapter, next to its one-line memory hook — for fast review.

Chapter 5: Finalizing Requirements Development

Maya, Sam, Priya, and Mr. Chen have already spent weeks gathering ideas for Homework Helper — but a pile of ideas isn't a finished plan yet. In this chapter, the team learns how to test their ideas with quick mock-ups, decide which features matter most, double-check everything for mistakes, borrow smart ideas instead of starting from zero, and finally hand off a rock-solid requirements list so Sam can start building and Priya can start testing.

> 📖 **New Word!** A **requirement** means something the app absolutely must be able to do — like a rule it has to follow, for example "the app must remind students about homework."

> 📖 **New Word!** A **stakeholder** means anyone who cares about the project or is affected by it — Mr. Chen, the students, and even Maya's own team are all stakeholders in Homework Helper.

Part 1: Prototyping — Trying Ideas Before You Build

PURPOSES OF PROTOTYPES

In Simple Words: Imagine you want to build a LEGO castle, but you're not sure it will look right. So first you snap together a tiny, rough version with spare pieces, just to see the shape. A prototype is exactly that — a quick practice version of part of an app, built before anyone spends real time and money on the real thing.

> 📖 **New Word!** A **prototype** means a rough, practice version of something, built to test an idea before making the real final thing. **The three big reasons teams build them:**

- To clarify, complete, and check ("validate") requirements
- To explore different design ideas before committing to one
- To create a small starting piece that can actually grow into the final product

Example — Homework Helper App: Maya isn't sure students will understand her idea for a "streak counter" that shows how many days in a row someone finished their homework. Before Sam writes any real code, Maya sketches three little screens on paper and asks five students at lunch, "Does this make sense to you?" That single sketch does double duty: it clarifies exactly what the streak counter should do, AND it lets Maya compare two design ideas (a flame icon vs. a number badge) before picking one.

🧠 **Remember It Like This:** A prototype is a "practice castle" — build it small and rough first so the real one doesn't collapse.

PAPER PROTOTYPES

In Simple Words: A paper prototype is the cheapest, fastest way to test an idea — you just draw it! Using paper, index cards, sticky notes, or a whiteboard, you sketch what a screen might look like, and you can redraw it in seconds if something's wrong. **Key facts:**

- Tools: paper, index cards, sticky notes, whiteboards
- It tests whether users and developers actually share the same understanding of a requirement — if two people point to two different boxes for the same idea, you've just found a mismatch worth fixing
- It's fast to redraw, so you can try idea after idea — that's called **iteration**

> 📖 **New Word! Iteration** means doing something again and again, improving it a little bit each time.

- It's great for polishing requirements before you move into a detailed screen design, a working prototype, or real construction
- It helps keep everyone's expectations realistic, since an obviously rough sketch never looks "finished"

Example — Homework Helper App: Maya grabs a stack of index cards and draws one box per screen: "Home Screen," "Assignment List," "Add Reminder." She lays them on the cafeteria table in front of three students and asks them to point to where they'd tap to see today's homework. When two students point to different cards, Maya knows right away her navigation idea is confusing — and she redraws it in two minutes, no code required.

🧠 **Remember It Like This:** Paper prototypes are "sketch it, don't code it" — if a crayon can fix the problem, don't call a programmer yet.

MOCK-UPS

In Simple Words: A mock-up is a prototype that only shows the outside — the screens, buttons, and colors — like a stage set in a school play that looks real from the audience but has nothing behind the walls. It's also called a **horizontal prototype**, because it slices across just the top layer (the UI) instead of going deep, and it lets people judge whether the app will actually let them do their job.

> 📖 **New Word!** A **horizontal prototype** (mock-up) means a prototype that shows a slice of the user interface only — no real working parts underneath. **A mock-up can demonstrate:**

- What functional options are available
- The look & feel — colors, layout, graphics, controls
- How navigation works between screens **Two more things worth knowing:**
- A mock-up "implies" what the app will do without actually doing it — nothing real is happening underneath, it just looks like it does
- Mock-ups are also a great way to explore specific behaviors so the team can refine requirements before any real code gets written

Example — Homework Helper App: Sam builds a clickable mock-up showing the Home Screen, the Assignment List, and a "Mark Done" button — but tapping "Mark Done" doesn't actually save anything, it just slides to the next screen so it *looks* like it worked. Mr. Chen clicks through it and says, "This helps me see if teachers could actually use this in three minutes between classes" — exactly the kind of judgment a mock-up is built for. Since this is a throwaway mock-up, Maya reminds Sam not to worry about picking exact colors yet — the point is testing the workflow, not the paint job.

🧠 **Remember It Like This:** A mock-up is a movie set — it looks completely real from the front, but there's nothing behind the door.

WIREFRAME

In Simple Words: A wireframe is a specific kind of throwaway sketch used just for designing custom screens or websites — like a blueprint that shows where each room goes in a house, without picking out the paint color or furniture yet. **A wireframe helps you understand three things:**

1. Conceptual requirements — what the screen is actually about, and what users are trying to do
2. Information architecture / navigation design — how you move between screens
3. High-resolution, detailed page design — the fine details, once you're ready for them

🖼️ **What this picture is telling you:** Two real wireframe examples exist in the source material — a computer-made clickable wireframe called "Directory Profile Page" (built in a tool called Balsamiq, showing placeholder photo boxes, filler text, and an attachments list) and a hand-drawn notebook sketch called "Find Fractions" (showing a home screen with boxes for "Product Selector," "Customer Home," and "Order History," with arrows pointing down to a "Number Menu," a data table, and an approval form). Both prove the same point: arrows between boxes show HOW you move between screens, long before anyone picks real colors or writes real words.

Example — Homework Helper App: Maya copies that exact "Find Fractions" style for Homework Helper: she draws a box for "Home Screen" with arrows pointing to boxes for "This Week's Assignments," "Add a Reminder," and "My Streak," so Sam can see the whole map of how a student clicks around the app before a single pixel is designed.

🧠 **Remember It Like This:** A wireframe is the house blueprint — it shows where the rooms connect, not what color the walls are.

PROOF OF CONCEPT

In Simple Words: A proof of concept is the opposite of a mock-up — instead of just looking real on the outside, it actually WORKS, all the way through, for one small piece. It's also called a **vertical prototype**, because it slices straight down through every technical layer, like digging one deep test hole instead of skimming the whole surface — so it behaves like the real system, at least for that one slice.

> 📖 **New Word!** A **vertical prototype** (proof of concept) means a prototype that makes one small piece of the system fully work, from the screen all the way down through every technical layer.

Example — Homework Helper App: Sam isn't sure the app can really sync with the school's Google Classroom homework portal — that's the scariest technical question. So instead of designing pretty screens, Sam spends two days building just enough real code to pull ONE real assignment from Mr. Chen's Google Classroom and show it on a plain, ugly test screen. It isn't pretty, but it proves the connection actually works — exactly what a proof of concept is for.

🧠 **Remember It Like This:** Proof of concept means "does the plumbing actually work?" — never mind how the bathroom looks yet.

TYPICAL APPLICATIONS OF SOFTWARE PROTOTYPES (TABLE 15-1)

In Simple Words: There are really two big questions you ask about any prototype: will you throw it away or grow it into the real product, and is it a mock-up (looks) or a proof of concept (works)? Crossing those two questions, like two roads meeting, gives you four different jobs a prototype can do.

	Throwaway	Evolutionary
Mock-up	Clarify and refine user and functional requirements. Identify missing functionality. Explore user interface approaches.	Implement core user requirements. Implement additional user requirements based on priority. Implement and refine websites. Adapt system to rapidly changing business needs.
Proof of concept	Demonstrate technical feasibility. Evaluate performance. Acquire knowledge to improve estimates for construction.	Implement and grow core multi-tier functionality and communication layers. Implement and optimize core algorithms. Test and tune performance.

🖼️ **What this picture is telling you:** This is Table 15-1 from the textbook. It shows the four honest jobs a prototype can do — for example, a throwaway mock-up's job is to clarify requirements and then get thrown away, while an evolutionary proof of concept's job is to slowly grow into the real, working engine of the app.

Example — Homework Helper App: Maya's paper sketches are a **throwaway mock-up** — built to clarify the streak-counter idea, then tossed. Sam's Google Classroom connection test is heading toward becoming an **evolutionary proof of concept** — if it keeps working, Sam won't throw it away, he'll keep building on top of that same code until it becomes the real syncing feature.

🧠 **Remember It Like This:** Two questions, four jobs — throw it away or grow it, looks or works — that's the whole table.

CLASSES OF PROTOTYPE ATTRIBUTES

In Simple Words: Every prototype can be described using three simple labels, kind of like describing a pizza by its size, its toppings, and whether it's frozen or fresh: what part it covers (Scope), whether you'll keep it (Future Use), and what it's made of (Form).

Dimension	Option A	Option B
Scope	Mock-up — tests the user experience / sample functionality (like an ATM screen)	Proof of concept — tests technical soundness
Future Use	Throwaway — gets discarded after feedback	Evolutionary — grows into the final product through more iterations
Form	Paper prototype — a sketch on paper, whiteboard, or a drawing tool	Electronic prototype — actual working software for part of the solution

> 📖 **New Word!** A **throwaway prototype** means one you build fast and delete once it's answered your question. An **evolutionary prototype** means one you keep improving until it becomes the real product.

Example — Homework Helper App: Maya's index cards are Scope = mock-up, Future Use = throwaway, Form = paper. Sam's Google Classroom test is Scope = proof of concept, Future Use = evolutionary (so far), Form = electronic. Same team, same app, two completely different prototypes described with the same three labels.

🧠 **Remember It Like This:** Every prototype has three tags, like a luggage tag: what it's testing, whether it's keeping its job, and what it's made of.

WAYS TO FIT PROTOTYPING INTO THE SDLC (FIGURE 15-1)

In Simple Words: Prototyping isn't one extra step tacked onto a project — it weaves in and out of the WHOLE process of building software (called the **SDLC**, or software development life cycle), feeding ideas back and forth the entire way through.

> 📖 **New Word! SDLC** stands for Software Development Life Cycle — all the stages a team goes through to build software, from the first idea to delivering the finished product.

1. **Elicit user requirements** sits at the top, feeding down into two paths at once.
2. On one side: **Develop throwaway mock-up** → loops back up to refine the requirements, and also feeds into **Construct evolutionary prototype** and **Design user interface**.
3. In the center: **Develop or refine architecture** → also feeds **Construct evolutionary prototype**.
4. On the other side: **Construct proof of concept** → loops back up to "begin next iteration" of refining the architecture, and also feeds **Construct evolutionary prototype** and **Design software architecture**.
5. **Construct evolutionary prototype** → feeds into **Verify and deliver product incrementally**.
6. **Design user interface** and **Design software architecture**, plus incremental delivery, all flow down into **Construct and verify product**.
7. Finally, **Construct and verify product** → **Deliver product**.

🖼️ **What this picture is telling you:** This is Figure 15-1. It shows that mock-ups and proofs-of-concept aren't a side project — they constantly send information back to the requirements and the architecture, like two scouts running back and forth to headquarters with news, before everything merges to build and deliver the real product.

Example — Homework Helper App: Maya's mock-ups keep sending fixes back to the requirements list ("students want a big Done button, not a small checkbox"), while Sam's proof of concept keeps sending fixes back to the technical plan ("we need a different way to talk to Google Classroom"). Both streams eventually merge into the real Homework Helper app that gets delivered to the school.

🧠 **Remember It Like This:** Prototyping is a boomerang, not an arrow — it always throws information back before the real building begins.

WORKING WITH PROTOTYPES (FIGURE 15-2)

In Simple Words: There's a natural order for turning a rough idea into a finished screen design — and at every step, you're allowed to send corrections backward if you discover a mistake, just like double-checking your math homework and erasing a wrong answer from three steps back.

1. **Use Case** (a written story of how someone uses the app)
2. **Dialog Map** (a map of how screens connect)
3. **Throwaway Prototype or Wireframe** (a rough sketch to test the idea)
4. **Detailed User Interface Design** (the polished, final-looking screens)

Feedback can flow backward at every stage: the Dialog Map can send fixes back to the Use Case, the Prototype/Wireframe can send fixes back to the Dialog Map, and the Detailed UI Design can send fixes ALL the way back to the original Use Case.

🖼️ **What this picture is telling you:** This is Figure 15-2. Going from "here's a story about what a user needs" to "here's the polished final screen" isn't a one-way street — mistakes found late can (and should) be reported all the way back to the beginning.

Example — Homework Helper App: Maya writes a use case: "A student opens the app before bed to check tomorrow's homework." That becomes a dialog map, then a wireframe, then Sam's detailed screen design. But while designing the detailed screen, Sam realizes the use case never mentioned what happens if there's no internet — so he sends that gap all the way back to Maya's original use case to fix it.

🧠 **Remember It Like This:** Four steps forward, but the eraser always works backward too.

DIALOG MAP

In Simple Words: A dialog map is like a subway map for an app — instead of showing train stations connected by lines, it shows screens connected by the taps that move you from one to the next.

> 📖 **New Word!** A **dialog map** means a diagram showing how a user moves between the different screens or states of an app.

Example — Homework Helper App: Maya draws circles for "Home Screen," "Assignment Details," and "Add Reminder," with arrows labeled "tap an assignment," "tap the + button," and "tap Save" showing exactly which button takes you where. When Priya asks, "How does a student get back to Home from Add Reminder?" Maya checks the dialog map and realizes there's no arrow for that! She adds a "Cancel" button and a new arrow before Sam starts building.

🧠 **Remember It Like This:** A dialog map is a subway map for buttons — every arrow is a tap that takes you somewhere new.

RISKS OF PROTOTYPING

In Simple Words: Prototypes are powerful, but they can trick people too — like a movie trailer that looks so good, someone assumes the whole movie must already be finished. **The four risks:**

- People pressuring you to ship the prototype as if it were the real, finished product
- Getting distracted by little visual details instead of covering all the needed options
- Everyone expecting the real app to be just as fast and polished as the rough prototype
- Spending way too much time and effort polishing something that was only supposed to answer one quick question

Example — Homework Helper App: After seeing Sam's slick clickable mock-up, Mr. Chen gets excited and asks, "Great, can we launch this to the whole school next week?" Maya has to gently explain, "It only *looks* finished — none of the buttons actually save real data yet!" That's exactly the "pressure to release the prototype" risk in action.

🧠 **Remember It Like This:** A shiny prototype is a movie trailer, not the movie — don't sell tickets yet.

PROTOTYPING SUCCESS FACTORS

In Simple Words: Just like a good school project needs a plan, a deadline, and clear goals, a good prototype needs a plan too — otherwise it can waste more time than it saves. **The six success factors:**

- Put prototyping tasks right into the project plan, with real time and resources set aside
- Decide the PURPOSE of each prototype before building it — and decide up front whether it will be thrown away or built upon
- Expect to build more than one — it's rare to get it right on the very first try
- Build throwaway prototypes as fast and cheaply as possible — just enough to answer the open question
- Don't bother prototyping things you already understand well (unless you're exploring design choices)
- Never expect a prototype to replace real, written-down requirements

Example — Homework Helper App: Before building anything, Maya writes at the top of her sketch: "Purpose: find out if students understand the streak counter icon. Type: throwaway." That one sentence keeps Sam from accidentally turning her five-minute sketch into a two-week coding project.

🧠 **Remember It Like This:** Say WHY you're building it before you build it — a prototype without a purpose is just doodling.

🎯 **Quick Recap:**

- **Purposes of Prototypes** — built to clarify, explore, and grow into the real thing.
- **Paper Prototypes** — the cheapest, fastest way to test an idea with sketches.
- **Mock-ups** — a horizontal prototype that looks real but doesn't work underneath.
- **Wireframe** — a throwaway sketch that maps out screens and navigation.
- **Proof of Concept** — a vertical prototype that actually works, deep down.
- **Typical Applications (Table 15-1)** — four jobs prototypes do: throwaway/evolutionary × mock-up/proof-of-concept.
- **Classes of Prototype Attributes** — every prototype has a Scope, a Future Use, and a Form.
- **Ways to Fit Prototyping into the SDLC** — prototyping constantly feeds information back into requirements and architecture.
- **Working with Prototypes** — Use Case → Dialog Map → Prototype/Wireframe → Detailed UI Design, with feedback loops.
- **Dialog Map** — a subway map for how screens connect.
- **Risks of Prototyping** — people can mistake a rough sketch for the finished, ready-to-ship app.
- **Prototyping Success Factors** — plan it, purpose it, and never let it replace real written requirements.

Part 2: Prioritizing — What Gets Built First?

WHY PRIORITIZE REQUIREMENTS?

In Simple Words: Imagine you only have enough money for three toppings on a pizza, but everyone wants five. You can't have everything at once, so you pick the toppings that matter most. Software teams face the same problem — high expectations and short timelines mean they must deliver the most critical, valuable features first.

> 📖 **New Word! Prioritization** means deciding what order to do things in, based on what matters most — it's how teams deal with too many demands and not enough time or people.

Example — Homework Helper App: Mr. Chen wants Homework Helper ready by the start of the new semester — only ten weeks away. Maya has a list of 20 requested features but Sam can realistically build about 8 in that time. So the team must prioritize: which 8 matter most to students and teachers right now?

🧠 **Remember It Like This:** You can't put every topping on the pizza — prioritizing is choosing the toppings that matter most.

PRIORITIZATION PRAGMATICS

In Simple Words: Before you can rank anything, you need to understand six real-world puzzle pieces about your project — like knowing the rules of a board game before you decide your strategy. **The six issues:**

1. What the customers actually need
2. How important each requirement is to those customers, compared to the others
3. WHEN each capability needs to be ready
4. Requirements that depend on other requirements being finished first
5. Requirements that must all be built together as one group
6. How much it costs to satisfy each requirement (in money, time, or effort — hard cost or soft cost)

> 📖 **New Word!** A **dependency** means one requirement can't be finished (or can't work) until another requirement is done first.

Example — Homework Helper App: Maya realizes REQ-005 ("sync with Google Classroom") is a dependency — the "auto-fill assignment details" feature (REQ-011) literally cannot work until the sync feature exists. So even if REQ-011 feels exciting, REQ-005 has to be built first no matter what the priority ranking says.

🧠 **Remember It Like This:** Before you rank the runners, you need to know the race rules — needs, importance, timing, dependencies, grouping, and cost.

IN OR OUT

In Simple Words: This is the simplest possible way to prioritize — for every single requirement, you just ask one yes-or-no question: is it IN this release, or is it OUT?

Example — Homework Helper App: The team goes through their list fast during a lunchtime meeting. “Reminder notifications?” “In.” “Dark mode theme?” “Out — maybe next year.” “Calendar view?” “In.” No ranking, no math — just a quick yes or no for each item.

🧠 **Remember It Like This:** In or Out is a light switch, not a dimmer — every requirement gets flipped one way or the other.

PAIRWISE COMPARISON AND RANK ORDERING

In Simple Words: Instead of judging all your requirements at once (which is confusing), you compare them two at a time, over and over, until you’ve built a complete ranked list — like a bracket in a sports tournament where every team eventually plays every other team.

Example — Homework Helper App: Priya compares REQ-001 (reminders) vs. REQ-007 (streak counter) — reminders win. Then REQ-001 vs. REQ-004 (filter by subject) — reminders win again. After comparing every possible pair on the list, a clear ranked order emerges, from most important to least.

🧠 **Remember It Like This:** Pairwise comparison is a tournament bracket — two requirements enter, one priority order comes out.

MOSCOW

In Simple Words: MoSCoW sorts every requirement into one of four buckets, using a name that’s easy to remember because it hides the words “Must,” “Should,” “Could,” and “Won’t” inside it.

> 📖 **New Word! MoSCoW** is a prioritization method that sorts requirements into four groups: Must, Should, Could, and Won’t.

Category	What it Means	Homework Helper Example
Must	Required for the solution to be a success	REQ-001: Send a reminder 24 hours before an assignment is due
Should	Important, include if possible, but not mandatory	REQ-004: Let students filter assignments by subject
Could	Desirable — implement only if time and resources permit	REQ-007: Show a “streak” counter for consecutive completed days
Won’t	Not implemented now — could be a future release	REQ-008: Let a parent view their child’s assignment list

Example — Homework Helper App: When Mr. Chen asks why the parent-viewing feature isn’t in version one, Maya points to the MoSCoW list: “It’s a ‘Won’t’ for this release — it’s a great idea, just not one the app collapses without. We’re saving it for version two.”

🧠 **Remember It Like This:** MoSCoW: Must happens, Should tries, Could waits, Won’t waits longer.

THREE-LEVEL SCALE (IMPORTANCE × URGENCY)

In Simple Words: Sometimes MoSCoW's four buckets are more than you need. Instead, you can just ask two questions about each requirement: how IMPORTANT is it, and how URGENT is it (does it need to happen soon)? Crossing those two answers gives you a simple High/Medium/Low priority — High means important AND urgent, Medium means important but not urgent yet (or urgent but not that important), and Low means neither important nor urgent.

Urgency \ Importance	Low	Medium	High
Low	Low/Low	Medium/Low	High/Low
Medium	Low/Medium	Medium/Medium	High/Medium
High	Low/High	Medium/High	High/High

🖼️ **What this picture is telling you:** This is the Importance × Urgency grid. The bottom-right corner (High importance / High urgency) is the top priority — build it first. The top-left corner (Low importance / Low urgency) is the bottom priority — it can wait indefinitely.

Example — Homework Helper App: REQ-001 (reminders) is High importance AND High urgency — the semester starts in two weeks and students need this on day one, so it lands in the top corner of the grid. A "custom app icon color" request is Low importance and Low urgency — it lands in the bottom corner and can wait forever if needed.

🧠 **Remember It Like This:** Important AND urgent wins the race — the further from that corner, the longer it can wait.

MULTIPASS PRIORITIZATION

In Simple Words: You can't just rank requirements once and call it done — you have to check your ranking a second time against any dependencies, because a "high priority" requirement that depends on a "low priority" one is a scheduling trap.

Example — Homework Helper App: The team first ranks "auto-fill assignment details" (REQ-011) as a top priority. But on a second pass, Maya notices it depends on the Google Classroom sync (REQ-005), which had been ranked lower. She bumps REQ-005's priority up too — otherwise Sam would hit a wall trying to build REQ-011 first.

🧠 **Remember It Like This:** Rank once, then check twice — priorities and dependencies have to agree with each other.

THE $100 GAME (RESOURCE-ALLOCATION FRAMING)

In Simple Words: Give everyone on the prioritization team a pretend $100 to "spend" on the requirements they care about most. The requirements that get the most money are clearly the ones people value most — and once someone's $100 is gone, that's it, nothing else can be "bought."

Example — Homework Helper App: Maya gives Sam, Priya, and Mr. Chen each $100 in pretend money to "buy" features for the first release. Mr. Chen decides reminders matter three times as much as the streak counter, so he spends $9 on REQ-001 and only $3 on REQ-007 — exactly the "3× as important" rule the technique is built on. Once everyone's $100 is spent, Maya adds up all the dollars per requirement across all three people — REQ-001 comes out on top with the most total dollars.

🧠 **Remember It Like This:** Pretend money reveals real priorities — people spend the most on what they truly want.

VALUE/COST/RISK-BASED PRIORITIZATION MODEL

In Simple Words: This is the most careful method of all — instead of guessing, you score every requirement with numbers for how VALUABLE it is, how much it COSTS, and how RISKY it is to build, then let a formula calculate the winner: **Priority = Value% ÷ (Cost% + Risk%)**. A requirement scores high when it gives a lot of value for not much cost or risk — like spotting a great deal at a store. **Who's involved:**

- **Project manager / business analyst** — leads the process and arbitrates conflicts
- **Customer representatives** (product champion / product owner) — supply the benefit and penalty ratings
- **Development representatives** — supply the cost and risk ratings **The steps:**

1. List every feature, use case, user story, or functional requirement in a spreadsheet
2. Customer reps rate relative **benefit** (1 = no one cares, 9 = extremely valuable)
3. Customer reps rate relative **penalty** if it's left out (1 = no one upset, 9 = serious downside) — note: low benefit + low penalty means it's cost with little value
4. The spreadsheet computes **Total Value** = benefit + penalty, and **Value%**
5. Developers rate relative **cost** (1 = quick/easy, 9 = time-consuming/expensive); the spreadsheet computes **Cost%**
6. Developers rate relative **technical risk** — the chance of NOT getting it right the first try (1 = trivial, 9 = serious feasibility/expertise/complexity concerns); the spreadsheet computes **Risk%**
7. The formula calculates **Priority = Value% ÷ (Cost% + Risk%)**

🖼️ **What this picture is telling you — Table 16-1, a real worked example (Chemical Tracking System):**

#	Feature	Rel. Benefit	Rel. Penalty	Total Value	Value%	Rel. Cost	Cost%	Rel. Risk	Risk%	Priority
1	Print a material safety data sheet.	2	4	8	5.2	1	2.7	1	3.0	**1.22**
2	Query status of a vendor order.	5	3	13	8.4	2	5.4	1	3.0	1.21
3	Generate a chemical stockroom inventory report.	9	7	25	16.1	5	13.5	3	9.1	0.89
4	See history of a specific chemical container.	5	5	15	9.7	3	8.1	2	6.1	0.87
5	Search vendor catalogs for a specific chemical.	9	8	26	16.8	3	8.1	8	24.2	0.83
6	Maintain a list of hazardous chemicals.	3	9	15	9.7	3	8.1	4	12.1	0.68
7	Change a pending chemical request.	4	3	11	7.1	3	8.1	2	6.1	0.64
8	Generate a laboratory inventory report.	6	2	14	9.0	4	10.8	3	9.1	0.59
9	Check training database for hazardous chemical training record.	3	~7	10	6.5	4	10.8	2	6.1	0.47

#	Feature	Rel. Benefit	Rel. Penalty	Total Value	Value%	Rel. Cost	Cost%	Rel. Risk	Risk%	Priority
10	Import chemical structures from structure drawing tools.	7	4	18	11.6	9	24.3	7	21.2	**0.33**
Totals		53	49	155	100.0	37	100.0	33	100.0	

This real table isn't about Homework Helper — it's from a chemical tracking system — but notice feature #1 (printing a safety sheet) wins with the HIGHEST priority score (1.22) even though its benefit was only "okay," because it's so cheap and low-risk. Feature #10 (importing chemical structures) has the LOWEST priority (0.33) because even though people wanted it, it was the most expensive and riskiest thing on the list.

Example — Homework Helper App: The team scores their own mini version: REQ-001 (reminders) gets Benefit 9, Penalty 8 (Total Value 17), Cost 2, Risk 1 — huge value for very little cost, so it scores a high priority. Meanwhile, "AI-powered study plan generator" gets Benefit 8, Penalty 3 (Total Value 11), but Cost 9, Risk 9 — high value, but so expensive and risky that it scores a low priority for release one, exactly like feature #10 in the real table.

🧠 **Remember It Like This:** Priority = Value ÷ (Cost + Risk) — the best deal wins, not just the fanciest feature.

🎯 **Quick Recap:**

- **Why Prioritize Requirements?** — limited time and money means you must choose what matters most.
- **Prioritization Pragmatics** — six things to understand first: needs, importance, timing, dependencies, grouping, cost.
- **In or Out** — the simplest method: a yes-or-no for every requirement.
- **Pairwise Comparison and Rank Ordering** — compare two at a time until a full ranking emerges.
- **MoSCoW** — sort into Must, Should, Could, Won't.
- **Three-Level Scale** — cross Importance and Urgency to get High/Medium/Low.
- **Multipass Prioritization** — double-check rankings against dependencies.
- **The $100 Game** — spend pretend money to reveal real priorities.
- **Value/Cost/Risk-Based Model** — score value, cost, and risk, then calculate Priority = Value% ÷ (Cost% + Risk%).

Part 3: Validating — Checking Your Work Before It's Too Late

VALIDATION AND VERIFICATION

In Simple Words: These two words sound alike but ask completely different questions. Verification asks, "Did we build the thing correctly?" — like checking your spelling. Validation asks, "Did we build the CORRECT thing?" — like checking you actually answered the question that was asked.

Term	The Question It Asks	Simple Version
Verification	Does the product meet its written requirements?	"Doing the thing right"
Validation	Does the product satisfy the customer's real needs?	"Doing the right thing"

> 📖 **New Word! Verification** means checking that you built the product exactly as the requirements said. **Validation** means checking that what you built actually solves the customer's real problem. **What validation tries to make sure of:**

1. Requirements accurately describe what stakeholders actually need
2. Requirements are correctly based on real business needs, system needs, and business rules
3. Requirements are complete, doable ("feasible"), and testable
4. Every requirement is necessary, and together they're enough to meet the goals
5. All requirement documents agree with each other — no conflicts
6. The requirements give a solid enough base to start designing and building

Example — Homework Helper App: Sam builds REQ-001 exactly as written: a reminder pops up 24 hours before a due date. Priya checks it and confirms it fires at exactly the right time — that's **verification**, it matches the spec. But then Mr. Chen tries it and says, "Wait, most students don't check notifications until AFTER school, not 24 hours ahead — this doesn't actually solve the 'kids forget their homework' problem." That's a **validation** failure — the team built the requirement correctly, but the requirement itself wasn't the right one.

🧠 **Remember It Like This:** Verification checks the recipe was followed. Validation checks the food actually tastes good.

REVIEWING REQUIREMENTS

In Simple Words: Before Sam starts coding, someone needs to read through the requirements the same way a teacher reads a rough draft before it's turned in — catching mistakes while they're still cheap and easy to fix.

Example — Homework Helper App: Maya finishes writing all 20 Homework Helper requirements in a document. Before showing it to Mr. Chen for final sign-off, she asks Sam and Priya to read through it first, hunting for anything confusing, missing, or contradictory.

🧠 **Remember It Like This:** Catching a typo before you print 100 copies is way cheaper than after.

PEER DESK CHECK

In Simple Words: This is the simplest kind of review — you just hand your work to ONE trusted colleague and ask them to look it over, like asking a friend to check your homework before you turn it in.

Example — Homework Helper App: Maya emails her requirements document to Priya alone and says, “Can you read through this before our team meeting? Just flag anything confusing.” Priya spots that REQ-003 never says whether the calendar shows weekends.

🧠 **Remember It Like This:** One friend, one careful read — that’s a peer desk check.

PASS-AROUND

In Simple Words: Instead of just one reviewer, a pass-around sends the same document to SEVERAL colleagues at once, and each one looks it over on their own time and adds their comments.

Example — Homework Helper App: Maya sends the requirements document to Sam, Priya, AND Mr. Chen at the same time, asking each to add comments by Friday. All three read it independently, so she gets three different sets of eyes without needing everyone in a meeting.

🧠 **Remember It Like This:** Pass-around is a group chat for reviewers — everyone reads it, nobody has to meet.

WALKTHROUGH

In Simple Words: In a walkthrough, the person who WROTE the document actually talks the group through it out loud, section by section, and asks for comments as they go.

Example — Homework Helper App: Maya gathers Sam, Priya, and Mr. Chen in a room and says, “Let’s go through the requirements together — stop me anytime something looks wrong.” When she reads REQ-009 (“works on iPhone and Android”), Sam immediately asks, “Wait, does that mean tablets too?” and they clarify it on the spot.

🧠 **Remember It Like This:** A walkthrough is a guided tour — the author drives, everyone else asks questions along the way.

FORMAL PEER REVIEWS

In Simple Words: A formal peer review isn’t casual — it follows a well-defined process with clear steps, and it ends with an actual written report saying what was checked, who checked it, and whether it’s good enough to move forward.

Example — Homework Helper App: Instead of just an informal chat, the team schedules an official review meeting with a written report at the end: “Reviewed by: Sam, Priya, Mr. Chen. Document: Homework Helper Requirements v1.2. Result: Approved with 3 minor changes needed.”

🧠 **Remember It Like This:** Formal peer review means it’s on the record, not just a hallway chat.

INSPECTION

In Simple Words: Inspection is the most powerful, most carefully structured kind of formal peer review there is — experts consider it one of the single best tools for catching software problems early, before they get expensive.

Example — Homework Helper App: For the final, official version of the requirements before Sam starts building, the team doesn't just do a quick walkthrough — they run a full, official inspection with assigned roles and a real checklist, because this document is too important to leave to chance.

🧠 **Remember It Like This:** Inspection is the "gold standard" review — when it really matters, this is the one you run.

THE INSPECTION PROCESS

In Simple Words: An inspection is like a very organized detective investigation of a document — different people play different jobs, there are rules about when you're even allowed to start, a clear sequence of steps to follow, and rules about when you're allowed to call it finished. **Who's involved:**

- The **author** of the document, plus peers of similar skill (the Analyst — Maya)
- The people whose information fed into it (the Customer — Mr. Chen)
- The people who will build things based on it (the Developer — Sam)
- Anyone responsible for systems that connect to it **The four inspection roles:**

Role	Job
Author	Wrote the document being inspected
Moderator	Runs the inspection meeting fairly and keeps it on track
Reader	Reads the document aloud, section by section, during the meeting
Recorder	Writes down every issue that gets raised

> 📖 **New Word!** A **moderator** means the person in charge of running a meeting fairly, making sure everyone gets heard and the group stays on track. **Entry criteria (you can't even START an inspection until):**

1. The document follows the standard template, with no obvious spelling, grammar, or formatting problems
2. Line numbers or unique IDs are printed so people can point to exact spots
3. Every open question is marked "TBD" (to be decided) or tracked in an issue-tracking tool
4. The moderator sampled 10 minutes of the document and found 3 or fewer major **defects**

> 📖 **New Word!** A **defect** means a mistake or problem found in a document or a product. **The inspection stages (Figure 17-2):**

1. **Initial Work Product** (the starting document)
2. **Planning**

3. **Preparation** (each reviewer reads it alone first)
4. **Inspection Meeting** (the group meets to discuss issues)
5. **Rework** (the author fixes the issues)
6. **Follow-Up** (checking the fixes were done right)
7. **Baselined Work Product** (the finished, locked-in document)

Two dotted feedback lines matter here: Rework can loop back to Preparation, and Follow-Up can loop back to Rework — meaning if the fixes were big enough, part of the whole process might repeat.

> 📖 **New Word! Baselined** means a document has been officially checked, approved, and locked in as the agreed-upon version — future changes have to go through a formal change process instead of just being edited freely. **Exit criteria (before you can call the inspection DONE):**

1. Every issue raised during the inspection has been addressed
2. Any changes made were made correctly
3. Every open issue is either fully resolved, or has a documented resolution process, target date, and owner

🖼️ **What this picture is telling you:** This is Figure 17-2. An inspection isn't just "read it once and done" — it's a real process with loops, meaning sloppy documents might have to be re-prepared and re-meeting'd until they're truly clean.

Example — Homework Helper App: Maya is the **Author** of the requirements document. Mr. Chen is the **Moderator**, keeping the meeting on schedule. Priya is the **Reader**, reading each requirement out loud. Sam is the **Recorder**, typing every issue into a shared list. Before they even start, Mr. Chen checks the entry criteria — line numbers are printed, and his 10-minute sample check found only 2 issues, so they're clear to begin. After the meeting, Maya reworks three unclear requirements, Priya follows up to confirm the fixes are correct, and the document finally gets **baselined**.

🧠 **Remember It Like This:** Four roles, seven steps, and two loops back — an inspection is a courtroom for your document, not a casual read.

DEFECT CHECKLIST — COMPLETENESS

In Simple Words: This checklist makes sure nothing important got left out — like checking a packing list before a camping trip to make sure you didn't forget the tent. **The checklist:**

- Do the requirements cover every known need, from customers and from the system itself?
- Is any needed information missing — and if so, is it marked TBD?
- Are the algorithms needed for functional requirements actually spelled out?
- Are all outside hardware, software, and communication connections defined?
- Is the expected behavior written down for every anticipated error condition?
- Do the requirements give designers and testers enough to work with?
- Does every requirement say how important (its priority) it is?
- Is every requirement actually in scope for this release or iteration?
- Is every requirement actually necessary for the product?

Example — Homework Helper App: During the inspection, Priya notices REQ-005 (Google Classroom sync) never says what should happen if the internet connection drops in the middle of syncing. That's a completeness gap — an error condition nobody described — so it gets flagged and marked TBD until Maya writes the missing rule.

🧠 **Remember It Like This:** Completeness asks: "Did we forget anything?" — like checking your backpack twice before the trip.

DEFECT CHECKLIST — CORRECTNESS

In Simple Words: This checklist makes sure everything that IS written down is actually right — no contradictions, no confusing sentences, no impossible promises. **The checklist:**

- Do any requirements conflict with or duplicate each other?
- Is every requirement clear, concise, unambiguous, and grammatically correct?
- Are any error messages the app shows actually clear and meaningful?
- Is every requirement technically feasible to build, given known limits?

Example — Homework Helper App: Sam spots that REQ-002 says "students can mark an assignment done" while REQ-014 (written weeks earlier) says "assignments cannot be edited once created." Those two directly contradict each other — that's a correctness defect the inspection catches before it confuses Sam mid-build.

🧠 **Remember It Like This:** Correctness asks: "Is what's written actually TRUE and clear?" — no contradictions allowed.

DEFECT CHECKLIST — QUALITY ATTRIBUTES

In Simple Words: This checklist zooms in specifically on the app's "personality traits" — things like speed, security, and reliability — making sure they're described with real, measurable numbers instead of vague wishes. **The checklist:**

- Are quality attribute goals properly specified, documented, and quantified, with acceptable trade-offs?
- Are all quality requirements measurable and verifiable?

Example — Homework Helper App: An early draft of REQ-015 just says "the app should be fast." That's not testable! During inspection, Priya flags it, and Maya rewrites it as: "REQ-015: The app shall load the Assignment List screen within 2 seconds on a standard school Wi-Fi connection." Now it's something Priya can actually test with a stopwatch.

🧠 **Remember It Like This:** "Fast" isn't testable. "Under 2 seconds" is — quality needs numbers, not just nice words.

REQUIREMENTS REVIEW TIPS

In Simple Words: Good reviews don't just happen — they need a bit of planning, just like a good study session works better with a plan than by randomly flipping through a textbook. **The seven tips:**

1. Plan the examination — focus on specific sections instead of the whole document at once
2. Start early, so major defects get caught sooner
3. Allocate sufficient time — set aside real, dedicated calendar hours
4. Provide context for reviewers who aren't familiar with the project
5. Set the review scope — tell reviewers exactly what to examine and what issues to look for
6. Limit re-reviews — don't review the same material more than 3 times
7. Prioritize review areas — focus on risky or frequently-used functionality

Example — Homework Helper App: Before the inspection meeting, Mr. Chen tells everyone: "Today, just focus on the reminder-notification requirements — that's the riskiest part since it depends on phone permissions. We'll review the calendar screen next week." That's tips #1 and #7 in action together.

🧠 **Remember It Like This:** A review with no plan is just skimming — plan it like you'd plan a study session.

REQUIREMENTS REVIEW CHALLENGES

In Simple Words: Even a great review plan can run into real-world obstacles — some documents are just huge, some teams are just big, and sometimes people aren't even in the same room. **The four challenges:**

- Large requirements documents
- Large inspection teams
- Geographically separated reviewers
- Unprepared reviewers

Example — Homework Helper App: When the team tries to add their school's IT coordinator to the inspection, she's only available by video call from a different building, and she admits she didn't have time to read the document beforehand. That's two challenges at once — a geographically separated AND unprepared reviewer — and Mr. Chen has to reschedule to make sure her feedback actually counts.

🧠 **Remember It Like This:** Big documents, big teams, far-apart people, and unprepared readers — the four things that make reviews go sideways.

PROTOTYPING IN REQUIREMENTS VALIDATION

In Simple Words: Remember all those prototypes from Part 1? They're not just for exploring ideas — they're also powerful tools for CHECKING requirements, because seeing an idea "come alive," even roughly, makes problems way more obvious than reading a sentence about it. **What prototypes do for validation:**

- They make requirements feel "real" instead of just words on a page
- They help find missing requirements before expensive development and testing happens
- They help confirm that stakeholders actually share a common understanding
- Proof-of-concept prototypes specifically demonstrate technical feasibility

Example — Homework Helper App: Reading the sentence "REQ-006: students can set a custom study reminder time" sounds fine on paper. But once Maya's mock-up actually shows the time-picker screen, three students immediately say, "Wait, what if I want a DIFFERENT reminder time for each subject?" — a missing

requirement that the prototype uncovered but the plain sentence never would have.

🧠 **Remember It Like This:** A sentence can hide a problem. A prototype can't — seeing it makes it real.

TESTING THE REQUIREMENTS

In Simple Words: Here's a clever trick: the people DESIGNING the app and the people TESTING the app should both start from the exact same original requirements, then work separately — and their results get compared against each other, like two students solving the same math problem separately and then checking if their answers match. **The flow (Figure 17-5):**

- **User Requirements** splits into two paths
- Analyst path: User Requirements → **Functional Requirements and Analysis Models** → **Technical and UI Designs**
- Tester path: User Requirements → **Test Cases and Scenarios** → **Test Procedures and Scripts**
- At each level, a "compare" arrow checks the two paths against each other: Functional Requirements compared with Test Cases/Scenarios, and Technical/UI Designs compared with Test Procedures/Scripts

🖼️ **What this picture is telling you:** This is Figure 17-5. Analysts and testers work from the SAME starting point but independently — and then their results get compared, so if they disagree, that mismatch reveals a hidden problem in the requirements.

Example — Homework Helper App: From the same REQ-001 (24-hour reminder), Sam designs the notification screen while, completely separately, Priya writes a test case checking that a reminder fires exactly 24 hours before the due date. When they compare notes, Priya realizes Sam's design only checks the due DATE, not the due TIME — a gap neither of them would've caught working alone.

🧠 **Remember It Like This:** Same starting line, two separate racers, then compare notes at the finish — that's how testing catches what one person alone would miss.

ACCEPTANCE CRITERIA

In Simple Words: Acceptance criteria are the final checklist that decides whether Mr. Chen is actually allowed to say "yes, this app is done and ready to use" — like the list of chores that must ALL be finished before you're allowed to say you cleaned your room.

> 📖 **New Word! Acceptance criteria** means the specific conditions that must be true before a customer will agree the product is finished and ready. **The conditions:**

- The most important, high-priority features actually work
- Essential non-functional criteria / quality metrics are satisfied
- Remaining major open issues and defects are resolved
- Any legal, regulatory, or contractual conditions are fully satisfied
- Supporting transition/infrastructure requirements are ready too (like training materials) **A note on acceptance tests:** They mostly focus on the NORMAL flows of use cases and their exceptions — they pay less attention to rarely-used alternative flows.

🖼️ **What this picture is telling you — a real Test Case Template example (Slide 37):**

- **Test Case ID:** Fun_10 I **Designed by:** (name)
- **Priority:** Medium I **Executed by:** (name)
- **Module Name:** Google login screen I **Test Date:** (date)
- **Title:** Verify login with valid username and password I **Execution date:** (date)
- **Description:** Test the Google login page
- **Pre-conditions:** User has a valid username and password
- **Dependencies:** (none)

Step	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Navigate to login page	—	Page loads	—	—
2	Provide valid username	User: example@gmail.com	User should be able to log in	User is navigated to dashboard with successful login	Pass
3	Provide valid password	Password: 1234	—	—	—
4	Click on Login button	—	—	—	—

Post-conditions: User is validated with the database and successfully logged into the account. The session details are logged in the database.

This is a real, complete example from the source material — a filled-in test case for checking a login screen, step by step.

Example — Homework Helper App: Priya writes a matching test case for Homework Helper: **Test Case ID:** Fun_02, **Module:** Reminder Notifications, **Title:** “Verify reminder fires 24 hours before due date,” **Pre-condition:** an assignment exists with a due date set. Step 1: create an assignment due tomorrow at 3:00 PM. Step 2: check the device at today’s 3:00 PM. **Expected Result:** a notification appears. **Actual Result:** notification appeared on time. **Status:** Pass. Only once every high-priority test case like this one passes does Mr. Chen sign off that Homework Helper meets its acceptance criteria.

🧠 **Remember It Like This:** Acceptance criteria are the finish-line checklist — cross every item off, or the race isn’t officially over.

🎯 **Quick Recap:**

- **Validation and Verification** — verification checks you built it right; validation checks you built the right thing.
- **Reviewing Requirements** — catching mistakes early, before they’re expensive.
- **Peer Desk Check** — one reviewer, one careful read.

- **Pass-Around** — several reviewers, each reading independently.
- **Walkthrough** — the author guides the group through the document live.
- **Formal Peer Reviews** — a structured process that ends in a written report.
- **Inspection** — the most powerful, most rigorous kind of formal review.
- **The Inspection Process** — four roles, entry criteria, seven stages, exit criteria.
- **Defect Checklist — Completeness** — did we forget anything?
- **Defect Checklist — Correctness** — is everything written down actually true and clear?
- **Defect Checklist — Quality Attributes** — are speed/reliability/etc. measurable, not just vague wishes?
- **Requirements Review Tips** — plan, start early, give context, limit re-reviews.
- **Requirements Review Challenges** — big documents, big teams, distant or unprepared reviewers.
- **Prototyping in Requirements Validation** — seeing an idea “alive” reveals problems a sentence hides.
- **Testing the Requirements** — analysts and testers work from the same source, then compare results.
- **Acceptance Criteria** — the final checklist before anyone can call the project done.

Part 4: Reusing — Don’t Reinvent the Wheel

WHY REUSE REQUIREMENTS?

In Simple Words: If your older sibling already wrote a great book report last year, it would be silly to start completely from scratch — you’d borrow their good structure and just change the details for your book. Reusing requirements from past projects works the same way.

> 📖 **New Word! Reuse** means using something that already exists (like a well-written requirement) again in a new project, instead of creating it from nothing. **The reasons to reuse:**

- Faster delivery and higher team productivity
- Fewer defects and lower development costs
- Reduced rework and saved review time
- Accelerated approval cycle
- Speeds up other project activities too, like testing
- Improves your ability to estimate, since you have real historical data
- Improves functional consistency across a **software product line** or related business applications

> 📖 **New Word!** A **software product line** means a family of related software products (or app versions) that share a lot of the same design and requirements.

- Saves stakeholders’ time — they don’t have to respecify similar requirements again and again

Example — Homework Helper App: Mr. Chen mentions that a nearby school built a similar homework app two years ago and is willing to share their requirements document. Instead of starting from a blank page, Maya reuses their well-written requirement for “send push notification reminders” and just tweaks the wording for Homework Helper — saving her a full day of writing and getting a requirement that’s already been tested by another real project.

🧠 **Remember It Like This:** Don't rewrite the book report from scratch — borrow the good parts and make them your own.

DIMENSIONS OF REQUIREMENTS REUSE

In Simple Words: When you "reuse" something, you can borrow a little or a lot, change a little or a lot, and grab it in different ways — just like borrowing a recipe: you might just borrow the ingredient list, or the whole recipe PLUS the cooking tips PLUS the photos.

Dimension	What Can Vary
Extent of Reuse	Just one requirement statement, OR a whole set of requirements plus their associated design, code, and test elements
Extent of Modification	The requirement's attributes (priority, rationale, origin, etc.) and related information (test, design, constraints, data definitions, etc.)
Reuse Mechanism	Copy-and-paste from another spec document, OR pull from a library of reusable requirements

Example — Homework Helper App: Maya doesn't just reuse the WORDING of the other school's reminder requirement — she also grabs their test case for it, changes the priority from "Should" to "Must" for Homework Helper, and pulls it via copy-paste since her school doesn't have a fancy requirements library yet.

🧠 **Remember It Like This:** Reuse has three dials — how much you borrow, how much you change, and how you grab it.

REQUIREMENT EVOLUTION

In Simple Words: A single good requirement doesn't just sit still forever — it can get modified over time within its own project, AND get borrowed and modified separately by totally different projects, like a family tree branching out. **Figure 18-1, the timeline:**

- **Project A**: Version 1 → (modify) → Version 2 → (modify) → Version 5
- From Project A's Version 1, an arrow labeled "reuse" branches up to **Project B**: Version 1 → (modify) → Version 3
- From Project A's Version 2, an arrow labeled "reuse" branches down to **Project C**: Version 2 → (modify) → Version 4

🖼️ **What this picture is telling you:** This is Figure 18-1. One original requirement can fork into three completely separate family branches, each getting modified independently in its own project, over time.

Example — Homework Helper App: The "send reminder notification" requirement started as Version 1 at the neighboring school (Project A). Maya's team reuses that Version 1 and evolves it into their own Version 2 for Homework Helper (Project B) — adding the "custom time" feature. Meanwhile, a THIRD school reuses the neighboring school's original Version 1 too, and evolves it their own separate way (Project C). Same starting requirement, three different family trees.

🧠 **Remember It Like This:** One requirement, many family branches — reuse doesn't clone it, it grows separate versions.

REQUIREMENT PATTERNS

In Simple Words: A requirement pattern is like a fill-in-the-blank Mad Libs template for writing a certain TYPE of requirement, so you don't have to figure out the wording from scratch every time.

> 📖 **New Word!** A **requirement pattern** means a reusable template for writing a certain kind of requirement, with blanks to fill in for your specific project. **The five parts of a pattern:**

1. **Guidance** — the kind of situation this pattern applies to
2. **Content** — the details of what can be modified
3. **Template** — the actual fill-in-the-blanks sentence structure
4. **Example** — a sample requirement showing the pattern in use
5. **Consideration for development and testing** — things developers and testers should keep in mind

Example — Homework Helper App: Maya finds a "notification reminder" pattern online with the template: "The system shall send a [notification type] to the [user role] [time amount] before [event]." She fills it in: "The system shall send a **push notification** to the **student 24 hours** before **an assignment due date**." Five minutes, done — instead of wrestling with the wording herself.

🧠 **Remember It Like This:** A requirement pattern is Mad Libs for requirement-writing — just fill in your blanks.

REUSE BARRIERS

In Simple Words: Reusing requirements sounds easy, but real obstacles can get in the way — like wanting to reuse a recipe, but it's written in a different style, uses ingredients your store doesn't sell, or belongs to a restaurant that won't share it.

Barrier	What It Means
Missing or poor requirements	There's nothing good available to reuse in the first place
NIH ("Not Invented Here")	It feels easier to write it yourself than to understand and fix someone else's generic version
NAH ("Not Applicable Here")	The old requirement just doesn't fit this new situation
Writing style differences	Different authors write requirements in very different styles
Inconsistent organization	One document is organized by feature, another by process — hard to match up
Project type	The old requirement is too tightly tied to a specific implementation environment or platform
Ownership	Copyright or intellectual-property concerns over requirements from another system

> 📖 **New Word! NIH ("Not Invented Here")** means resisting someone else's idea just because your team didn't create it. **NAH ("Not Applicable Here")** means an idea genuinely doesn't fit your situation, even if it worked somewhere else.

Example — Homework Helper App: Sam looks at the reused reminder requirement and grumbles, "I could just write this myself faster than reading through their version." Maya gently points out that's classic **NIH** — resisting it just because it wasn't their own idea — and convinces him it really will save time once he gives it a real look.

🧠 **Remember It Like This:** Reuse fails when pride (NIH), mismatch (NAH), or messy differences get in the way.

REUSE SUCCESS FACTORS

In Simple Words: To make reuse actually work well, a team needs a good "library" to borrow from, a way to know how pieces connect, shared vocabulary, and a culture that actually encourages borrowing instead of looking down on it. **The success factors:**

- **Repository** (a place to store reusable requirements) — three forms:
  1. A single shared network folder of previous requirements documents
  2. A collection stored in a requirements management tool, searchable across projects
  3. A database of hand-picked reusable requirement sets, tagged with keywords like origin, suitability, and limitations
- **Interactions** — using traceability links in a tool to find what a reusable requirement connects to

> 📖 **New Word! Traceability** means being able to follow the connections between a requirement and everything linked to it, like its design, code, and tests.

- **Terminology** — everyone agreeing on common words and definitions
- **Organizational culture** — a team culture that actively encourages and rewards reuse

Example — Homework Helper App: After Homework Helper launches, Mr. Chen suggests the school start a simple shared folder ("Repository" option 1) of all their app requirements documents, so the NEXT student team — maybe building a "Cafeteria Menu" app next year — won't have to start from zero either.

🧠 **Remember It Like This:** Good reuse needs a library, a map of connections, shared words, and a culture that says "yes, borrow it!"

🎯 **Quick Recap:**

- **Why Reuse Requirements?** — faster, cheaper, and more consistent than starting from scratch.
- **Dimensions of Requirements Reuse** — how much you borrow, how much you change, and how you grab it.
- **Requirement Evolution** — one requirement can branch into many separately evolving versions.
- **Requirement Patterns** — fill-in-the-blank templates for common requirement types.
- **Reuse Barriers** — pride, mismatch, style differences, and ownership concerns can block reuse.
- **Reuse Success Factors** — a good repository, clear connections, shared terms, and a reuse-friendly culture.

Part 5: Using Requirements — Where All This Work Goes

USE OF REQUIREMENTS

In Simple Words: Once requirements are finished and locked in (baselined), they don't just sit in a drawer — they become the master plan that steers THREE big parts of the project at once: the schedule, the actual building, and the testing. **Figure 19-1 — from Baselined Requirements, three branches:**

1. **Project Plans** — size the project or iteration using the requirements; base estimates on the size of the product; update plans as requirements change; use requirement priorities to drive iterations
2. **Designs & Code** — developers review the requirements; use quality attributes to drive the architecture; allocate requirements to specific components; trace requirements to designs and code
3. **Tests** — start test design early; have users create acceptance tests; base system testing on requirements; trace requirements to tests

🖼️ **What this picture is telling you:** This is Figure 19-1. A finished, baselined requirements document isn't the END of the work — it's the STARTING POINT that steers planning, building, and testing all at the same time, like a recipe that three different cooks all follow at once.

Example — Homework Helper App: Once Maya's requirements document is baselined and signed off by Mr. Chen, all three branches kick in together: Maya uses the requirement list to plan which features go in Sprint 1 vs. Sprint 2 (**Project Plans**); Sam reviews every requirement and starts writing code, tracing each

function back to its requirement ID like REQ-001 (**Designs & Code**); and Priya writes her acceptance test cases straight from the same requirements, months before the app is even close to finished (**Tests**). Three different jobs, one shared source of truth.

🧠 **Remember It Like This:** Baselined requirements are a recipe three cooks share at once — the planner, the builder, and the tester all read the same page.

🎯 **Quick Recap:**

- **Use of Requirements** — baselined requirements steer Project Plans, Designs & Code, and Tests, all from the same source.

🏆 Chapter Wrap-Up

By now, Maya's team has come a long, long way from that first messy pile of ideas. They sketched paper prototypes and clickable mock-ups to make sure everyone imagined Homework Helper the same way. They prioritized ruthlessly with MoSCoW and the $100 game so Sam wouldn't waste ten weeks building the wrong things first. They inspected every requirement like detectives, hunting down missing pieces, contradictions, and vague quality goals before a single line of code got written. They reused a neighboring school's great ideas instead of reinventing the wheel. And now, with the requirements officially baselined, that one careful document is quietly steering Maya's plans, Sam's code, and Priya's tests — all at the exact same time. Homework Helper isn't just an idea anymore. It's ready to be built.

If you remember ONLY these things…

- **Prototyping** lets you test an idea cheaply — on paper or in rough working code — before you build the real thing for real.
- **Prioritizing** means picking what matters most first, because you never have time or money for everything at once.
- **Validating** means checking your requirements are correct, complete, and actually solve the real problem — before it's too expensive to fix.
- **Reusing** means borrowing good ideas from past projects instead of starting from a blank page every single time.
- **Using requirements** means the final, locked-in document becomes the shared map that guides planning, building, and testing, all the way to launch.

Chapter 6: Requirements For Specific Project Classes

Maya, Sam, Priya, and Mr. Chen have gotten really good at figuring out what Homework Helper needs to do. But here's a secret every real software team learns eventually: **not every project works the same way.** Sometimes you buy software instead of building it. Sometimes you hire outside help. Sometimes you're drowning in giant piles of data, or building something that has to react in a split second. This chapter takes the Homework Helper crew (plus a couple of fun bonus adventures) on a tour through these special situations.

📦 Packaged Solution Projects (COTS & SaaS)

Before Sam writes a single line of code, Maya wonders: *"Wait — does something like this already exist? Could we just buy it instead of building it?"* That question opens up a whole new set of ideas.

WHAT IS A PACKAGED SOLUTION (COTS)?

In Simple Words: Sometimes, instead of baking a birthday cake from scratch, you buy one from the bakery and just add your own decorations. Buying ready-made software and adjusting it is the same idea.

> 📖 **New Word!** COTS means "Commercial Off-The-Shelf" — software that a company already built and sells, ready for you to buy and adapt, instead of building your own from zero.

Example — Homework Helper App: Maya says to Sam, "Before we spend three months building this ourselves, let's check if a school somewhere already sells something like it." She finds an existing product called "StudyTrackr" that other schools already use.

🧠 **Remember It Like This:** Why bake it from zero when you can buy the mix and just add sprinkles?

SOFTWARE AS A SERVICE (SAAS) & CLOUD SOLUTIONS

In Simple Words: Renting a bike is different from buying one — you pay a little every month, someone else keeps it in good shape, and you never have to store it yourself. Software can work the same way.

> 📖 **New Word!** SaaS (Software as a Service) means you don't install the software on your own computer — you pay to use it online, and it lives on the internet ("in the cloud").

Example — Homework Helper App: StudyTrackr turns out to be a SaaS product. It costs $2 per student each month, nothing gets installed on school computers, and students just log in through a web browser.

🧠 **Remember It Like This:** SaaS means you rent the software, and the cloud keeps it running while you sleep.

WHY REQUIREMENTS STILL MATTER FOR PACKAGED SOLUTIONS

In Simple Words: Even when you're buying a toy instead of building one, you still need a wish list — one to help you pick the best toy, and one to help you set it up right.

Example — Homework Helper App: Maya still writes requirements like "REQ-101: Students must be able to see assignments due within the next 7 days," even though they might buy StudyTrackr. She uses this list both to judge which app is best AND to set it up correctly afterward.

🧠 **Remember It Like This:** Buying doesn't mean skipping the wish list — the wish list is what picks the right toy AND makes it fit.

CONFIGURING, INTEGRATING & EXTENDING A PACKAGE

In Simple Words: Think of a LEGO set. You can build it exactly as shown, tweak it with the instructions, snap on pieces from another set, or design brand-new custom pieces. Packaged software can be adjusted in similar ways.

Example — Homework Helper App: Sam finds that StudyTrackr can be **configured** with the school's colors and class names, **integrated** with the school's login system, but can't send text-message reminders — so they'd have to **extend** it with custom code to add that.

🧠 **Remember It Like This:** Configure it, connect it, or create new pieces for it — that's how you make a package truly fit.

REQUIREMENTS FOR SELECTING A PACKAGED SOLUTION

In Simple Words: Before choosing a team for dodgeball, you compare your options against what you actually need. Picking software works the same way — you build a comparison list first.

Example — Homework Helper App: Maya sets up a comparison spreadsheet with three possible apps, scoring each one on what students need, what school rules require, and how well-built each app is.

🧠 **Remember It Like This:** Before you say yes, make the list, then check the box.

Developing User Requirements (for Selection)

In Simple Words: You have to know what people actually need *before* you can judge whether a product gives it to them.

Example — Homework Helper App: Maya interviews five students first. She learns most of them want a push notification the night before a big project is due — that becomes a must-have on her checklist.

🧠 **Remember It Like This:** Know what you need before you go shopping for it.

Considering Business Rules

In Simple Words: Every school (or business) already has rules it follows. New software has to obey those rules too, not invent its own.

> 📖 **New Word!** A business rule is a rule an organization already follows that the software must respect, like “no homework may be assigned over the weekend.”

Example — Homework Helper App: The school has a rule: no homework for grades 6–7 over weekends. Maya adds a requirement that any app they pick must let teachers mark certain days as “no homework allowed.”

🧠 **Remember It Like This:** The app has to follow the school’s rules — the school doesn’t follow the app’s rules.

Defining Quality Requirements

In Simple Words: It’s not enough for software to just “work” — it also needs to be fast, easy, adaptable, friendly with other systems, trustworthy, and safe. Those six qualities are like a report card for the app itself.

> 📖 **New Word!** A quality requirement describes *how well* something must work, not just *what* it must do.

Example — Homework Helper App: Maya builds this mini report card for comparing apps:

Quality	What It Means (Kid Words)	Homework Helper Example
Performance	How fast it works	The assignment list must load in under 2 seconds
Usability	How easy it is to use	A 6th grader must be able to add homework without any help
Modifiability	How easy it is to change later	Teachers can add a new class without calling tech support
Interoperability	How well it works with other systems	It must connect to the school’s Google Classroom accounts
Integrity	How well it keeps data correct and whole	Marking one assignment “done” must never erase another one
Security	How well it protects information	Only a student’s own family can see their grades

🧠 **Remember It Like This:** Good software should be fast, easy, flexible, friendly with others, honest with your data, and locked up tight.

Evaluating Solutions

In Simple Words: After you’ve gathered all the information, it’s like judging a talent show — score each act against your checklist to pick a winner.

Example — Homework Helper App: Maya scores three candidate apps out of 10 points on her spreadsheet. StudyTrackr wins overall, but loses points because its reminder feature is hidden and hard to find.

🧠 **Remember It Like This:** Score every option like a judge — then let the numbers help you choose.

THE SPECTRUM OF IMPLEMENTATION EFFORT

In Simple Words: Ordering pizza can mean grabbing a plain one as-is, adding a few toppings, or asking the chef to build a totally custom pizza just for your party. The more custom you go, the more work it takes.

🖼️ **Figure 22-3 — A spectrum of implementation effort for packaged solutions:**

`Out-of-the-box` → `Configured` → `Integrated` → `Extended`

🖼️ *What this picture is telling you: the further right you slide, the more extra work and coding it takes to make a package do what you need.*

Table 22-1 — COTS package implementation approaches:

Type of COTS Implementation	Description
Out-of-the-box	Install the software and use it as is.
Configured	Adjust settings in the software to suit your needs without writing new code.
Integrated	Connect the package to existing systems in your application ecosystem; usually requires some custom code.
Extended	Develop additional functionality with custom code to enhance the package's capabilities to close needs gaps.

🖼️ *What this picture is telling you: each row is a bigger step up in effort — from "just install it" all the way to "build new features onto it."*

Example — Homework Helper App: Sam figures out StudyTrackr needs the "Integrated" level — they must connect it to the school's login system with a bit of custom code, but they don't need to build brand-new features.

🧠 **Remember It Like This:** Out-of-the-box is the easiest slice; Extended is a whole custom pizza — know where on the spectrum you're standing.

COMMON CHALLENGES WITH PACKAGED SOLUTIONS

In Simple Words: Buying software sounds like the easy way out, but it comes with its own set of bumps in the road.

Example — Homework Helper App: Before committing to any purchase, Maya makes a "watch out for these" list, so the team doesn't get surprised later.

🧠 **Remember It Like This:** Buying software sounds easy — until you hit these five bumps.

Too Many Candidates

In Simple Words: Imagine 30 different party games to pick from, and every single box claims to be "the most fun game ever!" — so many choices, all looking good at first glance, that it's hard to even start deciding.

Example — Homework Helper App: Maya's search turns up 12 different homework-tracking apps, and at a glance almost every one of them claims it can do what Homework Helper needs. The team feels stuck just trying to make a shortlist.

🧠 **Remember It Like This:** Too many choices — all looking good on the surface — can freeze you like a game of statues.

Too Many Evaluation Criteria

In Simple Words: Judging a pizza on 50 different things — crust thickness, box color, cheese brand — is way harder than judging it on taste, price, and speed.

Example — Homework Helper App: Sam wants to compare apps on 40 different features. Maya narrows it down to just the 5 that actually matter for the school's real goals — reminders, ease of use, price, security, and phone compatibility — instead of digging into a deep, feature-by-feature study of every single app.

🧠 **Remember It Like This:** Pick your top few must-haves based on what you actually need — don't drown in a checklist ocean.

Vendor Misrepresents Package Capabilities

In Simple Words: Sometimes the person selling you something doesn't fully understand what it can and can't do.

> 📖 **New Word!** A vendor is the company (or person) selling you a product or service.

Example — Homework Helper App: A sales rep tells Maya, "Our app absolutely does text-message reminders!" But it turns out it only sends emails — the rep just didn't know the real feature list.

🧠 **Remember It Like This:** Trust, but verify — even a friendly salesperson can be wrong.

Incorrect Solution Expectations

In Simple Words: A toy commercial can make a toy look amazing, but the real toy in the box is smaller and simpler.

Example — Homework Helper App: The StudyTrackr demo looked perfect and polished. But after installing it for real, teachers discovered the reminder button was buried three menus deep.

🧠 **Remember It Like This:** Demos are movie trailers — always watch the whole movie before you clap.

Users Reject the Solution

In Simple Words: Buying a video game nobody in your class actually wants to play means it just sits on the shelf, unused.

Example — Homework Helper App: Even though Mr. Chen approves StudyTrackr, students say it feels clunky and refuse to use it. The team decides to keep gathering user feedback before finalizing anything.

🧠 **Remember It Like This:** Buying it isn't the finish line — people actually using it is.

🎯 **Quick Recap:**

- **COTS** = buying ready-made software instead of building from scratch
- **SaaS/Cloud** = renting software online instead of installing it
- **Why requirements still matter** = you still need a wish list to pick AND set up a package
- **Configuring/Integrating/Extending** = three ways to adjust a package to fit your needs
- **Selecting a package** = compare candidates using user requirements, business rules, quality requirements, and evaluation scores
- **Quality requirements** = Performance, Usability, Modifiability, Interoperability, Integrity, Security
- **Spectrum of implementation effort** = Out-of-the-box → Configured → Integrated → Extended
- **Common challenges** = too many candidates, too many criteria, vendor mistakes, wrong expectations, and users refusing to use it

🤝 Outsourced Projects

Now imagine the whole school district wants Homework Helper for every school — that's too big a job for Sam alone. Mr. Chen decides to hire an outside company, "BrightCode Studios," to help build the bigger version.

APPROPRIATE LEVELS OF REQUIREMENTS DETAIL

In Simple Words: Giving directions to a friend who already knows your neighborhood is easy. Giving directions to a stranger from another town needs way more detail.

> 📖 **New Word!** Outsourcing means hiring an outside company to do work instead of doing it with your own team.

Example — Homework Helper App: Maya writes much more detailed requirements for BrightCode Studios than she ever needed for Sam, because BrightCode has never set foot in their school and doesn't know how things work there.

🧠 **Remember It Like This:** Strangers need more directions than friends do.

ACQUIRER–SUPPLIER INTERACTIONS

In Simple Words: Ordering a custom cake from a bakery means going back and forth — you explain what you want, they ask questions, and eventually they hand over the finished cake.

> 📖 **New Word!** The acquirer is the person or team who wants the work done (and pays for it); the supplier is the company doing the work.

🖼️ **Figure 23-1 — Requirements are the cornerstone of an outsourced project:**

1. **Acquirer** sends → *"Request for proposal, requirements, acceptance criteria"* → **Supplier**
2. **Acquirer** ↔ *"Review, negotiate, agree"* ↔ **Supplier**
3. **Supplier** delivers → *"Software, documentation"* → **Acquirer**

🖼️ *What this picture is telling you: an outsourced project is a back-and-forth conversation, not a one-time order — both sides review and agree before anything gets delivered.*

Example — Homework Helper App: Maya's team (the acquirer) sends BrightCode (the supplier) their requirements and a list of how they'll know the work is "done" (acceptance criteria). They go back and forth reviewing details before BrightCode builds anything, and BrightCode finally delivers the finished app plus a user guide.

🧠 **Remember It Like This:** Ask, agree, then deliver — that's the outsourcing dance.

PLAN TIME FOR MULTIPLE REVIEW CYCLES

In Simple Words: You wouldn't proofread an important essay just once — you'd check it, fix it, and check it again.

Example — Homework Helper App: Maya schedules three separate requirement review meetings with BrightCode between March 1st and March 20th, instead of cramming everything into one rushed meeting.

🧠 **Remember It Like This:** One look is never enough — plan for round two AND round three.

HOLD REQUIREMENTS WORKSHOPS IMMEDIATELY FOLLOWED BY IMPLEMENTATION TASKS

In Simple Words: It's easiest to build something right after you've just explained it clearly, while everyone still remembers exactly what was said.

> 📖 **New Word!** A requirements workshop is a focused meeting where a team works out exactly what's needed for one part of a project.

Example — Homework Helper App: On Monday, Maya runs a two-hour workshop about the "study reminder" feature, and on a separate track that same week, another workshop covers the "grade sync" feature. BrightCode's developers start building each exact feature the very next day, while the details from each workshop are still fresh in everyone's mind — instead of waiting weeks and having to re-explain everything.

🧠 **Remember It Like This:** Strike while the whiteboard's still warm.

PEER REVIEWS AND PROTOTYPES REVEAL SUPPLIER INTERPRETATION

In Simple Words: A quick rough sketch shows you whether someone understood your idea correctly — long before they've spent weeks building the real thing.

Example — Homework Helper App: BrightCode sends Maya a clickable prototype of the reminder screen. Priya reviews it and realizes they misunderstood "urgent" to mean the color red, when the team actually meant a ringing bell icon. They catch the mix-up early, before it's built for real.

🧠 **Remember It Like This:** A rough sketch now saves a rebuild later.

RISK: CONTRACTORS LACKING DOMAIN KNOWLEDGE

In Simple Words: An outside company might be brilliant at coding but still be total strangers to how your school actually works day to day.

> 📖 **New Word!** Domain knowledge means understanding the specific world a project lives in — like knowing how a school, hospital, or store really operates.

Example — Homework Helper App: BrightCode has built apps for hospitals and stores before, but never for a middle school. They assume students check email regularly — but Maya has to explain that students almost never check email, and only look at text alerts and app notifications.

🧠 **Remember It Like This:** An outside team may be expert coders but rookie hallway-walkers — spell out what only insiders know.

🎯 **Quick Recap:**

- **Requirements detail level** = strangers (outside companies) need much more detail than your own team
- **Acquirer–supplier interactions** = request, review/negotiate, then deliver — a back-and-forth cycle
- **Multiple review cycles** = plan more than one round of checking requirements
- **Workshops right before building** = explain it, then build it while it's fresh
- **Peer reviews and prototypes** = catch misunderstandings early with a quick sketch or model
- **Domain knowledge risk** = outside teams may not know your specific world unless you tell them

🔄 Business Process Automation Projects

Before Homework Helper existed, how did students actually track their homework? Mostly paper planners — that got lost, forgotten, or left in lockers. This whole section is about studying and improving *how things are done*, which is exactly what Maya had to do before designing the app.

> 📖 **New Word!** A business process is a series of steps an organization follows to get something done — like the steps for tracking homework, from "assignment given" to "assignment turned in."

BUSINESS PROCESS ANALYSIS (BPA)

In Simple Words: Before you can fix a recipe, you have to watch someone cook it first, step by step, and notice where it goes wrong.

Example — Homework Helper App: Maya watches how students currently track homework: write it in a paper planner → planner gets lost in a backpack → student forgets the assignment. She writes down every single step before designing anything new. (This is exactly like a pizza delivery company studying its own

process to figure out how to make deliveries faster and cheaper — understanding the process first is what makes improving it possible.)

🧠 **Remember It Like This:** Watch it first, fix it second.

BUSINESS PROCESS REENGINEERING (BPR)

In Simple Words: Sometimes you don't just oil a squeaky bike chain — you throw out the whole bike and design a brand-new go-kart instead.

Example — Homework Helper App: Instead of just improving the paper planner, the team throws the old system out completely and redesigns the entire "track homework" process around a phone app with automatic reminders.

🧠 **Remember It Like This:** Sometimes you don't fix the old wheel — you build a whole new cart.

BUSINESS PROCESS IMPROVEMENT (BPI)

In Simple Words: Instead of buying a new bike, you just tighten a loose screw here and oil a squeaky chain there — small steady tweaks that keep something already-good running even better.

Example — Homework Helper App: After Homework Helper launches, Priya notices reminders arrive at 11:00 PM — way too late. She suggests changing it to 6:00 PM instead. That's a small, measured improvement, not a full redesign.

🧠 **Remember It Like This:** Little fixes, again and again, keep things running smooth.

BUSINESS PROCESS MANAGEMENT (BPM)

In Simple Words: A coach doesn't just watch one player — they manage the whole team's plays and keep improving all of them together, over time.

Example — Homework Helper App: Mr. Chen thinks about every process at the school — grading, attendance, AND homework tracking — and works with Maya's team plus other staff to analyze each one for how well it's working and keep improving it, not just the app.

🧠 **Remember It Like This:** Managing every process, not just one — that's the big-picture job.

BUSINESS PROCESS MODEL AND NOTATION (BPMN)

In Simple Words: A comic strip uses special panels and speech bubbles to tell a story clearly. BPMN uses special shapes and arrows to tell the story of a process clearly.

> 📖 **New Word!** Notation means a set of symbols and rules used to draw or write something in a standard way that everyone can understand.

Example — Homework Helper App: Maya draws a BPMN diagram with boxes and arrows: `[Assignment Given]` → `[Student Opens App]` → `[Reminder Sent]` → `[Homework Marked Done]` — so everyone on the team pictures the process the exact same way.

🧠 **Remember It Like This:** BPMN is a process's own comic strip — shapes and arrows telling the story.

🎯 **Quick Recap:**

- **BPA** = studying how a process works today
- **BPR** = completely redesigning a process for something way better
- **BPI** = making small, steady improvements to a process
- **BPM** = managing and improving ALL of an organization's processes over time
- **BPMN** = a drawing system (shapes + arrows) for showing a process step by step

📊 Big Data Projects

A small school app like Homework Helper doesn't deal with *big* data — but let's imagine it got really, really popular. This section leans on a 🌟 Bonus Example: **"Homework Helper Goes Nationwide"** — picture the same app used by every school in the whole country!

WHAT IS BIG DATA?

In Simple Words: One paper cup of water is easy to carry. An entire ocean is a totally different challenge — so much, arriving so fast, in so many different shapes.

> 📖 **New Word!** Big data means information that's characterized by huge volume (a LOT of it), high velocity (arriving FAST), and/or high complexity (many DIFFERENT kinds mixed together).

🌟 **Bonus Example:** Imagine all 5,000 schools in the country used Homework Helper. Every second, thousands of students would check assignments, submit homework, and send messages. That flood is "big data": **volume** = millions of homework entries, **velocity** = new updates every second, **complexity** = text, photos of homework, and voice memos all mixed together.

🧠 **Remember It Like This:** Big data means lots, fast, and messy — like a school hallway at the bell, times a million.

MANAGING BIG DATA

In Simple Words: Organizing a giant, messy toy box means you need a system to find, gather, sort, and use each toy quickly — not just dump it all on the floor. 🌟 **Bonus Example:** If Homework Helper had millions of nationwide users, engineers would need special big computer systems just to discover, collect, store, and process all that data quickly and effectively — way beyond what fits on Sam's laptop.

🧠 **Remember It Like This:** Big data needs big organizing — find it, keep it, use it, fast.

MODELING RELATED DATA WITH ERDS

In Simple Words: If your data pieces are "best friends" with each other (like a student and their assignments), you can draw a map showing exactly how they're connected.

> 📖 **New Word!** An Entity-Relationship Diagram (ERD) is a picture-map showing how different pieces of data relate to each other.

Example — Homework Helper App: Maya, acting as the Business Analyst (BA), draws an ERD showing that one "Student" connects to many "Assignments," and one "Class" connects to many "Students."

🧠 **Remember It Like This:** An ERD is a map of how your data pieces are best friends with each other.

EXAMPLES OF BIG-DATA-GENERATING ORGANIZATIONS

In Simple Words: Some companies are so enormous that no filing cabinet in the world could ever hold their information — they need massive, specialized systems just to keep up. 🌟 **Bonus Example:** Companies like Grameenphone (GP), Facebook, YouTube, and Google handle billions of records every single day. Maya researches how they manage that scale, just to understand what "really big" looks like compared to their small Homework Helper database.

🧠 **Remember It Like This:** When millions of people use something every day, the data becomes a mountain, not a molehill.

DATA VS. INFORMATION

In Simple Words: Flour, eggs, and sugar are raw ingredients. A finished, decorated cake is something you can actually understand and enjoy. Data is the ingredients; information is the cake.

Example — Homework Helper App: Raw data: `37, 42, 19, 88` — just numbers. Information: "88% of 6th graders turned in their math homework this week" — now it actually means something to Mr. Chen.

🧠 **Remember It Like This:** Data is the pile of puzzle pieces. Information is the finished picture.

THE SIMPLE BUSINESS ANALYTICS FRAMEWORK

In Simple Words: Three friends standing in a triangle, each one always passing something to the next, around and around forever.

🖼️ **Figure 25-1 — The components of a simple business analytics framework:**

- **Data** (Source, Storage, Management, Governance, Extraction) → feeds →
- **Analysis** (Transformations, Computations) → feeds →
- **Information Usage** (Delivery mechanism, Format, Adaptability) → feeds back into →
- **Data** again — and the cycle repeats

🖼️ *What this picture is telling you: raw data gets analyzed into something useful, people use that information, and what they learn sends the team back to collect even better data — round and round forever.*

🌟 **Bonus Example:** In the nationwide Homework Helper, raw data (millions of daily check-ins) flows into **Analysis** (crunching the numbers), which becomes **Information Usage** (a chart Mr. Chen's district boss looks at), and lessons learned from that usage send the team back to collect new, better data. The circle keeps spinning.

🧠 **Remember It Like This:** Data, Analysis, and Usage chase each other in a circle forever.

INFORMATION USAGE BY PEOPLE

In Simple Words: Having great information isn't enough — you also have to decide exactly how real people will actually receive it and use it. 🌟 **Bonus Example:** In the nationwide version, Maya realizes it's not enough to just calculate the numbers — she has to decide how principals, teachers, and parents will actually see and use them.

🧠 **Remember It Like This:** Great information is useless if nobody can actually get it or use it.

Delivery Mechanism

In Simple Words: A birthday invitation can arrive by text, by mail, or in person — the message is the same, but HOW it reaches you is a totally separate choice. 🌟 **Bonus Example:** Should nationwide reports reach principals by email, a website dashboard, or a mobile app alert? Maya decides on an email summary plus an in-app dashboard.

🧠 **Remember It Like This:** It's not just what you say — it's how it gets delivered to the door.

Format

In Simple Words: The same information can wear different outfits — a full written report, a colorful dashboard, or plain raw numbers. 🌟 **Bonus Example:** Teachers get a colorful dashboard with bars showing the percentage of homework completed; the tech team gets the same information as a plain spreadsheet instead.

🧠 **Remember It Like This:** Same information, different outfits, depending on who's looking.

Flexibility

In Simple Words: Some people are happy with a sundae made exactly the way the shop serves it. Others want a build-your-own sundae bar where they pick every topping themselves.

> 📖 **New Word!** Flexibility (or adaptability) means how much a person can rearrange or dig into information themselves after they receive it.

🌟 **Bonus Example:** Mr. Chen wants to filter the district dashboard by grade level himself. Maya makes sure the report lets him click and adjust it, instead of showing only one fixed view.

🧠 **Remember It Like This:** Some people want it served exactly as-is; others want to mix it themselves.

BIG DATA STRUCTURE

In Simple Words: Not all data lines up neatly in rows and columns like a spreadsheet — some of it is a wild, tangled mess. Big data is often one of these two messier kinds. 🌟 **Bonus Example:** As Homework Helper scales up, Maya learns that some incoming information (like parent voicemails) is nothing like a tidy spreadsheet.

🧠 **Remember It Like This:** Some data lines up in rows. Some data is a wild, tangled mess.

Unstructured Data

In Simple Words: A junk drawer has everything jammed inside with no order — good luck finding the tape when you need it.

> 📖 **New Word!** Unstructured data is information that doesn't fit neatly into rows and columns, like voicemails or text messages, making it hard to know where or how to even begin searching for what you need.

🌟 **Bonus Example:** Voicemails parents leave the school office, or free-form text messages between students about homework, don't fit into neat rows and columns at all.

🧠 **Remember It Like This:** Unstructured data is a junk drawer — everything's in there, but good luck finding the tape.

Semi-Structured Data

In Simple Words: Even a messy junk drawer is easier to search if it at least has labeled sections — some order hiding inside the chaos.

> 📖 **New Word!** Semi-structured data has some organizing metadata (like Subject, To, Content, Attachment) attached to it, even though the main content is still free-form.

🌟 **Bonus Example:** An email to Mr. Chen has clear labeled parts — Subject, To, Content, Attachment — even though the message body itself is free-form text. That structure (metadata) helps Maya build an ERD and a data dictionary around it.

🧠 **Remember It Like This:** Semi-structured data has labels on the junk drawer, even if the stuff inside is still loose.

DATA-BASED REQUIREMENTS

In Simple Words: Before building anything with data, you have to ask four big kinds of questions — like a detective gathering clues before writing a report. 🌟 **Bonus Example:** Before designing the nationwide version, the team sits down and works through four categories of data questions, one at a time.

🧠 **Remember It Like This:** Four big data questions to answer before you build anything.

Data Sources

In Simple Words: Before cooking, you need to know exactly which grocery store each ingredient comes from, whether that store might change its recipe someday, and whether you also need to bring in old ingredients from the pantry. 🌟 **Bonus Example:** Maya asks: exactly which pieces of information do we need (student name, class, due date…), and do we pull them straight from Google Classroom (an existing system), or type them in fresh? She also checks how likely Google Classroom's data is to change over time, and whether old paper-planner records from past years need to be migrated into the new system too. This is just like how a phone company must pull identity data from the National ID (NID) database during SIM registration, instead of asking the customer to re-type everything.

🧠 **Remember It Like This:** Know your ingredients' grocery store before you start cooking — and check if you're also bringing in leftovers from before.

Data Storage

In Simple Words: A growing collection of anything — trading cards, photos, homework records — needs a bigger, safer shelf every single year, and you need to know exactly what kinds of things you're storing. 🌟 **Bonus Example:** How much homework data already exists? What kinds of data are they storing — text assignments, photos, voice memos? How fast will it grow if 5,000 schools join? How long must old records be kept, and how securely?

🧠 **Remember It Like This:** A growing pile of data needs a bigger and safer shelf every year.

Data Management & Governance

In Simple Words: Every collection needs a rulebook — who's allowed to touch it, how it's kept safe from harm, and what happens to old, unused pieces.

> 📖 **New Word!** Governance means the rules for managing something responsibly — including who can access it, how it's protected from being lost or corrupted, and when it should be deleted or archived.

🌟 **Bonus Example:** What happens to a student's data after they graduate — deleted or archived? Maya writes a rule: "Student records are archived (not deleted) one year after graduation, only school staff can access them, and backups are kept so nothing is ever accidentally lost or corrupted."

🧠 **Remember It Like This:** Governance is the rulebook for who touches the data, how it's kept safe, and when it gets thrown away.

Data Extraction

In Simple Words: Sometimes you need an answer right this second; other times, waiting until tomorrow morning is perfectly fine.

> 📖 **New Word!** Data extraction is about how (and how fast) information gets pulled out for people to use — instantly (real-time) or all at once later (batch).

🌟 **Bonus Example:** How fast do teachers expect their dashboard to update — instantly, or once a night? This is just like a payroll system that only recalculates paychecks once a month, or a billing system that mails out statements once a month, instead of processing everything the instant it happens.

🧠 **Remember It Like This:** Extraction asks: do you need the answer right now, or is overnight okay?

DEFINING ANALYSES THAT TRANSFORM THE DATA

In Simple Words: A detective asks about the past, a doctor checks your symptoms right now, and a fortune teller guesses the future — three totally different kinds of questions about the same topic.

🌟 **Bonus Example — the three timeframes Maya explores for the nationwide dashboard:**

- **Past** — "What happened?" → *How many students missed homework last semester?*

- **Present** — "What's happening right now?" → *Which students are missing homework today, so a teacher can check on them?*
- **Future** — "What might happen?" → *Which students are likely to fall behind next month, so we can send extra reminders early?*

🖼️ *What this picture is telling you: the same data can answer very different questions depending on whether you're looking backward, at today, or ahead.*

🧠 **Remember It Like This:** Ask about yesterday, today, and tomorrow — each needs a different kind of answer.

🎯 **Quick Recap:**

- **Big data** = huge volume, high velocity, and/or high complexity of information
- **Managing big data** = discovering, collecting, storing, and processing it quickly
- **ERDs** = maps showing how data pieces relate to each other
- **Big-data organizations** = companies like Grameenphone, Facebook, YouTube, Google
- **Data vs. Information** = raw ingredients vs. the finished, understandable cake
- **Business analytics framework** = Data → Analysis → Information Usage → back to Data
- **Information usage by people** = Delivery mechanism, Format, and Flexibility all matter
- **Big data structure** = unstructured data (no order) vs. semi-structured data (some labels)
- **Data-based requirements** = questions about Sources, Storage, Management & Governance, and Extraction
- **Defining analyses** = ask about the past, the present, and the future

🤖 Embedded and Real-Time Systems Projects

Homework Helper is fine if a reminder arrives a few seconds late — nobody falls down the stairs over it! But some systems must react in a split second, every single time. For this section, meet our 🌟 Bonus Example: **Rosie**, a smart robot vacuum.

SYSTEM REQUIREMENTS, ARCHITECTURE, AND ALLOCATION

In Simple Words: Think of a treehouse blueprint. It shows the overall plan, which parts are made of wood, which parts are metal, and which jobs a person (not the treehouse) has to do. 🌟 **Bonus Example:** Rosie the robot vacuum has overall "system requirements" — like "clean the whole floor safely." Those get broken down into decisions about which parts are software, which are hardware, and which jobs need an actual person.

🧠 **Remember It Like This:** Big requirement first, then figure out who or what handles which piece.

A SYSTEM'S ARCHITECTURE HAS THREE ELEMENTS

In Simple Words: Any built thing — a treehouse, a robot, an app — is made of parts, what those parts show the world, and how the parts talk to each other. 🌟 **Bonus Example:** Designing Rosie means listing what she's made of, what each part promises to do, and how the parts connect and communicate.

🧠 **Remember It Like This:** Every system is built from parts, what they show the world, and how they connect.

Components

In Simple Words: A "component" isn't just a gadget — it can be a piece of software, a physical device, or even a person doing a job.

> 📖 **New Word!** A component is a piece of a system — it can be a software module, a physical device, or a person.

🌟 **Bonus Example:** Rosie has a software module (the "brain" program), a physical device (a dust sensor), and even a person (the owner who empties the dustbin) — all three count as components.

🧠 **Remember It Like This:** A component can be code, a gadget, OR a person — anything doing a job in the system.

Externally Visible Properties

In Simple Words: You don't need to see how a toaster works inside — you just need to know it promises to make toast when you push the lever.

> 📖 **New Word!** An externally visible property is what a component promises to do, without anyone needing to know how it works inside.

🌟 **Bonus Example:** Rosie's bump sensor has a visible property: "detects an obstacle within 2 cm." Other parts of Rosie's system don't need to know how the sensor works inside — just that it reliably does this.

🧠 **Remember It Like This:** You don't need to see the engine — just know what the part promises to do.

Connections

In Simple Words: Wires (or messages) let different parts of a system talk to each other, just like a walkie-talkie lets two people communicate.

> 📖 **New Word!** A connection is the link that lets one component send information (or signals) to another.

🌟 **Bonus Example:** Rosie's brain module sends a "stop" signal to the wheel motor the instant the bump sensor fires — that link between the two parts is a connection.

🧠 **Remember It Like This:** Connections are the wires (or messages) letting parts talk to each other.

SYSTEM REQUIREMENTS DECOMPOSITION & ALLOCATION

In Simple Words: One big requirement can be split into smaller pieces, and then each piece gets handed to exactly the right part of the system — software, hardware, or a person.

🖼️ **Figure 26-1 — System requirements decomposed and allocated:**

- **System Requirements** —(decomposition and derivation)→ splits into three branches:
  - **Software Requirements** —(allocation)→ Component A, Component B
  - **Manual Requirements** —(allocation)→ People
  - **Hardware Requirements** —(allocation)→ Component C, Component D

🖼️ *What this picture is telling you: a big system requirement breaks into three kinds of smaller requirements, and each one gets assigned to the exact part (or person) responsible for it.*

🌟 **Bonus Example:** Rosie's big requirement "must clean the whole floor without falling down stairs" breaks into: a **software requirement** (a stair-detection algorithm, allocated to the sensor-processing chip), a **hardware requirement** (a physical drop sensor, allocated to the sensor itself), and a **manual requirement** (the owner must charge Rosie overnight, allocated to the person).

🧠 **Remember It Like This:** One big requirement branches into software, hardware, and people jobs, each given to the right part.

TIMING REQUIREMENTS FOR REAL-TIME SYSTEMS

In Simple Words: A soccer goalie has to block the ball the instant it's kicked — not five minutes later. Real-time systems work the same way: reacting on time, every time.

> 📖 **New Word!** A real-time system is a system that must respond within a strict deadline, or something goes seriously wrong.

🌟 **Bonus Example:** Rosie must detect a staircase edge and stop within a fraction of a second — waiting too long isn't an option.

🧠 **Remember It Like This:** Real-time means on time, every time — late is a fail.

Predictability

In Simple Words: You could set your watch by something predictable — it happens at the exact same time, over and over.

> 📖 **New Word!** Predictability means a repeated (periodic) event happens with consistent, reliable timing.

🌟 **Bonus Example:** Rosie is scheduled to empty her memory logs at exactly 2:00 AM every single night, without fail — just like a system that archives all its data on the last day of every month, every month, without fail.

🧠 **Remember It Like This:** Predictable means you could set your watch by it.

Execution Time

In Simple Words: The stopwatch time from when a task starts to when it's completely finished.

> 📖 **New Word!** Execution time is the elapsed time from when a task begins to when it finishes.

🌟 **Bonus Example:** Rosie's nightly log-archiving task always runs from 3:00 AM to 4:00 AM — that one-hour span is its execution time.

🧠 **Remember It Like This:** Execution time is the stopwatch time from start to finish of a task.

Latency

In Simple Words: The tiny pause between something happening and the system actually noticing it happened.

> 📖 **New Word!** Latency is the time lag between a trigger event occurring and the system beginning to respond to it.

🌟 **Bonus Example:** When Rosie's bump sensor detects a wall, there's a tiny 0.05-second latency before her brain starts telling the wheels to turn.

🧠 **Remember It Like This:** Latency is the little pause between "something happened" and "the system notices."

ADDITIONAL TIMING & SCHEDULING ISSUES TO EXPLORE

In Simple Words: Building something that must react instantly means asking a whole checklist of extra timing questions — way more than a regular app ever needs. 🌟 **Bonus Example:** Before Rosie's engineers finalize her design, they run through eleven extra timing questions, one by one.

🧠 **Remember It Like This:** Real-time systems need a whole extra checklist that regular apps can skip.

Periodicity (Frequency) of Task Execution

In Simple Words: How often does something need to happen — like an alarm clock ringing every morning? 🌟 **Bonus Example:** How often does Rosie need to check her battery level — every 10 seconds? Every minute?

🧠 **Remember It Like This:** Periodicity is how often the alarm clock rings.

Deadlines and Tolerances (Float)

In Simple Words: A deadline is the line you can't cross; tolerance (float) is how much wiggle room you have before you cross it. 🌟 **Bonus Example:** Rosie must respond to a stair-edge signal within 0.2 seconds, with barely any wiggle room (float) before it becomes too late.

🧠 **Remember It Like This:** A deadline is the line; tolerance is how far you can wiggle before crossing it.

Typical and Worst-Case Execution Time per Task

In Simple Words: You should plan for a normal day AND for the messiest possible day, not just the easy one. 🌟 **Bonus Example:** Rosie's obstacle-detection task usually takes 0.01 seconds (typical), but in a room full of furniture, it could take up to 0.05 seconds (worst case) — engineers must plan for both.

🧠 **Remember It Like This:** Plan for the normal day AND the messiest possible day.

Consequences of Missing a Deadline

In Simple Words: Always ask, "What actually breaks if this is late?" 🌟 **Bonus Example:** If Rosie's brain misses the 0.2-second stair-edge deadline, she could tumble down the stairs — a serious consequence that must be planned for.

🧠 **Remember It Like This:** Ask what breaks (or falls down the stairs) if the deadline is missed.

Minimum, Average, and Maximum Arrival Rate of Data

In Simple Words: Data can trickle in slowly, arrive at a normal steady pace, or flood in all at once — a good system has to handle the whole range, not just the average. 🌟 **Bonus Example:** Rosie's dust sensor might send data as rarely as once per second (minimum), about 10 times per second on a normal cleaning day (average), or up to 100 times per second in a very dusty room (maximum) — and each of those rates might look different depending on whether Rosie is cruising, docked, or emptying her bin.

🧠 **Remember It Like This:** Data can trickle in, flow steadily, or flood in — be ready for all three.

Maximum Time Before First Input/Output Is Expected

In Simple Words: Nobody wants to stare at silence for too long — a system should show *something* is happening quickly. 🌟 **Bonus Example:** After Rosie powers on, she must show her first status light within 3 seconds, or the owner will think she's broken.

🧠 **Remember It Like This:** Don't leave people staring at silence for too long.

Timeout — What To Do If Data Isn't Received in Time

In Simple Words: Every good plan needs a backup for when the expected information never actually shows up.

> 📖 **New Word!** A timeout is what happens when a system decides "I've waited long enough" and takes a backup action instead of waiting forever.

🌟 **Bonus Example:** If Rosie's Wi-Fi sensor doesn't get a signal back within 5 seconds, she switches to "offline cleaning mode" instead of just freezing in place.

🧠 **Remember It Like This:** Always have a backup plan for when the expected data never shows up.

The Sequence in Which Tasks Must Run

In Simple Words: Some tasks are picky about order — they absolutely need to happen before others. 🌟 **Bonus Example:** Rosie must finish mapping the room *before* she starts vacuuming it — the order matters.

🧠 **Remember It Like This:** Some tasks are picky — they need their turn, in the right order.

Tasks That Must Begin/End Before Other Tasks Begin

In Simple Words: You put your socks on before your shoes — some tasks simply must finish first. 🌟
Bonus Example: Rosie's "charge battery" task must fully finish before her "start cleaning" task can begin.

🧠 **Remember It Like This:** Socks before shoes — some tasks are simply first.

Task Prioritization & Pre-emption

In Simple Words: The most urgent task gets to cut in line, ahead of whatever was happening before.

> 📖 **New Word!** To pre-empt means to interrupt a task that's already running so a more urgent one can take over.

🌟 **Bonus Example:** If Rosie is mid-vacuum and her "low battery emergency" alert fires, that task pre-empts (interrupts) the vacuuming task and takes over immediately — because "don't die with an empty battery in the middle of the hallway" is judged more urgent than "finish this one room."

🧠 **Remember It Like This:** The most urgent task gets to cut in line.

Functions That Depend on System Mode

In Simple Words: The same machine can follow different rules depending on what "mode" it's currently in.
🌟 **Bonus Example:** Rosie has a "normal cleaning mode" and a "child-safety mode" (she won't run if a toddler is nearby) — just like a real elevator behaves differently in normal mode versus firefighter service mode during a fire.

🧠 **Remember It Like This:** Same machine, different rules, depending on the mode it's in.

🎯 **Quick Recap:**

- **System requirements, architecture, allocation** = the overall blueprint before assigning jobs to parts
- **Architecture's three elements** = Components, Externally visible properties, Connections
- **Decomposition & allocation** = one big requirement splits into software/manual/hardware, each assigned to a part
- **Timing requirements** = Predictability, Execution time, and Latency
- **Extra timing/scheduling checklist** = periodicity, deadlines/tolerances, typical/worst-case time, consequences of missing deadlines, data arrival rates, max wait before first I/O, timeouts, task sequence, precedence, prioritization/pre-emption, and mode-dependent functions

🏆 Chapter Wrap-Up

Maya, Sam, Priya, and Mr. Chen started this chapter thinking there was only one way to build software — writing every line yourself. Now they know better! Sometimes you buy and adapt a package instead of building from scratch. Sometimes you bring in outside help and manage that relationship carefully. Every

project has a “current way of doing things” worth studying and improving. Some projects swim in oceans of data, and some — like Rosie the robot vacuum — must react in the blink of an eye. Different projects, different rules, but the same careful thinking about requirements underneath it all.

If you remember ONLY these things…

- **Packaged Solutions:** Sometimes buying and adapting ready-made software (COTS/SaaS) beats building from zero — but you still need requirements to pick and fit it right.
- **Outsourced Projects:** When you hire outside help, requirements become the shared language that keeps the acquirer and supplier working together correctly.
- **Business Process Automation:** Before you improve or automate anything, study how it’s actually done today — then decide whether to tweak it (BPI) or reinvent it (BPR).
- **Big Data:** Huge, fast, complicated data needs its own special questions about sources, storage, governance, extraction, and how people will actually use the results.
- **Embedded & Real-Time Systems:** When a system must react instantly, every millisecond of timing — deadlines, latency, priorities — becomes its own critical requirement.

Chapter 7: Requirements Management

Welcome back! Maya, Sam, and Priya are deep into building **Homework Helper**, the app that will help every kid at Lincoln Middle School remember their assignments. But here's the problem: requirements don't just sit still once you write them down. Teachers ask for new features, students spot mistakes, and priorities shift — so in this chapter, we'll learn how Maya's team keeps everything organized, tracked, and under control, no matter how much changes along the way.

What Is Requirements Management?

DEFINITION OF REQUIREMENTS MANAGEMENT

In Simple Words: Imagine you're planning a class camping trip. You don't just make one list of supplies and forget about it — you keep checking it, updating it when someone forgets a flashlight, deciding who's in charge of buying more marshmallows, and telling everyone when the plan changes. Requirements management is exactly that, but for a software project: it's the ongoing job of writing down what the app should do, checking it, tracking changes to it, deciding what's most important, getting everyone to agree, and making sure everyone knows what's going on — from the very first day of the project until the very last.

> 📖 **New Word!** A **requirement** means a specific thing the app must be able to do or must have — like "the app must send a reminder the day before homework is due."

> 📖 **New Word! Requirements management** means the ongoing job of writing, checking, tracking, prioritizing, and agreeing on requirements — and making sure everyone affected knows what's happening with them, all the way through the project.

Example — Homework Helper App: On September 8, 2026, Maya writes down her first batch of requirements after talking to Mr. Chen and a group of 7th graders. But the project doesn't stop there! Every week, Maya updates a shared spreadsheet, checks off requirements that are done, adds new ones students suggest, and lets Sam and Priya know what changed. She does this every single week from September all the way to the app's launch in December — that ongoing work is requirements management.

> 📖 **New Word!** A **stakeholder** means anyone who cares about the project or is affected by it — like Mr. Chen, the students, Maya, Sam, and Priya.

🧠 **Remember It Like This:** Requirements management is like watering a garden — you don't plant the seeds once and walk away, you tend to it the whole time it's growing.

REQUIREMENTS BASELINE

In Simple Words: Think about a board game with house rules. Before you start playing, everyone agrees: "Okay, THESE are the rules we're using tonight." Once everyone agrees, you don't change the rules in the middle of the game without everyone talking about it first. A baseline is the same idea — it's the official, agreed-upon set of requirements that everyone has signed off on for a certain version of the app.

> 📖 **New Word!** A **baseline** means a set of requirements that everyone has agreed to and approved — usually for one specific version or release of the app. It's not "frozen forever," but you can't change it without going through a proper process.

Example — Homework Helper App: Maya writes up 20 requirements for "Homework Helper Version 1.0." The team reviews them together on October 3, 2026. They find two mistakes (one requirement said "send reminder 2 minutes before" instead of "2 hours before"!), fix them, and everyone — Maya, Sam, Priya, and Mr. Chen — signs off. That approved list of 20 requirements officially becomes the **Version 1.0 Baseline**. From then on, if anyone wants to add or change something, they can't just sneak it in — they have to go through the change process (which we'll learn about later in this chapter!).

> 📖 **New Word! Configuration management** means a system for keeping careful control over versions of things (like documents or code) so nothing gets lost or changed by accident — a baseline is one of its key tools. It lets the team control change without blocking a good, justified requirement change from happening.

🧠 **Remember It Like This:** A baseline is a "line in the sand" — everyone agreed to it, so you don't cross it without a good reason and everyone's okay with it.

🎯 **Quick Recap:**

- **Definition of Requirements Management** — the whole ongoing job of writing, tracking, and agreeing on requirements throughout a project.
- **Requirements Baseline** — the official, agreed-upon version of requirements that the team has signed off on.

The Four Pillars: How We Manage Requirements

MAJOR REQUIREMENTS MANAGEMENT ACTIVITIES (THE FOUR PILLARS)

In Simple Words: Picture a sturdy table standing on four legs. If even one leg is wobbly, the whole table shakes. Requirements management stands on four "legs," or pillars, that hold the whole process steady: Version Control, Change Control, Requirements Status Tracking, and Requirements Tracing.

Example — Homework Helper App: Maya's team uses all four pillars every single week without even thinking about it: they keep old copies of requirements (Version Control), they have a process for approving new ideas (Change Control), they know which requirements are done and which aren't (Status Tracking), and they can point to exactly which part of the code matches which requirement (Tracing).

Here's the real breakdown, straight from the lecture diagram:

Pillar	What It Covers
Version Control	Defining a version identification scheme; Tracking individual requirement versions; Tracking versions of requirement sets
Change Control	Proposing changes; Analyzing impact; Making decisions; Updating individual requirements; Updating sets of requirements; Updating plans; Measuring requirements volatility
Requirements Status Tracking	Defining possible requirement statuses; Recording the status of each requirement; Tracking the status distribution of all requirements
Requirements Tracing	Defining links to other requirements; Defining links to other system elements

🖼️ **What this picture is telling you:** All four boxes connect to one big “Requirements Management” box at the top — these four jobs are the whole toolkit for keeping requirements under control.

🧠 **Remember It Like This:** Four pillars, one steady table: Version, Change, Status, and Tracing hold up the whole project.

VERSION CONTROL

In Simple Words: It’s like saving your school essay as “Essay_draft1,” then “Essay_draft2,” then “Essay_FINAL” — so you never lose your earlier work and always know which copy is the newest one. Version control means giving requirements version numbers and keeping track of every version, both one at a time and as a whole set.

> 📖 **New Word! Version control** means keeping track of different versions of something over time — like requirement v1, v2, v3 — so you always know which one is current and can go back to an older one if needed.

Example — Homework Helper App: Requirement REQ-014 starts as “Send a reminder the day an assignment is due” (Version 1, written Sept. 10, 2026). After talking to students, Maya updates it to “Send a reminder 24 hours before an assignment is due” (Version 2, Oct. 1, 2026). Then Priya suggests adding a second reminder, so it becomes “Send reminders 24 hours AND 2 hours before an assignment is due” (Version 3, Oct. 15, 2026). Maya keeps all three versions in her tracking sheet, so if Version 3 causes problems, they can always look back at exactly what Version 2 said.

🧠 **Remember It Like This:** Version control is like a video game’s save files — you can always go back to an earlier save if the newest one breaks something.

CHANGE CONTROL

In Simple Words: This is like the “house rules” for changing house rules! It’s the official steps a team follows whenever someone wants to change a requirement — you can’t just change it on a whim; you propose it, check what it affects, decide together, and then update everything that needs updating.

> 📖 **New Word! Change control** means a formal process for proposing, reviewing, deciding on, and applying changes to requirements — instead of letting anyone change them whenever they feel like it.

Example — Homework Helper App: When Mr. Chen asks, “Can we add a feature where the app shows a fun sticker when you finish all your homework for the week?” Maya doesn’t just add it. She writes it up as a proposed change, Sam checks how much work it would take, the team decides together whether it fits into Version 1.0 or should wait, and only then does Maya update the requirements list and Sam’s task plan.

> 📖 **New Word! Requirements volatility** means how much (and how often) the requirements are changing — a way of measuring how “wobbly” the requirements are.

🧠 **Remember It Like This:** Change control is the “raise your hand and ask first” rule for requirements — no surprise changes allowed!

REQUIREMENTS STATUS TRACKING

In Simple Words: It’s like a chore chart on the fridge that shows which chores are “Not Started,” “In Progress,” or “Done.” Requirements status tracking means giving every requirement a status label and always knowing where each one stands.

> 📖 **New Word! Requirements status tracking** means keeping track of what stage each requirement is at (like “Proposed,” “Approved,” or “Verified”) and looking at the big picture of how many requirements are at each stage.

Example — Homework Helper App: Maya’s spreadsheet has a “Status” column next to every requirement. REQ-021 (“Let students mark an assignment as Done”) shows “Verified” because Priya already tested it and it works. REQ-045 (“Add a study-buddy chat feature”) shows “Proposed” because nobody has decided on it yet. At any moment, Mr. Chen can glance at the sheet and see that 12 requirements are Verified, 5 are In Progress, and 3 are still Proposed.

🧠 **Remember It Like This:** Status tracking is a chore chart for requirements — everyone can see what’s done, what’s doing, and what’s still waiting.

REQUIREMENTS TRACING

In Simple Words: Imagine a treasure map where a dotted line connects the “X marks the spot” to a list of tools you’ll need to dig there. Requirements tracing draws those same kinds of connecting lines — between one requirement and another, and between a requirement and the actual pieces of the system (like the code, the design, or the tests) that make it happen.

> 📖 **New Word! Requirements tracing** (also called **traceability**) means keeping links between a requirement and everything connected to it — other requirements, the code that implements it, and the tests that check it.

Example — Homework Helper App: REQ-014 (“Send a reminder 24 hours before an assignment is due”) is linked to: the piece of Sam’s code called `ReminderScheduler.swift`, the test Priya wrote called “Test Case 7: Reminder Timing,” and a related requirement REQ-015 (“Let students turn reminders on or off”). If Sam

ever changes `ReminderScheduler.swift`, Maya can trace backward and instantly know exactly which requirement and which test might be affected.

🧠 **Remember It Like This:** Tracing is the dotted line on the treasure map — follow it, and you always find what connects to what.

🎯 **Quick Recap:**

- **Major Requirements Management Activities** — the four pillars (Version Control, Change Control, Status Tracking, Tracing) that hold up the whole process.
- **Version Control** — keeping numbered versions of each requirement so nothing gets lost.
- **Change Control** — the official steps for proposing and approving a change.
- **Requirements Status Tracking** — labeling each requirement's stage, like a chore chart.
- **Requirements Tracing** — drawing connecting lines between requirements and the code/tests/other requirements linked to them.

When Things Need to Change: Handling New Requirements

FOUR WAYS TO ACCOMMODATE NEW OR CHANGED REQUIREMENTS

In Simple Words: Imagine you're packing a suitcase for a trip, and it's already full, but your mom says, "Please pack one more sweater!" You have choices: take something else out, get a bigger suitcase, pack a second bag, or squeeze it in and risk the zipper breaking. Teams have the same kinds of choices when a new or changed requirement shows up after the plan is already set.

Example — Homework Helper App: It's November 1, 2026, and the app is supposed to launch December 15. Mr. Chen suddenly asks for a brand-new feature: a "Parent View" so parents can also see assignments. Maya's team is already full up on work. Here are their four choices, straight from the lecture:

1. **Defer lower-priority requirements to later iterations or cut them completely.** (Maya could push the "fun sticker rewards" feature to Version 2.0 to make room.)
2. **Obtain additional staff or outsource some of the work.** (Maybe Maya asks another classmate, Diego, to help Sam code for two weeks.)
3. **Extend the delivery schedule or add iterations to an agile project.** (The team could move launch day to January 5, 2027 instead.)
4. **Sacrifice quality to ship by the original date.** (Skip testing the Parent View properly just to hit December 15 — this sounds tempting, but it's really a bad idea, because a broken feature makes everyone unhappy later!)

🧠 **Remember It Like This:** When the suitcase won't zip, you either take something out, get help, get a bigger bag, or you can risk breaking the zipper — but breaking the zipper is never a good plan!

🎯 **Quick Recap:**

- **Four Ways to Accommodate Changes** — defer/cut something else, get more help, extend the schedule, or (as a last, bad resort) sacrifice quality.

Keeping Good Records: Attributes, Status, and Issues

REQUIREMENT ATTRIBUTES

In Simple Words: Think of a trading card for your favorite athlete or game character. It doesn't just say their name — it also lists their stats, team, and special powers. A requirement can have its own "trading card" full of extra facts about it, called attributes.

> 📖 **New Word!** An **attribute** means a piece of extra information that describes something — like a label with useful details attached to it.

Example — Homework Helper App: Here's the "trading card" Maya keeps for REQ-014:

Attribute	Homework Helper Example
Date the requirement was created	September 10, 2026
Current version number	Version 3
Author	Maya Rodriguez
Priority	High
Origin/source of the requirement	Suggested by 8th-grade student council
Rationale behind the requirement	Students said they forget homework because they don't see reminders early enough
Release number / iteration allocated to	Version 1.0
Stakeholder to contact with questions/decisions	Mr. Chen
Validation method or acceptance criteria	Test on 3 different phones; reminder must appear exactly 24 hours before the due date/time

> 📖 **New Word! Acceptance criteria** means the specific test a requirement has to pass for everyone to agree it's really finished and working correctly.

🧠 **Remember It Like This:** Every requirement gets its own trading card, packed with stats that tell its whole story.

TRACKING REQUIREMENTS STATUS

In Simple Words: Remember the chore chart idea? Here's the exact set of labels real software teams use, like stickers you'd put next to each chore.

Example — Homework Helper App: Priya keeps this status list taped above her desk so the whole team uses the same words. Here's Table 27-1, "Suggested requirement statuses," exactly as it appears in the source material — and next to it, a Homework Helper requirement that's currently at that stage:

Status	Definition	Homework Helper Example
Proposed	The requirement has been requested by an authorized source.	REQ-050 “Add a dark mode” — Mr. Chen just suggested it.
In Progress	A business analyst is actively working on crafting the requirement.	Maya is still interviewing students about REQ-052.
Drafted	The initial version of the requirement has been written.	REQ-045 has a first written draft, not reviewed yet.
Approved	The requirement has been analyzed, its impact estimated, allocated to the baseline for a specific release; key stakeholders agreed to incorporate it; dev group committed to implement it.	REQ-021 was approved for Version 1.0 on Oct 3.
Implemented	The code implementing the requirement has been designed, written, unit tested; traced to design/code elements; ready for testing/review/verification.	Sam finished coding REQ-014, ready for Priya to test.
Verified	The requirement has satisfied its acceptance criteria (correct functioning confirmed); traced to pertinent tests. Considered complete.	Priya confirmed REQ-021 works on all 3 test phones.
Deferred	An approved requirement is now planned for implementation in a later release.	REQ-060 “sticker rewards” moved to Version 2.0.
Deleted	An approved requirement has been removed from the baseline. Include explanation of why/by whom.	REQ-033 removed — Mr. Chen said the school won’t allow that feature after all.
Rejected	The requirement was proposed but never approved and is not planned for any upcoming release. Include explanation of why/by whom.	REQ-041 “connect to social media” rejected by Mr. Chen for privacy reasons.

🖼️ **What this picture is telling you:** This is the path a requirement travels — like a passport getting stamped at each stop on its journey from “just an idea” to “done and tested” (or sometimes turned away at “Rejected”!).

🧠 **Remember It Like This:** Every requirement has a passport, and each status is a stamp on its journey.

RESOLVING REQUIREMENTS ISSUES

In Simple Words: Have you ever played a group project where questions kept popping up — “Wait, who’s bringing the poster board?” — and someone had to write them all down so nothing got forgotten? That’s what an issue-tracking tool does for requirements: it collects every open question so nothing slips through the cracks.

> 📖 **New Word!** An **issue-tracking tool** means a shared list or software tool where a team writes down every open question or problem about the requirements, so nothing gets forgotten and everyone can see the status.

Example — Homework Helper App: During a review meeting on October 3, Priya asks, “Wait, what happens if a student has TWO assignments due at the exact same time?” Nobody knows the answer yet. Instead of just talking about it and forgetting, Maya logs it in their shared issue tracker as “Issue #12,” assigns herself to find the answer, and everyone can see it’s still open. Using a tracker like this gives the team real benefits:

- Issues from multiple reviews are collected so none get lost.
- The project manager (Maya) can see the current status of all issues at a glance.
- A single owner is assigned per issue (so it’s always clear who’s responsible).
- The history of discussion around an issue is kept, not lost in someone’s memory.
- The team (Sam) can start coding earlier, because they know exactly which issues are still open instead of waiting for a perfectly finished requirements document.

Here’s Table 27-2, “Common types of requirements issues,” with a Homework Helper example for each:

Issue type	Description	Homework Helper Example
Requirement question	Something isn’t understood or decided about a requirement.	”Does ‘due date’ mean midnight or when class starts?”
Missing requirement	Developers uncovered a missed requirement during design or implementation.	Sam realizes there’s no requirement for what happens with no internet connection.
Incorrect requirement	A requirement was wrong; should be corrected or removed.	REQ-014 originally said “2 minutes before” — that was a typo for “2 hours.”
Implementation question	As developers implement requirements, they have questions about how something should work or about design alternatives.	Sam asks, “Should the reminder be a pop-up or a sound?”
Duplicate requirement	Two or more equivalent requirements are discovered — delete all but one.	REQ-018 and REQ-039 both say “let students check off finished homework.”
Unneeded requirement	A requirement simply isn’t needed anymore.	REQ-005 “print a paper schedule” — nobody actually wants this feature.

🖼️ **What this picture is telling you:** These are the six most common “uh-oh, we need to talk about this” moments every requirements team runs into — knowing the type helps the team know exactly how to fix it.

🧠 **Remember It Like This:** An issue tracker is a group project’s “parking lot” list — every stray question gets parked somewhere safe instead of driving off and getting lost.

🎯 **Quick Recap:**

- **Requirement Attributes** — the “trading card” facts attached to each requirement (date, author, priority, and more).
- **Tracking Requirements Status** — the 9-stage “passport stamps” every requirement travels through.
- **Resolving Requirements Issues** — using a shared tracker to catch and solve the 6 common types of requirement problems.

Why and How We Manage Change

WHY MANAGE CHANGES

In Simple Words: Imagine your class is building a giant LEGO castle for a school fair, and halfway through, ten different kids start adding random pieces wherever they want. Chaos! Managing change means making sure that whenever someone wants to add or change something, it happens carefully — not chaotically.

Example — Homework Helper App: Mr. Chen’s school takes this seriously. Before any change to Homework Helper happens, the team makes sure of four things:

- Proposed changes are thoughtfully evaluated before anyone commits to them. (Maya doesn’t say “sure!” the second Mr. Chen asks for something — she checks with Sam and Priya first.)
- Appropriate people make informed business decisions about requested changes. (Big decisions go through Mr. Chen, not just whoever is in the room.)
- Change activity is visible to affected stakeholders. (Everyone can see what’s being proposed, not just Maya.)
- Approved changes are communicated to all affected participants. (Once something IS approved, Sam and Priya both get told right away — not two weeks later.)

🧠 **Remember It Like This:** Ten kids randomly adding LEGO pieces makes a mess — one plan, checked and shared with everyone, keeps the castle standing strong.

MANAGING SCOPE CREEP

In Simple Words: Scope creep is like asking for “just one more topping” on a pizza, over and over, until suddenly your simple cheese pizza has 15 toppings, costs way more, and takes an hour longer to cook. It’s when little extra requests keep sneaking in until the project becomes way bigger than planned.

> 📖 **New Word! Scope creep** means when a project slowly grows bigger and bigger from lots of small added requests, until it’s way more than what was originally planned.

Example — Homework Helper App: First Mr. Chen asks for reminders. Then a parent asks for a Parent View. Then a student asks for a chat feature. Then someone asks for a badge system. Each request sounds small and nice on its own — but if Maya says “yes” to every single one, Homework Helper would balloon from a simple 6-week project into a 6-month one! The most effective technique for handling this is simply learning to say “no” (or at least “not yet”). Teams often feel pressure to always say “yes” — ideas like “the customer is always right” sound nice, but change is never actually free; it always costs time or money. One clever trick: instead of a harsh “No,” a wise team lead might say “**not now**” — it feels much kinder, and it holds out the promise that the idea could come in a future version, without blowing up the current plan.

Example dialogue: Mr. Chen asks, “Can we add video calling to Homework Helper by next week?” Maya smiles and says, “Not now, Mr. Chen — let’s get Version 1.0 solid first, and I’ll put video calling on our list for Version 2.0!”

🧠 **Remember It Like This:** Every “just one more topping” adds up — a good project lead knows how to say “not now” before the pizza gets too big to lift.

🎯 **Quick Recap:**

- **Why Manage Changes** — so changes are evaluated, decided by the right people, made visible, and clearly communicated.
- **Managing Scope Creep** — saying “no” (or “not now”) to keep small requests from snowballing into a huge, unplanned project.

Setting Up the Rules: Policy, Process, and the CCB

CHANGE CONTROL POLICY

In Simple Words: This is like the official rulebook for a school sports league — everyone has to follow the same rules for how a game is played and scored, no exceptions, no matter how much you beg the referee.

> 📖 **New Word!** A **Change Control Policy** means a set of official rules an organization follows for how requirement changes get proposed, reviewed, and approved.

Example — Homework Helper App: Mr. Chen’s school actually writes these rules down for the Homework Helper project, based on the same 7 rules real software teams use:

1. All changes must follow the defined process; a change request not submitted the proper way will not be considered. (Mr. Chen can’t just text Sam at midnight — he has to fill out the change form.)
2. No design or coding work (other than checking if it’s even possible) happens on a change that isn’t approved yet. (Sam can’t start building the Parent View until it’s officially approved.)
3. Asking for a change doesn’t guarantee it happens — the Change Control Board decides. (We’ll meet this board next!)
4. The change list must be visible to everyone on the project. (Anyone on the team can see every requested change and its status.)
5. Every change must go through impact analysis first. (The team must check what a change would affect before saying yes.)
6. Every change that actually gets made must be traceable back to an approved change request. (No mystery changes allowed!)
7. The reason behind every approval or rejection must be written down. (So nobody has to guess later why something was decided.)

🧠 **Remember It Like This:** No official rulebook, no game — the Change Control Policy is the rulebook that keeps every change fair and orderly.

CHANGE CONTROL PROCESS DESCRIPTION

In Simple Words: This is like the instruction manual that explains exactly HOW your class runs a fair class-president election — not just "we vote," but who's in charge, what counts as a valid vote, and the exact steps from nomination to final announcement.

> 📖 **New Word!** A **Change Control Process Description** means a written document that spells out, step by step, exactly how the change process works for a project.

Example — Homework Helper App: Maya writes a one-page document for the Homework Helper team, following the same outline real teams use (Figure 28-1):

1. Purpose and scope
2. Roles and responsibilities
3. Change request states
4. Entry criteria
5. Tasks
    - 5.1 Evaluate change request
    - 5.2 Make change decision
    - 5.3 Implement the change
    - 5.4 Verify the change
6. Exit criteria
7. Change control status reporting
    - Appendix: Attributes stored for each request

🖼️ **What this picture is telling you:** This is the table of contents for the team's own "change rulebook" — it shows every stage a change request travels through, from someone asking for it to someone double-checking it actually works.

Example in action: When Mr. Chen requests the Parent View feature, it moves through Maya's document exactly like this: it's logged (state: Submitted), Sam evaluates it (5.1), the team decides together (5.2), Sam builds it (5.3), and Priya checks it works (5.4) — following the very same steps written in Maya's one-page document.

🧠 **Remember It Like This:** A process description is the instruction manual for making changes — follow the steps in order, and nothing gets skipped.

THE CHANGE CONTROL BOARD (CCB)

In Simple Words: Think of the judges' panel on a baking competition show. They're the ones who officially taste-test every change and decide, together, whether it's a "yes" or a "no." A Change Control Board (CCB) is the group of people with the authority to approve or reject requirement changes.

> 📖 **New Word!** The **Change Control Board (CCB)** means the group of people who officially review and decide on proposed changes to requirements.

Example — Homework Helper App: For a project this size, Homework Helper's "CCB" is small but mighty: Maya, Sam, Priya, and Mr. Chen meet every Friday at lunch for 15 minutes to go through the list of proposed changes and vote on each one.

🧠 **Remember It Like This:** The CCB is the judges' panel — nothing gets served (or shipped) without their official thumbs-up.

CCB COMPOSITION

In Simple Words: A good judges' panel doesn't have five bakers and zero people who actually like cake — it needs a mix of different viewpoints. The same is true for a CCB: it should include representatives from every group needed to make the decisions within its authority.

Example — Homework Helper App: In a big real company, a CCB would include representatives from these six areas — and here's who plays that role on Team Homework Helper:

CCB Role (Real-World)	Homework Helper Stand-In
Project or program management	Maya
Business analysis / product management	Maya (she wears two hats!)
Development	Sam
Testing / QA	Priya
Marketing, the business unit, or customer representatives	Mr. Chen (he represents the school)
Technical support / help desk	A volunteer 8th grader who helps classmates use the app

🧠 **Remember It Like This:** A CCB needs a seat at the table for everyone the decision touches — not just the loudest voice in the room.

CCB DECISION-MAKING

In Simple Words: Before a class votes on anything — like picking a field trip destination — you need to agree on the rules first: how many people need to vote for it to count? Does the teacher get the final say? A CCB has to answer those same kinds of questions before it starts deciding on changes.

Example — Homework Helper App: Before their first Friday meeting, Maya's team agrees on four ground rules, matching what every real CCB must define:

- **The quorum** — how many members (or which key roles) need to be present for a decision to count. (Homework Helper's rule: at least 3 of the 4 members must be there.)

> 📖 **New Word!** A **quorum** means the minimum number of people who must be present for a group's decision to officially count.

- **The decision rules** — how they decide (Homework Helper's rule: majority vote, 3 out of 4).
- **Whether the Chair can overrule the group** — (Homework Helper's rule: Mr. Chen, as Chair, can overrule if it's a school-policy issue, like privacy.)
- **Whether a higher-level board or manager must approve it too** — (Homework Helper's rule: any change costing more than 10 extra hours of work needs the school principal's okay too.)

🧠 **Remember It Like This:** Agree on the voting rules BEFORE the vote — not in the middle of an argument about who wins.

🎯 **Quick Recap:**

- **Change Control Policy** — the official 7-rule rulebook every change must follow.
- **Change Control Process Description** — the step-by-step instruction manual for how changes move through the system.
- **The Change Control Board (CCB)** — the "judges' panel" that officially decides on changes.
- **CCB Composition** — making sure every affected group has a seat at the table.
- **CCB Decision-Making** — agreeing ahead of time on quorum, voting rules, chair power, and higher approval.

Tools and Measurement

CHANGE CONTROL TOOLS

In Simple Words: Imagine trying to run that class election with just scraps of paper passed around the room — messy, right? A digital voting app that tracks every ballot, sends reminders, and shows results instantly would work way better. Change control tools are the software that helps a team manage the whole change process smoothly.

> 📖 **New Word!** A **change control tool** means software built to help a team manage the entire lifecycle of a change request — from proposal to final decision.

Example — Homework Helper App: Maya sets up a simple shared spreadsheet-and-form combo (a mini version of a real change control tool) that does everything a good tool should:

- Lets the team define what information (attributes) a change request needs (like title, description, requester).
- Lets a change request move through a life cycle of statuses (Submitted → Under Review → Approved/Rejected → Implemented → Verified).
- Enforces who's allowed to move a request between statuses — only lets Maya (as the owner) move a request from "Under Review" to "Approved"; Sam and Priya can't skip that step.
- Records the date of every status change and who made it (so there's a full history).
- Sends an automatic, customizable email to the whole team whenever someone submits a new request or its status changes.

- Can produce standard and custom reports and charts, like “how many change requests came from students vs. from Mr. Chen.”

🧠 **Remember It Like This:** A good change control tool is a digital ballot box — it counts every vote correctly and shows you exactly what happened and when.

MEASURING CHANGE ACTIVITY

In Simple Words: A doctor checks your temperature and heart rate to see how healthy you are. A project needs check-ups too — and measuring change activity is how a team takes the project’s temperature.

> 📖 **New Word! Measuring change activity** means tracking numbers about how many changes are happening, where they come from, and how much work they cost — to understand the health of the project.

Example — Homework Helper App: At the end of October, Maya pulls together some numbers for Mr. Chen:

- Total number of change requests received / currently open / closed: 18 received, 4 open, 14 closed.
- Number of added, deleted, modified requirements: 9 added, 2 deleted, 7 modified.
- Number of requests from each origin (students, Mr. Chen, Maya’s own team, etc.)
- Number of changes made against each requirement since it was baselined (REQ-014 has been changed 3 times since Version 1.0 was approved!)
- Total effort spent processing and implementing all these change requests: about 22 hours so far.

Here’s Figure 28-4, “Sample chart of requirement change origins,” recreated with its real numbers:

Change Origin	Number of Change Requests
Marketing	30
Management	17
Customer	15
Software Engineering	10
Hardware Engineering	7
Testing	6

```
Marketing            ############################## 30
Management           #################               17
Customer             ###############                 15
Software Engineering ##########                       10
Hardware Engineering #######                           7
Testing              ######                            6
```

🖼️ **What this picture is telling you:** This bar chart shows WHERE change requests are coming from — in this real-world example, most changes were requested by Marketing, not by developers or testers. For Homework Helper, Maya makes her own version of this chart and notices most of her changes come from "Students," which tells her she should talk to students even earlier next time!

🧠 **Remember It Like This:** Measuring change activity is the project's thermometer — the numbers tell you if things are running smoothly or running a fever.

🎯 **Quick Recap:**

- **Change Control Tools** — software that manages the change life cycle so nothing gets lost or skipped.
- **Measuring Change Activity** — tracking numbers on changes to check the health of the project.

Change Impact Analysis: Being a Requirements Detective

IMPACT ANALYSIS PROCEDURE

In Simple Words: Before you decide to move your pet hamster's cage to a new spot in your room, you'd think it through: will it still get enough light? Will the cage bump into your desk? Will your hamster be scared by the new noise from the hallway? Impact analysis is doing that same careful thinking — but for a requirement change, before you commit to it.

> 📖 **New Word! Impact analysis** means carefully figuring out everything a proposed change would affect before deciding whether to approve it.

Example — Homework Helper App: When Mr. Chen requests the Parent View feature, Maya doesn't just say yes or no on the spot. She follows the three-step impact analysis procedure:

1. **Understand the possible implications of the change.** (A single change can create a big ripple effect — it might touch other requirements, the app's design, Sam's code, Priya's tests, or even clash with other requirements or hurt things like speed or privacy.)
2. **Identify all requirements, files, models, and documents that might need modification.** (Maya lists: REQ-014, the login screen design, `ParentAccount.swift`, and the Privacy Policy document.)
3. **Identify the tasks required to implement the change and estimate effort.** (Sam figures out he'd need to build a new login type, and Priya figures out she'd need 5 new test cases.)

🧠 **Remember It Like This:** Think it through before you move the hamster cage — one small move can ripple out further than you'd expect.

SIMPLE EVALUATION PROCEDURE

In Simple Words: This is like a recipe for how to actually DO an impact analysis, step by step, so you don't forget anything — just like following a recipe card so your cookies come out right every time.

Example — Homework Helper App: For the Parent View request, Maya's team works through these exact eight steps:

1. Work through the checklist in Figure 28-5 (understanding implications — we'll see this next).
2. Work through the checklist in Figure 28-6 (what work products are affected). Some requirements management tools can even auto-generate part of this report automatically, using the traceability links we learned about earlier — since REQ-014 is already linked to its code and tests, the tool can instantly show what else might be touched!
3. Use the worksheet in Figure 28-7 to estimate effort for the tasks involved (most changes only need part of the worksheet).
4. Sum up the effort estimates. (Sam adds it all up: about 14 hours total.)
5. Identify the order tasks must happen in, and how they fit with tasks already planned. (Sam realizes the login redesign has to happen before the Parent View screens can be built.)
6. Estimate the impact on the project's schedule and cost. (This might push launch back by 3 days.)
7. Evaluate the change's priority against other pending requirements. (Is Parent View more important than the "dark mode" request? The team decides it is.)
8. Report the impact analysis results to the Change Control Board. (Maya presents all of this at Friday's CCB meeting.)

🧠 **Remember It Like This:** Follow the recipe card in order — skip a step, and the cookies (or the analysis) don't turn out right.

FIGURE 28-5 — CHECKLIST: UNDERSTANDING THE IMPLICATIONS

In Simple Words: This is like a detective's checklist of questions before closing a case: "Does this clue conflict with another clue? What happens if I ignore it? Who else does this affect?" This checklist helps the team think through every angle of a proposed change.

Example — Homework Helper App: For the Parent View change, Sam and Maya go through this exact list of questions (Figure 28-5), answering each one:

1. Will the change enhance or impair the ability to satisfy any business requirements?
2. Do any existing requirements in the baseline conflict with the proposed change?
3. Do any other pending requirements changes conflict with the proposed change?
4. What are the business or technical consequences of not making the change?
5. What are possible adverse side effects or other risks of making the proposed change?
6. Will the proposed change adversely affect performance or other quality attributes?
7. Is the proposed change feasible within known technical constraints and current staff skills?
8. Will the proposed change place unacceptable demands on any resources required for development, test, or operating environments?
9. Must any tools be acquired to implement and test the change?
10. How will the proposed change affect the sequence, dependencies, effort, or duration of any tasks currently in the project plan?
11. Will prototyping or other user input be required to validate the change?
12. How much effort already invested in the project will be lost if this change is accepted?

13. Will the proposed change cause an increase in product unit cost, such as by increasing third-party product licensing fees?
14. Will the change affect any marketing, manufacturing, training, or customer support plans?

🖼️ **What this picture is telling you:** This is the full detective checklist — going through all 14 questions helps the team catch problems BEFORE they say yes, instead of discovering them halfway through building it.

🧠 **Remember It Like This:** A good detective never closes the case without checking every question on the list first.

FIGURE 28-6 — CHECKLIST: WHAT WORK PRODUCTS ARE AFFECTED

In Simple Words: If you decide to repaint your bedroom a new color, you'd make a checklist of everything that touches: the walls, sure, but also the curtains that might clash, the light switch cover, maybe even the trim. This checklist helps the team spot every single piece of the project a change might touch.

Example — Homework Helper App: Sam works through this checklist (Figure 28-6) for the Parent View change:

1. Identify any user interface changes, additions, or deletions required.
2. Identify any changes, additions, or deletions required in reports, databases, or files.
3. Identify the design components that must be created, modified, or deleted.
4. Identify the source code files that must be created, modified, or deleted.
5. Identify any changes required in build files or procedures.
6. Identify existing unit, integration, and system tests to be modified or deleted.
7. Estimate the number of new unit, integration, and system tests needed.
8. Identify help screens, training or support materials, or other user documentation that must be created or modified.
9. Identify other applications, libraries, or hardware components affected by the change.
10. Identify any third-party software to be acquired or modified.
11. Identify any impact the proposed change will have on the project management plan, quality assurance plan, configuration management plan, or other plans.

🖼️ **What this picture is telling you:** This is the "everything the paint might touch" checklist — it makes sure nobody forgets a hidden piece, like the database or the help screens, that also needs updating.

Example results: Sam finds this touches: the login screen UI (#1), the user database (#2), a new `ParentAccount` design component (#3), two new source code files (#4), and 5 new tests (#7).

🧠 **Remember It Like This:** Before you repaint, check the curtains AND the trim — a change always touches more than just the one wall you were thinking of.

FIGURE 28-7 — WORKSHEET: ESTIMATING THE EFFORT

In Simple Words: This is like filling out a chore worksheet before a big garage cleanup — writing down every single task and guessing how many minutes each one will take, so you can add it all up and know how long the whole cleanup will really take.

Example — Homework Helper App: Sam fills out a copy of this exact worksheet (Figure 28-7) for the Parent View change, writing hours next to each task that applies:

Hours	Task
___	Update the SRS or requirements repository
___	Develop and evaluate a prototype
2	Create new design components
___	Modify existing design components
4	Develop new user interface components
___	Modify existing user interface components
1	Develop new user documentation and help screens
___	Modify existing user documentation and help screens
5	Develop new source code
1	Modify existing source code
___	License and integrate third-party software
___	Modify build files and procedures
2	Write new unit and integration tests
___	Modify existing unit and integration tests
1	Perform unit and integration testing after implementation
___	Write new system and acceptance tests
___	Modify existing system and acceptance tests
___	Modify automated test suites
___	Perform regression testing
___	Develop new reports
___	Modify existing reports
___	Develop new database elements
1	Modify existing database elements
___	Develop new data files
___	Modify existing data files
___	Modify various project plans
___	Update other documentation
1	Update the requirements traceability matrix

Hours	Task
1	Review modified work products
___	Perform rework following reviews and testing
___	Other tasks
19	**Total Estimated Effort**

🖼️ **What this picture is telling you:** This is a full task-by-task checklist for “how much work is this, really?” — most changes only need a handful of the rows filled in, and adding them up gives the real total (here, Sam estimates 19 hours for the Parent View feature).

🧠 **Remember It Like This:** Add up the minutes for every chore on the worksheet, and suddenly “quick little change” turns into an honest, real number.

FIGURE 28-8 — IMPACT ANALYSIS TEMPLATE

In Simple Words: This is like the official form a school nurse fills out after a kid gets hurt at recess — name, what happened, when, how bad it is. An impact analysis template is the official form a team fills out to summarize everything they found out about a proposed change.

Example — Homework Helper App: Here’s the filled-in Figure 28-8 form Maya brings to Friday’s CCB meeting:

Field	Homework Helper Example
Change request ID	CR-012
Title	Add Parent View feature
Description	Allow a parent account to log in and see (read-only) their child's assignments and due dates.
Evaluator	Sam Patel
Date prepared	November 3, 2026
Estimated total effort (labor hours)	19 hours
Estimated schedule impact (days)	+3 days
Additional cost impact (dollars)	$0 (student project, no budget impact)
Quality impact	Low risk, but needs careful privacy testing
Other components affected	Login screen, user database, privacy policy document
Other tasks affected	Priya's test plan needs 5 new test cases
Life-cycle cost issues	None expected — no ongoing cost to maintain

🖼️ **What this picture is telling you:** This one-page form is the final summary of the whole investigation — everything the detective checklist and worksheet found, boiled down into one clear report the CCB can quickly read and decide on.

🧠 **Remember It Like This:** The impact analysis template is the "incident report" — one clean page that tells decision-makers everything they need to know, fast.

🎯 **Quick Recap:**

- **Impact Analysis Procedure** — the 3-step big-picture process: understand implications, find what's affected, estimate the work.
- **Simple Evaluation Procedure** — the practical 8-step recipe for actually doing an impact analysis.
- **Figure 28-5 Checklist** — 14 detective questions to understand a change's implications.
- **Figure 28-6 Checklist** — 11 items to find every affected work product.
- **Figure 28-7 Worksheet** — a task-by-task worksheet for estimating total hours.
- **Figure 28-8 Template** — the one-page report that summarizes the whole analysis for the CCB.

Managing Change on Agile Teams

CHANGE MANAGEMENT ON AGILE PROJECTS

In Simple Words: Picture a lunch tray with three sections: what you're eating right now, what's coming up next, and the extra snacks waiting in your backpack for later. Agile teams manage their to-do list the same way, using something called a "product backlog," split into zones: the current iteration, the next iteration, and future iterations. Things can move between the zones as priorities shift.

> 📖 **New Word!** A **product backlog** means a prioritized to-do list of everything a team might build, organized by how soon each item is planned to happen.

> 📖 **New Word!** An **iteration** (sometimes called a "sprint") means a short, fixed chunk of time — like two weeks — during which a team works on a specific batch of items from the backlog.

Example — Homework Helper App: Suppose in December, Maya's team switches to working in two-week iterations. Here's their Prioritized Product Backlog, matching Figure 28-9's three zones:

1. **Current iteration** (this two weeks): finishing REQ-014 (reminders) and REQ-021 (mark as done).
2. **Next iteration** (the next two weeks): REQ-045 (study-buddy chat) and CR-012 (Parent View).
3. **Future iterations** (someday): REQ-060 (sticker rewards) and REQ-050 (dark mode).

And here's how items move between the zones, exactly like the arrows in Figure 28-9:

- **New requirement or task** — a brand-new idea can land in either of two spots depending on how urgent it is. When Mr. Chen suggests a "grade tracker" feature that a teacher needs soon, it drops straight into the **next iteration**. But when a student suggests a "confetti animation" that's fun but not urgent, it drops further down into the **future iterations** zone instead.
- **Lower priority requirement or task** — REQ-045 (chat feature) turns out to be harder than expected, so it moves backward from the next iteration into future iterations.
- **Reprioritized requirement or task** — REQ-050 (dark mode) suddenly becomes popular after lots of students ask for it, so it moves forward from future iterations up into the next iteration.
- **Deleted requirement or task** — REQ-005 (print a paper schedule) gets removed from the backlog entirely — nobody wants it, so it's deleted, not just delayed.

🖼️ **What this picture is telling you:** The backlog isn't a frozen list — it's alive! Items constantly slide between "doing it now," "doing it soon," and "maybe someday," and some get deleted altogether, which is exactly how agile teams handle change without needing a giant, slow change-control meeting for every little shift.

🧠 **Remember It Like This:** The backlog is a lunch tray with three sections — what's on your fork right now can change, and snacks can move up front if you get hungry for them sooner.

🎯 **Quick Recap:**

- **Change Management on Agile Projects** — using a living, prioritized backlog with current/next/future zones, so change flows naturally instead of needing heavy formal approval for every shift.

🏆 Chapter Wrap-Up

Maya, Sam, Priya, and Mr. Chen didn't just write a list of requirements once and walk away — they built a whole system to keep track of every version, every change, every status, and every connection, all the way from September to launch day. That's the real secret of requirements management: it's not a single step, it's a habit the whole team keeps up together, every single week, so that Homework Helper actually turns out to be the app everyone agreed to build.

If you remember ONLY these things…

- Requirements management is an ongoing job — write it down, then keep tracking, prioritizing, and agreeing on it the whole project, and baseline it once everyone signs off.
- The four pillars (Version Control, Change Control, Status Tracking, and Tracing) are the toolkit that keeps every requirement organized and connected.
- New or changed requirements always have to fit somewhere — defer something, get help, extend the schedule, or (never!) sacrifice quality.
- Good records matter — attributes, status labels, and an issue tracker keep nothing from getting lost or forgotten.
- Managing change means having real rules (a policy, a process, and a CCB) so changes are evaluated fairly instead of chaotically — and knowing when to kindly say "not now."
- Before saying yes to a change, be a detective: do an impact analysis to find out everything it will really touch and cost.
- Agile teams manage change with a living backlog instead of a big slow approval process — priorities can shift week to week.

Chapter 8: Requirements Tracing and Tools

Maya has 47 sticky notes stuck to her bedroom wall — one for every single thing a teacher or student has ever asked the Homework Helper app to do. She just realized she has no idea which notes are still important, which ones Sam already built, or which ones nobody actually asked for. In this chapter, you'll learn how Maya, Sam, Priya, and Mr. Chen learn to connect every requirement to exactly where it came from and exactly where it ends up — a skill called requirements tracing — plus the smart tools that make it all possible.

> 📖 **New Word!** A requirement means something the app must do or have — like "remind students about homework due tomorrow."

> 📖 **New Word!** A stakeholder means anyone who cares about a project or is affected by it — like Mr. Chen, the teachers, and the students who will use Homework Helper.

> 📖 **New Word!** A Business Analyst (BA) means the person — like Maya — whose job is to talk to stakeholders and turn what they need into clear requirements.

> 📖 **New Word!** Tracing (also called traceability) means following the connections between requirements — from where an idea came from, to how it was built, to how it was tested.

Motivations for Tracing Requirements

Why bother connecting all those dots at all? Here are all nine reasons Maya's team cares about tracing.

FINDING MISSING REQUIREMENTS

In Simple Words: Imagine a chain of paper clips linking a big wish to a medium wish to a tiny task. If any clip in the chain is missing, the tiny task never gets done — even though it should have. Tracing checks every clip so nothing important slips through.

Example — Homework Helper App: Mr. Chen shares the school's big goal: "Cut late homework submissions by 30% this semester" (a business requirement). Maya turns that into something a student needs, written as Use Case UC-1: "As a student, I want to see tomorrow's homework so I don't forget it" (a user requirement). Then Sam needs an exact instruction, like FR-1: "The app shall show a red badge on the icon when homework is due within 24 hours" (a functional requirement). When Maya checks her tracing chart, she notices UC-1 has no functional requirement for "repeating reminders" — so she catches a missing requirement before Sam ever starts coding! This is exactly the check tracing is built for: every business requirement should lead to at least one user requirement, and every user requirement should lead to at least one functional requirement — any gap in that chain is a missing requirement hiding in plain sight.

> 📖 **New Word!** A business requirement means the big-picture goal of the whole project, like "reduce late homework."

> 📖 **New Word!** A user requirement means what a specific person needs the app to do for them, often written as a use case.

> 📖 **New Word!** A functional requirement means an exact, detailed instruction telling Sam precisely what to build, like “FR-1: show a red badge.”

> 📖 **New Word!** A use case means a specific example of how someone uses the app to reach a goal, written almost like a mini-story.

🧠 **Remember It Like This:** No broken links — every big wish needs a small task, or the wish falls through the cracks.

FINDING UNNECESSARY REQUIREMENTS

In Simple Words: It’s like a pizza arriving with mushrooms nobody ordered. If a feature doesn’t trace back to anything anyone actually asked for, it might just be wasted time, toppings, and money.

Example — Homework Helper App: Sam spends a weekend building a fun confetti animation that plays whenever homework is marked done. When Maya checks the trace chart, she can’t find any use case or requirement it connects to — nobody asked for it. Mr. Chen says, “It’s neat, but we didn’t ask for it, and it cost us a whole weekend we needed for reminders.” That’s exactly what tracing catches — sometimes called scope creep or gold-plating, when extra stuff sneaks into the project.

> 📖 **New Word!** Scope creep (also called gold-plating) means extra features sneaking into a project that nobody actually requested.

🧠 **Remember It Like This:** If it doesn’t trace back, cut it back.

CERTIFICATION AND COMPLIANCE

In Simple Words: Some rules aren’t optional — like a lunchroom rule that everyone must wash their hands before eating. Tracing gives you proof, like a receipt, that you actually followed the required rules.

Example — Homework Helper App: The school district has a privacy rule saying apps can’t collect a student’s full name without a parent’s permission. Maya writes this as FR-20 and links it forward to Sam’s actual signup-screen code and Priya’s privacy test. When a school district reviewer later asks, “Did you follow our privacy rule?” Maya pulls up the trace link as instant proof — no digging required.

> 📖 **New Word!** Compliance means following official rules, like a school district’s privacy policy or a government regulation.

🧠 **Remember It Like This:** Traces are your receipts — proof you didn’t skip the rules.

CHANGE IMPACT ANALYSIS

In Simple Words: Pull one loose thread on a sweater, and you can see the whole sweater wobble. Tracing shows you the whole wobble before you even pull the thread.

Example — Homework Helper App: Maya wants to change UC-2 (reminder timing) from “1 day before” to “3 days before.” Using her trace links, she instantly sees this touches FR-3 (the notification rule), the ReminderManager code, and tests reminder.5 and reminder.6 — so Sam and Priya both know exactly what

needs updating before Maya even finishes explaining the change. Without those trace links, it would be easy to overlook one of the affected pieces entirely.

> 📖 **New Word!** Impact analysis means figuring out everything that will be affected before you make a change.

🧠 **Remember It Like This:** Pull one thread, see the whole sweater.

MAINTENANCE

In Simple Words: It's like keeping your LEGO instruction booklet neat, so that months later, when a piece breaks, you can fix exactly the right part instead of taking the whole set apart.

Example — Homework Helper App: A year later, the school district changes its grading policy, requiring new assignment categories. Because Maya's trace links are reliable, she can search for every requirement that mentions "assignment categories" and update all of them correctly and completely, instead of guessing which ones matter.

🧠 **Remember It Like This:** Good traces make future fixes fast, not scary.

PROJECT TRACKING

In Simple Words: It's like a treasure hunt scoreboard that shows exactly how many clues your team has actually solved — not just a guess of "we're pretty close."

Example — Homework Helper App: Mr. Chen asks, "How much of the app is actually done?" Maya pulls up her trace matrix: 18 out of 25 functional requirements have both finished code AND a passing test linked to them — that's 72% done, for real, not a guess. The trace data collected all through development gives an honest, accurate record of exactly how much planned functionality is actually implemented.

🧠 **Remember It Like This:** Traces turn "I think we're close" into "we're 72% done, for real."

REENGINEERING

In Simple Words: Before you tear down an old treehouse to build a new one, you should list everything the old treehouse could do — a rope ladder, a lookout window — so the new one doesn't accidentally leave something out.

Example — Homework Helper App: The school used to use a paper planner (an old, existing system called a legacy system). Maya lists every feature the paper planner had — a weekly grid, a spot for teacher initials — and traces each one to where it now lives in Homework Helper's requirements and components. She discovers "teacher initials box" has no digital replacement yet, so she adds FR-19 to cover it.

> 📖 **New Word!** A legacy system means an older system that's already in use, which a new system might replace or must work alongside.

🧠 **Remember It Like This:** Before you replace something old, map out everything it used to do.

REUSE

In Simple Words: It's like keeping a full LEGO mini-set — instructions, pieces, and box — together, so you can hand the whole thing to a friend instead of digging through a giant bin for loose parts.

Example — Homework Helper App: Maya realizes the whole traced package for "homework reminders" — requirement, design, code, and test — is basically what's needed for "test reminders" too. She reuses the entire bundle instead of starting from scratch.

🧠 **Remember It Like This:** Trace it once, reuse the whole bundle forever.

TESTING

In Simple Words: It's like a treasure map with an X — when something goes wrong, the map points straight to the spot to dig, instead of making you search the whole island.

Example — Homework Helper App: Priya runs test "reminder.5" and it fails. She checks the trace link, which points straight to FR-3 and the exact code method, `sendDueSoonAlert()`, inside ReminderManager. She tells Sam precisely where to look for the defect instead of him searching the entire app.

> 📖 **New Word!** A defect (also called a bug) means a mistake in the code that makes the app do something wrong.

🧠 **Remember It Like This:** Trace links turn "good luck finding it" into "look right here."

🎯 **Quick Recap:**

- Finding missing requirements — gaps in the chain reveal forgotten requirements.
- Finding unnecessary requirements — anything that doesn't trace back might not be needed.
- Certification and compliance — traces prove you followed the required rules.
- Change impact analysis — traces show everything a change will touch before you touch it.
- Maintenance — reliable traces make future changes correct and complete.
- Project tracking — traces give an honest percentage of what's really done.
- Reengineering — traces map old-system features onto the new system.
- Reuse — traces bundle related requirements, design, code, and tests for reuse.
- Testing — traces point straight from a failed test to the likely defect.

The Requirements Traceability Matrix (RTM)

WHAT IS A REQUIREMENTS TRACEABILITY MATRIX (RTM)?

In Simple Words: Think of a subway map that shows every station connecting to the next one. An RTM is a map like that, but for requirements — a table showing how each functional requirement links backward to the specific use case it came from, and forward to one or more design, code, and test elements.

> 📖 **New Word!** A matrix (just like in math class) means a grid of rows and columns, like a multiplication table — perfect for showing how things connect.

Example — Homework Helper App: Maya builds a big table. One row says: functional requirement "reminder.notify.dueSoon" links backward to UC-1 (the use case) and links forward to the ReminderManager design, the `sendDueSoonAlert()` code, and test reminder.3.

🧠 **Remember It Like This:** The RTM is the map that shows how every requirement connects, backward and forward.

TABLE 29-1 STYLE: ONE COMBINED MATRIX

In Simple Words: It's like writing one long recipe card with every step for a dish on it, instead of flipping between five separate cards to make one meal.

Example — Homework Helper App: Maya builds her own version of this style:

User requirement	Functional requirement	Design element	Code element	Test
UC-1	reminder.notify.dueSoon	Class ReminderManager	sendDueSoonAlert()	reminder.3, reminder.4
UC-5	homework.list.filterBySubject	Class AssignmentList	FilterBySubject()	list.9, list.10

Here's what the real textbook's version of this table looks like (TABLE 29-1, from a library catalog project):

User requirement	Functional requirement	Design element	Code element	Test
UC-28	catalog.query.sort	Class catalog	CatalogSort()	search.7, search.8
UC-29	catalog.query.import	Class catalog	CatalogImport(), CatalogValidate()	search.12, search.13, search.14

🖼️ **What this picture is telling you:** Each row is one requirement's whole life story — where it came from (a use case), what part of the program handles it (design and code), and how we check it works (tests) — all on one line.

🧠 **Remember It Like This:** One row, whole story — from wish to working code.

TABLE 29-2 STYLE: MULTIPLE SEPARATE MATRICES

In Simple Words: It's like sorting LEGO pieces by type — all the wheels in one bin, all the windows in another — instead of dumping every part from every set into one giant heap. You make separate, simpler grids for separate jobs.

Example — Homework Helper App: Maya makes a simple grid connecting just her use cases and functional requirements, with no code or test columns cluttering it up. Here's what the real textbook's version looks like (TABLE 29-2), showing functional requirements FR-1 through FR-6 linked to use cases UC-1

through UC-4:

Functional Requirement	UC-1	UC-2	UC-3	UC-4
FR-1	↵			
FR-2	↵			
FR-3			↵	
FR-4			↵	
FR-5		↵		↵
FR-6			↵	

🖼️ **What this picture is telling you:** Each arrow shows which use case a functional requirement came from. FR-5 even has two arrows, because it came from two different use cases (UC-2 and UC-4).

🧠 **Remember It Like This:** Separate simple grids beat one giant messy table when you only need one kind of link.

SUSPECT LINKS

In Simple Words: Imagine your recipe changed from “chocolate cake” to “vanilla cake.” Every step that mentioned chocolate now needs a check mark next to it, to make sure it still makes sense.

Example — Homework Helper App: Maya’s team changes UC-3 (“mark assignment done”) so teachers can now give partial credit. The moment that happens, every functional requirement linked to UC-3 gets flagged as a suspect link — a warning that it might need a review, not proof that it’s broken. Here’s the real textbook’s version (TABLE 29-4), showing the same grid as before after UC-3 changed:

Functional Requirement	UC-1	UC-2	UC-3	UC-4
FR-1	↵			
FR-2	↵			
FR-3			↵?	
FR-4			↵?	
FR-5		↵		↵
FR-6			↵?	

> 📖 **New Word!** A suspect link means a trace connection flagged for review because something it’s linked to just changed.

🖼️ **What this picture is telling you:** The plain arrows (FR-1, FR-2, FR-5) are untouched and fine. But FR-3, FR-4, and FR-6 now show an arrow-plus-question-mark — the tool’s way of shouting “double-check me, my use case just changed!” Maya assigns Sam to review each flagged one.

🧠 **Remember It Like This:** A question-mark arrow means “look again,” not “something’s broken.”

🎯 **Quick Recap:**

- Definition of the RTM — a table connecting every functional requirement backward to its use case and forward to design, code, and test elements.
- Table 29-1 style — one combined row carries the whole chain for a requirement.
- Table 29-2 style — separate simple grids just for one kind of link.
- Suspect links — a flag telling the team to double-check requirements after something they’re linked to changes.

A Requirements Tracing Procedure

Once Maya’s team decides tracing matters, they need a real plan for setting it up. Here are the nine steps, in order.

STEP 1: EDUCATE THE TEAM AND MANAGEMENT

In Simple Words: Before starting a board game with new rules, everyone should learn the rules together first — otherwise someone breaks a rule by accident.

Example — Homework Helper App: Maya calls a team meeting with Sam, Priya, and Mr. Chen. She explains the tracing concepts and why they matter, shows exactly where the trace data will live, walks through how links get made, and asks each person to commit to keeping their part updated.

🧠 **Remember It Like This:** Teach the rules before you start the game.

STEP 2: SELECT WHICH LINK RELATIONSHIPS TO DEFINE

In Simple Words: It’s like choosing to track only 3 important chores closely this week, instead of trying to micromanage all 10 — pick what matters most first.

Example — Homework Helper App: There are many possible kinds of links Maya’s team could trace — use case to functional requirement, requirement to design, design to code, code to test, and more. Maya decides they’ll trace use case → functional requirement → test for now, and skip tracing to detailed design diagrams, since Sam is coding solo and doesn’t need that extra layer yet.

🧠 **Remember It Like This:** Don’t try to trace everything at once — you’ll drown.

STEP 3: CHOOSE THE MATRIX STYLE AND STORAGE MECHANISM

In Simple Words: Decide whether you’ll use one big combined card (Table 29-1 style) or several smaller cards (Table 29-2 style) — and decide where those cards will actually live: a table in a plain document, a spreadsheet, or special software.

Example — Homework Helper App: Maya considers just typing a table into their Word document, but picks the “several matrices” style and starts with a shared spreadsheet instead, since it’s easier to update than a document. She tells the team that if Homework Helper grows bigger, they should upgrade to a real requirements management (RM) tool — by far the best of the three options.

> 📖 **New Word!** An RM tool means special software built just to store, link, and track requirements — far better than a plain document for big projects.

🧠 **Remember It Like This:** Pick your table style and your storage box before you start collecting cards.

STEP 4: IDENTIFY WHICH PARTS OF THE PRODUCT NEED TRACEABILITY

In Simple Words: Not every part of a treehouse needs a detailed blueprint — the rope ladder that holds your weight does, but the paint color doesn’t.

Example — Homework Helper App: Maya decides they’ll definitely trace the “reminder notifications” feature — it’s a critical core function, and it’s likely to keep changing and getting more attention — but won’t bother tracing the “app color theme” (low risk, unlikely to change, and not expected to need much maintenance).

🧠 **Remember It Like This:** Trace the risky, important stuff first — not the wallpaper.

STEP 5: IDENTIFY WHO SUPPLIES EACH TYPE OF LINK, AND THE COORDINATOR

In Simple Words: Like a group project where each teammate is responsible for their own slide, but one person double-checks that all the slides actually fit together.

Example — Homework Helper App: Sam supplies the design-and-code links, Priya supplies the test links, and Maya (the Business Analyst) is the coordinator who manages the whole tracing activity and makes sure everything connects properly.

🧠 **Remember It Like This:** Everyone links their own piece; one person makes sure it all fits.

STEP 6: MODIFY DEVELOPMENT PROCEDURES TO UPDATE LINKS PROMPTLY

In Simple Words: It’s easier to put your shoes away right after taking them off than to hunt for them under the couch a month later.

Example — Homework Helper App: Sam adds a rule to his own checklist: “Right after I finish coding a requirement, I update the trace sheet — before I move to the next task,” instead of waiting two weeks and forgetting which method did what.

🧠 **Remember It Like This:** Update the link the moment the work is done, not “someday.”

STEP 7: DEFINE LABELING CONVENTIONS

In Simple Words: Like giving everyone at a birthday party a name tag, so nobody gets mixed up about who's who.

Example — Homework Helper App: The team agrees every use case gets a unique label like "UC-#," every functional requirement gets "FR-#," and every test gets a name like "reminder.3." Now when Priya says "reminder.3 failed," everyone instantly knows exactly what she means, and it can be linked cleanly to everything else.

🧠 **Remember It Like This:** Give everything a name tag so nobody gets confused at the party.

STEP 8: PROVIDE TRACE INFO AS SMALL UNITS OF WORK ARE COMPLETED

In Simple Words: It's like putting away one toy after you finish playing with it, rather than leaving a giant mess to clean up at the very end of the day.

Example — Homework Helper App: Instead of waiting until the whole app is finished, Sam updates the trace sheet right after finishing the reminders feature this week, and again after finishing the assignment list next week — a little bit at a time, instead of trying to assemble it all at the very end.

🧠 **Remember It Like This:** A little bit of tracing every day beats a giant scramble at the end.

STEP 9: AUDIT THE TRACE INFORMATION PERIODICALLY

In Simple Words: Like checking your piggy bank once a month to make sure the amount you wrote down still matches what's actually inside.

Example — Homework Helper App: Every other Friday, Maya spot-checks the trace sheet. She finds FR-9 marked "done and tested" — but it has no test link at all. That's a red flag that the tracing process isn't being followed correctly, so she talks to Priya about it right away.

> 📖 **New Word!** To audit means to carefully check something to make sure it's accurate and being done correctly.

🧠 **Remember It Like This:** Check your traces regularly, or "done" might really mean "not really done."

🎯 **Quick Recap:**

- Step 1: Educate the team and management before starting.
- Step 2: Select which link relationships to trace — not everything at once.
- Step 3: Choose the matrix style and where trace data will be stored.
- Step 4: Identify which parts of the product actually need tracing.
- Step 5: Identify who supplies each link, and who coordinates it all.
- Step 6: Update development procedures so links get updated promptly.
- Step 7: Define labeling conventions so every element has a unique name.
- Step 8: Add trace info continuously as small work units finish, not all at the end.
- Step 9: Audit the trace data periodically to make sure it's actually current.

Limitations of the Document-Based Approach to Tracing

Before RM tools existed — and sometimes even today — teams tried tracing requirements using plain documents, like Word files or spreadsheets. Here's why that's like trying to build a LEGO castle with the instructions scattered across ten different rooms.

DIFFICULT TO KEEP DOCUMENTS CURRENT AND SYNCHRONIZED

In Simple Words: It's like keeping five grocery lists on five different fridges — someone always forgets to update one, and now they don't match.

Example — Homework Helper App: Maya updates the "official" requirements document on her laptop, but Sam still has last week's version open on his computer, and codes the outdated version by mistake.

🧠 **Remember It Like This:** Paper documents drift apart like boats without anchors.

COMMUNICATING CHANGES TO ALL AFFECTED TEAM MEMBERS IS A MANUAL PROCESS

In Simple Words: A document can't tap anyone on the shoulder — if you want people to know about a change, you have to walk over and tell them yourself, every single time.

Example — Homework Helper App: When Mr. Chen changes the due-date rule, Maya has to personally message Sam AND Priya to let them know — there's no automatic "everyone affected gets notified" button in a Word document.

🧠 **Remember It Like This:** Documents don't tap people on the shoulder — you have to do it yourself, every single time.

NOT EASY TO STORE SUPPLEMENTARY ATTRIBUTES ABOUT EACH REQUIREMENT

In Simple Words: A plain document is like a trading card with only a name on it — no stats for speed or power. A good tool holds the full stat sheet.

Example — Homework Helper App: Maya wants to track, for every requirement, who requested it, how risky it is, and its priority — but cramming all that extra information into a Word paragraph gets messy fast.

> 📖 **New Word!** Attributes mean extra facts about something — like a requirement's priority, who requested it, or how risky it is.

🧠 **Remember It Like This:** A plain document is a name tag; a good tool is a full trading card.

HARD TO DEFINE LINKS BETWEEN REQUIREMENTS AND OTHER SYSTEM ELEMENTS

In Simple Words: Words printed on a page can't reach out and hold hands with words on another page.

Example — Homework Helper App: In her Word document, Maya can't click "FR-3" and instantly jump to the code and test connected to it — she just has to remember, or scroll and search.

🧠 **Remember It Like This:** Words on a page can't hold hands with other words on other pages.

TRACKING STATUS OF INDIVIDUAL REQUIREMENTS AND THE WHOLE SET IS CUMBERSOME

In Simple Words: Counting jellybeans one by one takes forever compared to a machine that counts them for you instantly.

Example — Homework Helper App: Maya wants to know how many of the 25 requirements are "done." In a Word document, she has to read through every single one and manually count. A real tool would show her instantly.

🧠 **Remember It Like This:** Counting by hand is slow; a real tool counts for you.

CONCURRENTLY MANAGING REQUIREMENT SETS PLANNED FOR DIFFERENT RELEASES IS TRICKY

In Simple Words: Moving furniture between two rooms without a dolly means a lot of heavy manual lifting.

Example — Homework Helper App: The team wants "reminder notifications" in Version 1 (due next month) but "study group finder" in Version 2 (next semester). If they decide to push a Version 1 feature to Version 2, someone has to manually cut and paste it into a different document to keep the two release specs straight.

🧠 **Remember It Like This:** Moving a requirement between versions by hand is like moving furniture without a dolly.

REUSING A REQUIREMENT MEANS MANUALLY COPYING TEXT FROM ONE DOCUMENT INTO ANOTHER

In Simple Words: Copy-pasting a recipe into a new cookbook means you now have two copies — and if you improve one later, the other one never finds out.

Example — Homework Helper App: When a sister school wants their own version of Homework Helper, Maya can't just "link" to the reminder requirement — she has to copy-paste the text into a brand-new document, and if the original ever improves, the copy won't automatically update.

🧠 **Remember It Like This:** Copy-paste reuse means two copies that can quietly grow apart.

DIFFICULT FOR MULTIPLE, ESPECIALLY GEOGRAPHICALLY SEPARATED, PARTICIPANTS TO MODIFY REQUIREMENTS TOGETHER

In Simple Words: If five kids try to color the same single piece of paper at the same time, someone's crayon marks are going to get scribbled over.

Example — Homework Helper App: Maya is at school, Sam is coding at home, and Priya is testing at her grandma's house across town. If all three try editing the same Word document at once, someone's changes might overwrite someone else's.

🧠 **Remember It Like This:** One document, many hands, equals many accidents.

NO CONVENIENT PLACE TO STORE REJECTED/DELETED REQUIREMENTS

In Simple Words: Once you erase a pencil line, there's no note left explaining why you erased it.

Example — Homework Helper App: The team decided against a "sell Homework Helper to other schools" feature (too big for now). In their plain document, once they delete that paragraph, it's just gone — no record of what was considered and why it was turned down.

🧠 **Remember It Like This:** A document has no memory bin for the ideas you said "not now" to.

IDENTIFYING MISSING, DUPLICATE, AND UNNECESSARY REQUIREMENTS IS DIFFICULT

In Simple Words: It's easy to miss that you wrote the same joke twice in a 40-page story — the bigger the pile of words, the easier it is to hide a duplicate.

Example — Homework Helper App: Buried in a 40-page requirements document, nobody notices that "FR-8: notify students of overdue homework" and "FR-15: alert students about late assignments" are actually the exact same requirement written twice.

🧠 **Remember It Like This:** Duplicates and gaps hide easily in a giant wall of text.

🎯 **Quick Recap:**

- Documents are hard to keep current and synchronized across the team.
- Communicating changes to everyone affected is a fully manual job.
- Documents can't easily hold extra attributes about each requirement.
- Documents can't easily link requirements to other system elements.
- Tracking status by hand, for one requirement or all of them, is cumbersome.
- Managing requirements across different releases at once is tricky by hand.
- Reusing a requirement means manually copying text, which drifts out of sync.
- Multiple people editing the same document, especially from far apart, causes conflicts.
- There's no good place to keep rejected or deleted requirements.
- Finding missing, duplicate, or unnecessary requirements is hard in a big document.

Requirements Development Tools

These are different from RM tools — RD tools help you create and shape requirements in the first place, before you ever start tracing them.

ELICITATION TOOLS

In Simple Words: It's like bringing a recorder or camera to a birthday party, so you capture every fun moment instead of trying to remember it all later.

Example — Homework Helper App: During her interview with Mr. Chen, Maya uses a note-taking app on her tablet to record his exact words, like "Students keep forgetting field trip permission slips too" — so she doesn't misremember it later.

> 📖 **New Word!** Elicitation means the process of asking questions and gathering information from people to figure out what they need.

🧠 **Remember It Like This:** Capture it as it happens — memory fades, notes don't.

PROTOTYPING TOOLS

In Simple Words: It's like building a cardboard mock-up of a treehouse before nailing real wood together, so you can see if the design even makes sense first. Real prototyping tools range from simple electronic mock-ups all the way up to full working simulations of the whole app.

Example — Homework Helper App: Before Sam writes any real code, Maya sketches a clickable mock-up of the reminders screen using a simple prototyping app, and shows it to five students to get their reactions before Sam builds the real thing.

> 📖 **New Word!** A prototype means a rough, early model of something you're building, used to test ideas before doing the real work.

🧠 **Remember It Like This:** Try the cardboard treehouse before you build the real one.

MODELING TOOLS (E.G., UML)

In Simple Words: It's like drawing a blueprint of a house before building it, so the builders know exactly where every wall and door goes.

Example — Homework Helper App: Sam uses a modeling tool to draw a diagram showing how the "Assignment" part of the code connects to the "Reminder" part, so he plans the structure before typing a single line.

> 📖 **New Word!** UML (Unified Modeling Language) means a set of standard diagrams engineers use to draw blueprints of software.

🧠 **Remember It Like This:** Blueprints before bricks — draw the plan before you build the app.

🎯 **Quick Recap:**

- Elicitation tools help capture what stakeholders say during requirements-gathering.
- Prototyping tools let you try a rough mock-up (or a full simulation) before building the real thing.
- Modeling tools (like UML) let you draw a blueprint of the software's structure.

Requirements Management (RM) Tools and Their Benefits

MANAGE VERSIONS AND CHANGES

In Simple Words: It's like a video game's save history, letting you see every past save and exactly what changed between them.

Example — Homework Helper App: When Maya edits FR-3's wording, the RM tool automatically keeps a version history — she can see exactly what it said last Tuesday, and who changed it.

🧠 **Remember It Like This:** The tool remembers every version, so you don't have to.

STORE REQUIREMENTS ATTRIBUTES

In Simple Words: Now imagine that trading card finally gets its full stat sheet — priority, risk, who asked for it, all searchable.

Example — Homework Helper App: For FR-3, the RM tool has fields for Priority (High), Requested By (Mr. Chen), Risk (Medium), and Status (In Progress) — all searchable, unlike a paragraph buried in a Word doc.

🧠 **Remember It Like This:** Real trading cards, not just name tags — the tool holds all the stats.

FACILITATE IMPACT ANALYSIS

In Simple Words: Instead of hunting page by page for every thread connected to the one you pulled, the tool shows you the whole ripple in one click.

Example — Homework Helper App: When Maya wants to change UC-2, the tool instantly shows her every linked functional requirement, design element, and test — no manual hunting required.

🧠 **Remember It Like This:** One click shows the whole ripple, not just the splash.

IDENTIFY MISSING AND EXTRANEOUS REQUIREMENTS

In Simple Words: It's like a flashlight that shines into every dark corner of your closet, showing you both the empty hangers and the clothes you never needed.

Example — Homework Helper App: The tool runs a report showing which use cases have zero linked functional requirements (missing!) and which functional requirements link to no use case at all (extra, maybe scope creep).

🧠 **Remember It Like This:** The tool shines a flashlight into the gaps and the extras.

TRACK REQUIREMENTS STATUS

In Simple Words: It's a scoreboard that updates itself the instant something changes, instead of you tallying it by hand.

Example — Homework Helper App: Maya filters the tool by “Status = Not Started” and instantly sees 7 requirements Sam hasn’t touched yet — no manual counting needed.

🧠 **Remember It Like This:** Status at a glance, not status by guesswork.

CONTROL ACCESS

In Simple Words: Not every player at the party needs a key to every room in the house — some rooms are open to everyone, some are locked.

Example — Homework Helper App: The RM tool lets Maya give Sam permission to edit code-related fields, but only “view” permission on requirements Mr. Chen has already approved, so nobody accidentally changes something they shouldn’t.

> 📖 **New Word!** Access control means deciding who is allowed to view, edit, or delete something — not everyone gets the same permissions.

🧠 **Remember It Like This:** Not everyone needs the keys to every room.

COMMUNICATE WITH STAKEHOLDERS

In Simple Words: Instead of running around the house telling everyone the news yourself, imagine a magic doorbell that rings automatically for exactly the right people.

Example — Homework Helper App: The RM tool automatically emails Mr. Chen every time a requirement he cares about changes status, so he’s always in the loop without Maya typing a separate message each time.

🧠 **Remember It Like This:** The tool taps stakeholders on the shoulder automatically.

REUSE REQUIREMENTS

In Simple Words: It’s like handing a friend your whole labeled LEGO mini-set — instructions, pieces, and box — instead of just describing it to them from memory.

Example — Homework Helper App: When a sister campus wants their own version of Homework Helper, Maya copies the entire linked “bundle” of the reminder requirement — with its design, code, and test links intact — into the new project, right inside the tool.

🧠 **Remember It Like This:** Reuse the whole traced bundle, not just the words.

TRACK ISSUE STATUS

In Simple Words: It’s like a lost-and-found board where every reported problem gets a tag showing whether it’s still lost, being looked for, or found.

Example — Homework Helper App: Priya logs a bug she found — “Reminder shows the wrong date” — as an issue inside the RM tool and links it to FR-3. Everyone can see whether it’s Open, In Progress, or Fixed.

> 📖 **New Word!** An issue means a problem, bug, or open question that needs to get resolved.

🧠 **Remember It Like This:** Every issue gets a visible tag, from “uh-oh” to “all fixed.”

GENERATE TAILORED SUBSETS

In Simple Words: It’s like asking a librarian, “Show me only mystery books written after 2020,” instead of reading every single book on every shelf yourself.

Example — Homework Helper App: Maya asks the RM tool to show her “all High-priority requirements requested by Mr. Chen that aren’t tested yet,” and it instantly filters the whole list for her, instead of her reading all 40 requirements by hand.

> 📖 **New Word!** A query means a specific question you ask a database, like “show me only the requirements that match these rules.”

🧠 **Remember It Like This:** Ask the tool a smart question, get exactly the pile you need.

🎯 **Quick Recap:**

- Manage versions and changes — the tool keeps history so you don’t have to.
- Store requirements attributes — extra facts stay attached to each requirement.
- Facilitate impact analysis — see the whole ripple of a change in one click.
- Identify missing and extraneous requirements — a built-in flashlight for gaps and extras.
- Track requirements status — an instant, self-updating scoreboard.
- Control access — set who can view, edit, or delete what.
- Communicate with stakeholders — automatic notifications instead of manual messages.
- Reuse requirements — copy a whole linked bundle instead of retyping text.
- Track issue status — every bug or open question gets a visible tag.
- Generate tailored subsets — ask a smart question, get exactly the list you need.

RM Tool Features (Figure 30-1)

RM TOOL FEATURE FAMILIES

In Simple Words: An RM tool is like a Swiss Army knife — one tool, but with lots of different blades and gadgets for different jobs. Figure 30-1 groups those “blades” into six families, all connected to the central tool.

Example — Homework Helper App: When Maya and Sam explore their new RM tool, they realize it isn’t just “one big list.” It’s a whole toolbox: some features for managing requirements day to day, some for capturing brand-new ones, some for tracing links, some for spitting out reports and connecting to other tools, and even built-in help.

Here’s the real fishbone-style diagram from the textbook, recreated as a table:

Category ("Bone")	Features
Manage Requirements	Review Requirements; Manage Permissions; Baseline Requirements; Allocate Requirements to Systems; Delete Requirements; Organize Requirements by Groups; Manage History of Requirements; Create Custom Views of Requirements; Track Requirements Status; Manage Workflows
Capture Requirements	Define Requirements Architecture; Add Requirements; Edit Requirements; Collaborate on Requirements; Define Requirements Attributes; Import Requirements; Assign Requirements Types
Access Tool Help	Support within Tool; Training is Included
Trace Requirements	User Documentation; Trace Requirements Dependencies; Trace Requirements; Associate Requirements to Models
Output Requirements	Create Document Templates; Export Requirements to Documents
(Extra output-side features)	Create Reports; Integrate to Testing Tools; Integrate to Development Tools; Integrate to Other Tools

> 📖 **New Word!** To baseline a requirement means to lock in an approved version as the official one, so any future change has to be tracked as a real change, not quietly edited.

> 📖 **New Word!** A workflow means the series of steps something (like a requirement) goes through, like Draft → Reviewed → Approved → Done.

🖼️ **What this picture is telling you:** This looks like a fishbone skeleton — the "spine" is the RM Tool itself, and each "rib" bone is a whole family of related features, from capturing brand-new requirements, to tracing them, to printing a neat final report.

🧠 **Remember It Like This:** One tool, six toolboxes — managing, capturing, help, tracing, outputting, and reporting/integrating — from capturing an idea to tracing it to printing the final report.

🎯 **Quick Recap:**

- RM tool features (Figure 30-1) — one central tool, six connected families of features: managing, capturing, help, tracing, outputting, and reporting/integrating with other tools.

RM Tool Capabilities and Integration With Other Tools (Figure 30-2)

RM TOOLS INTEGRATE WITH OTHER KINDS OF TOOLS (FIGURE 30-2)

In Simple Words: A smartphone doesn't just make calls — it also talks to your smartwatch, your car speakers, and your smart TV. An RM tool doesn't work alone either; it constantly talks back and forth with other tools around it.

Example — Homework Helper App: Homework Helper's RM tool sends Sam's finished links to a version control tool (where his actual code lives), sends test-related requirements to Priya's test management tool, and receives new bug reports from an issue-tracking tool — all automatically, in both directions.

Here's the real diagram from the textbook, recreated as a table:

Connected Tool Type	What It Does
Requirements Development Tool	Helps create and elicit brand-new requirements
Version Control Tool	Keeps track of the actual code files and their history
Design Modeling Tool	Holds diagrams (like UML) showing how the software is structured
Issue-Tracking Tool	Tracks bugs and open problems
Change-Request Tool	Tracks formal requests to change something already approved
Test Management Tool	Tracks test cases and their pass/fail results

> 📖 **New Word!** Version control means a system that saves every change made to code over time, so you can always go back and see (or undo) earlier versions.

> 📖 **New Word!** A change request means a formal ask to modify something that's already been approved.

🖼️ **What this picture is telling you:** The Requirements Management Tool sits in the middle like a hub, with double-headed arrows to six other tools around it — information flows both in and out, with every single one of them.

🧠 **Remember It Like This:** No tool is an island — the RM tool is the central hub that keeps everyone's tools talking.

🎯 **Quick Recap:**

- RM tool integration (Figure 30-2) — the RM tool is a two-way hub connected to requirements-development, version control, design modeling, issue-tracking, change-request, and test management tools.

Setting Up the Tool and Processes

Once Maya's team decides to really use an RM tool, they need a smart rollout plan.

ASSIGN AN EXPERIENCED BA TO OWN TOOL SETUP AND PROCESS ADAPTATION

In Simple Words: Let the person who best understands "why" a rule matters be the one who decides "how" to set it up.

Example — Homework Helper App: Maya, the most experienced at gathering requirements, is put in charge of setting up the RM tool's categories and rules for the whole team, since she understands the impact a wrong setting or process choice could have on everyone later.

🧠 **Remember It Like This:** Let the person who understands the "why" set up the "how."

THINK CAREFULLY ABOUT THE REQUIREMENT TYPES YOU DEFINE

In Simple Words: Don't make 30 tiny labeled folders for every little section of your plan, but don't dump everything into one giant junk drawer either — find the sensible middle.

Example — Homework Helper App: Maya doesn't create a separate type for every section of their SRS (the official document listing all requirements) — that would mean 30 tiny folders. But she also doesn't dump business, user, and functional requirements all into one giant bucket. She picks a sensible middle ground: Business, User, and Functional types.

> 📖 **New Word!** An SRS (Software Requirements Specification) means the big official document that lists out all of a project's requirements.

🧠 **Remember It Like This:** Not 30 tiny folders, not 1 giant junk drawer — find the middle.

USE THE TOOL TO FACILITATE COMMUNICATION WITH STAKEHOLDERS IN VARIOUS LOCATIONS

In Simple Words: Open the doors wide enough to let good ideas in, but not so wide that anyone can wander in and wreck the place.

Example — Homework Helper App: Since Priya tests from her grandma's house and Sam codes from his bedroom, the RM tool lets both log in from anywhere to comment on requirements — but Maya sets access and change privileges so testers can comment, not delete things, giving everyone enough input without handing out full edit freedom to all.

🧠 **Remember It Like This:** Open the doors wide enough to let ideas in, not so wide anyone can wreck the place.

DON'T TRY TO CAPTURE REQUIREMENTS DIRECTLY IN THE RM TOOL DURING EARLY ELICITATION WORKSHOPS

In Simple Words: Brainstorm on scrap paper first; move ideas to the real filing cabinet only once they calm down and stop changing every five minutes.

Example — Homework Helper App: During her first brainstorming session with 10 students, Maya uses sticky notes and a whiteboard, not the RM tool, because typing into the tool live would slow the fast-moving conversation down. Once the ideas start to settle, she types them into the RM tool — and once they're in there, the whole team can see them and keep refining them together, instead of Maya being the only one who has the full picture.

🧠 **Remember It Like This:** Brainstorm on scrap paper first; move to the real filing cabinet once ideas calm down.

USE REQUIREMENTS-DEVELOPMENT (RD) TOOLS DURING ELICITATION ONLY IF CONFIDENT THEY WON'T SLOW DISCOVERY DOWN

In Simple Words: A fancy tool that interrupts a good conversation isn't helping — it's getting in the way.

Example — Homework Helper App: Maya considers using a fancy prototyping app during her interview with Mr. Chen, but decides against it — fiddling with the app might interrupt the flow of his ideas or waste his time — so she sticks to simple notes for that meeting instead.

🧠 **Remember It Like This:** A fancy tool that slows down the conversation isn't helping — it's hurting.

DON'T DEFINE TRACE LINKS UNTIL REQUIREMENTS STABILIZE

In Simple Words: Don't tie the ribbon on a present while someone is still rewrapping it — wait until it stops changing shape.

Example — Homework Helper App: While the "reminders" feature keeps changing every day (Mr. Chen keeps tweaking it), Maya waits to build its trace links, since linking too early would just mean redoing all that work over and over as the requirement evolves. Once it stays the same for two whole weeks, she finally links it to its use case, design, code, and tests.

🧠 **Remember It Like This:** Don't tie the ribbon until the present stops changing shape.

SET A CUTOVER DATE

In Simple Words: Pick a moving day, mark it on the calendar, and actually move everything on that day — not "sometime eventually."

Example — Homework Helper App: Maya announces: "Starting September 1st, the RM tool is the ONLY official place for requirements. Anything still sitting in a Google Doc after that date doesn't count until someone enters it into the tool." Setting that firm date is what actually speeds up the move from documents to the tool.

> 📖 **New Word!** A cutover date means a specific day when a team officially switches from an old way of doing something to a new way.

🧠 **Remember It Like This:** Pick a moving day, and actually move on that day.

🎯 **Quick Recap:**

- Assign an experienced BA to own the tool setup and process choices.
- Think carefully about requirement types — not too many, not too few.
- Use the tool to facilitate stakeholder communication, with sensible access limits.
- Don't capture requirements directly in the tool during early brainstorming.
- Only use RD tools during elicitation if they won't slow discovery down.

- Don’t define trace links until requirements have stabilized.
- Set a cutover date so the tool becomes the one official source.

Facilitating User Adoption

Setting up the tool is only half the job — getting people to actually use it is the other half.

IDENTIFY A TOOL ADVOCATE

In Simple Words: Every new tool needs a biggest fan — someone who learns it inside and out first, then helps everyone else fall in love with it too.

Example — Homework Helper App: Maya becomes Homework Helper’s official tool advocate. She learns every corner of the RM tool first, then teaches Sam and Priya, answering their questions and encouraging them to actually use it instead of going back to sticky notes. As more school projects start using the tool, she keeps mentoring new teams too.

> 📖 **New Word!** An advocate means someone who champions and supports something, helping other people understand and use it well.

🧠 **Remember It Like This:** Every new tool needs a biggest fan who helps everyone else fall in love with it too.

OVERCOME THE BELIEF THAT “THE TOOL WON’T ADD VALUE”

In Simple Words: Swap the complaint “this tool is pointless” for a real story about the time not having a tool actually hurt.

Example — Homework Helper App: Sam grumbles, “Why can’t I just keep using sticky notes?” Maya reminds him about the time they lost track of a requirement for two weeks and he built the wrong feature — and asks Sam to share his own frustrating sticky-note story too.

🧠 **Remember It Like This:** Trade “the tool is pointless” for a real story about the pain of not having one.

TRAIN USERS RATHER THAN EXPECTING THEM TO SELF-DISCOVER THE TOOL

In Simple Words: Left completely alone, people usually figure out the basics — but they miss the superpowers hiding just beneath the surface.

Example — Homework Helper App: Instead of just handing Sam and Priya a login and hoping they figure it out, Maya runs a 30-minute training session showing them features they’d never have found on their own, like the automatic suspect-link flags.

🧠 **Remember It Like This:** Left alone, people learn the basics; taught properly, they learn the superpowers.

DON'T EXPECT INSTANT RESULTS

In Simple Words: Test the water in the shallow end of the pool before you cannonball into the deep end.

Example — Homework Helper App: Maya doesn't bet the whole Homework Helper launch on the team using the RM tool perfectly right away. She treats this semester as a small pilot project on a lower-stakes part of the app to learn how much time it really takes to administer and support the tool, before recommending it for the school district's much bigger app next year.

> 📖 **New Word!** A pilot project means a small trial run of something new, done on a smaller or safer scale before using it everywhere.

🧠 **Remember It Like This:** Test the water in the shallow end before you dive into the deep end.

🎯 **Quick Recap:**

- Identify a tool advocate to master the tool and mentor everyone else.
- Overcome doubts about the tool's value with real stories, not just arguments.
- Train users properly instead of hoping they'll figure it out alone.
- Don't expect instant results — start with a small pilot project first.

Reference

WHERE THESE IDEAS COME FROM (WIEGERS & BEATTY)

In Simple Words: Just like Maya learns tips from a favorite teacher, none of the ideas in this chapter were made up on the spot — they come from a real book that real software teams study, kind of like a recipe book that professional chefs actually use.

Example — Homework Helper App: If Maya's team ever gets stuck deciding how to trace their requirements, they could open *Software Requirements* (Third Edition) by Karl Wiegers and Joy Beatty — the very same book real companies use to answer exactly these kinds of questions.

🧠 **Remember It Like This:** Even superheroes have training manuals — this chapter's ideas come from requirements engineering's favorite one.

🎯 **Quick Recap:**

- This chapter's ideas come from a real, trusted source: Karl Wiegers and Joy Beatty's *Software Requirements* (Third Edition).

🏆 Chapter Wrap-Up

Maya's 47 sticky notes didn't stand a chance against the mess of a real project — but by the end of this chapter, her team has a real system. Every requirement for Homework Helper now traces back to a real request from Mr. Chen or the students, and forward to Sam's actual code and Priya's actual tests. When something changes, suspect links warn them what to double-check. When a test fails, the trace points

straight at the problem. And instead of fighting with scattered documents, the team is slowly moving everything into a real RM tool, with Maya as the patient advocate helping everyone trust it. Tracing didn't just organize their sticky notes — it made Homework Helper a project they could actually finish, together.

If you remember ONLY these things…

- **Motivations for tracing** — tracing catches missing and unnecessary requirements, proves compliance, powers impact analysis, maintenance, and project tracking, and points straight from a failed test to the fix.
- **The RTM** — the traceability matrix is the map connecting every requirement backward and forward, and suspect links flag what to double-check after something changes.
- **The tracing procedure** — set up tracing step by step: educate the team, choose what and how to trace, assign owners, label things clearly, update often, and audit regularly.
- **Limitations of documents** — plain documents make poor trace-keepers: they drift out of sync, can't hold links or extra attributes, and hide duplicates.
- **Requirements development tools** — elicitation, prototyping, and modeling tools help shape requirements before they're ever locked in.
- **RM tools and their benefits** — a real RM tool manages versions, attributes, access, status, reuse, and issues, and answers smart questions automatically.
- **RM tool features** — a good RM tool is a whole toolbox: capturing, managing, tracing, and outputting requirements, with help built in.
- **RM tool integration** — RM tools are hubs, talking both ways with development, version control, design, testing, issue, and change-request tools.
- **Setting up and adopting the tool** — roll out a new tool thoughtfully, with a clear owner and cutover date, and win people over with a champion, real training, and patience.

Chapter 9: Improving Requirements Processes

Homework Helper Version 1.0 is live on students' phones at Riverside Middle School — but Maya, Sam, and Priya know it wasn't a smooth ride to get there. Sam rewrote the reminder feature three times, two requirements were completely missing until students complained, and nobody agreed on how new ideas should get added mid-project. This chapter is about how a team stops and asks, "How do we get BETTER at gathering and handling requirements?" — so Version 2.0 goes a lot more smoothly.

Why Bother Improving? (The Big Picture)

OBJECTIVE OF PROCESS IMPROVEMENT

> 📖 **New Word!** A **requirement** means something the software MUST do — like a rule the app has to follow.

> 📖 **New Word! Process improvement** means getting better, on purpose, at the steps you follow to do your work — not just hoping things go better by accident.

In Simple Words: Imagine your basketball team loses a big game. A good coach doesn't just shrug it off — the team watches the game film, figures out exactly what went wrong, and practices those exact moves before the next game. Getting better at requirements works the same way. The whole point is to spend less money and time BUILDING the software AND keeping it running afterward (that's called "maintaining" it) — while giving people more of what they actually needed all along.

Example — Homework Helper App: After V1.0 launched, Maya called a team meeting. "Sam rewrote the reminders three times, and two requirements were missing completely. Before V2.0, let's fix how we work." There are three ways her team can do that:

1. **Correcting problems** from V1.0 caused by process shortcomings (like having no rule for approving new feature requests)
2. **Anticipating and preventing** future problems (like training everyone in how to interview students *before* V2.0 starts)
3. **Adopting better practices** they've never tried before (like a requirements review checklist)

🧠 **Remember It Like This:** Watch the game film, fix the plays — that's how every team (and every requirements process) levels up.

🎯 **Quick Recap:**

- **Objective of Process Improvement** — the goal is to spend less (building AND maintaining) and deliver more, by correcting past problems, preventing future ones, and adopting better habits.

How Requirements Connect to Everything Else on the Project

HOW REQUIREMENTS RELATE TO OTHER PROJECT PROCESSES

In Simple Words: Picture a bicycle wheel. **Software Requirements** sits right in the hub, the center circle — and six spokes connect out to every other part of the project. Nothing spins without the hub.

Example — Homework Helper App: Maya realized requirements weren't just Priya's testing checklist — they touched planning, tracking, changes, testing, coding, AND the user guide. Here's the real picture, straight from the project:

Connected Process	What Requirements Give It	What It Gives Back to Requirements
Project Planning Process	serve as input to	adjusts scope
Project Tracking and Control Process	status is tracked by	requests changes in scope
Change Control Process	are the baseline for	modifies
Acceptance & System Testing Processes	are reference for	verify correct implementation of
Construction Process	are basis for; work products are traced to	verifies correctness of implementation
User Documentation Process	are basis for	—

🖼️ **What this picture is telling you:** Requirements are the hub of the whole project wheel — every other process either feeds them, follows them, or reports back to them.

🧠 **Remember It Like This:** Requirements sit in the middle of the wheel — every other process spins around them.

PROJECT PLANNING PROCESS

In Simple Words: It's like a recipe's shopping list — the recipe (requirements) tells you what ingredients you need, but sometimes you have to trim the recipe if you can't get everything in time.

Example — Homework Helper App: Maya's project plan needed 6 weeks and 3 people to build all 40 requirements. But when REQ-014 (group project reminders) turned out to need 2 extra weeks, Maya and Mr. Chen agreed to cut it from V1.0 and save it for V2.0. That's planning adjusting scope based on requirements — and requirements getting adjusted right back.

🧠 **Remember It Like This:** Requirements tell the plan what's needed; the plan tells requirements what actually fits.

PROJECT TRACKING AND CONTROL PROCESS

In Simple Words: It's like checking off clues on a treasure hunt — you track which clues are solved, and sometimes the hunt organizer changes a clue partway through.

Example — Homework Helper App: Every Friday, Priya updated a spreadsheet marking each requirement "Done," "In Progress," or "Not Started" — that's requirements status being tracked. In November, Mrs. Diaz from the PTA asked for a brand-new "parent view" feature — that request became a scope change Maya had to track too.

🧠 **Remember It Like This:** Tracking watches where requirements stand; changes ask requirements to grow.

CHANGE CONTROL PROCESS

> 📖 **New Word!** A **baseline** is a version of the requirements everyone has agreed on and locked in — like a photograph of exactly what was promised, at one point in time.

> 📖 **New Word! Change control** means having a clear, fair way to decide whether a new idea gets added to the project — instead of it just sneaking in.

In Simple Words: It's like the rules of a board game everyone agreed to before the game started. If someone wants a new rule halfway through, you don't just add it — everyone votes first.

Example — Homework Helper App: REQ-001 through REQ-040 were baselined on September 15. Two weeks later, Mr. Chen texted Maya asking for a "leaderboard" feature. Instead of Sam coding it that night, Maya said, "Let's write it up as a change request and review it Thursday." Change control modifies the baseline — and the baseline is exactly what change control measures against.

🧠 **Remember It Like This:** No baseline, no fair fight — lock in the rules before you change the game.

ACCEPTANCE & SYSTEM TESTING PROCESSES

In Simple Words: It's like a judge at a talent show, checking each act against a scorecard.

Example — Homework Helper App: Priya tested the app against every requirement, one by one. For REQ-009 ("The app shall let a student mark an assignment as complete"), she tapped the checkbox, watched it turn green and save — pass! Testing is Priya's proof that the requirements were actually built correctly.

🧠 **Remember It Like This:** Requirements write the test; testing proves the promise was kept.

CONSTRUCTION PROCESS

> 📖 **New Word! Traceability** means being able to point at any piece of code and say exactly which requirement it came from — like a receipt that shows what you paid for.

In Simple Words: It's like following a LEGO instruction booklet page by page — every brick you snap on traces back to a picture in the booklet.

Example — Homework Helper App: When Sam wrote the reminders code, he labeled it “Implements REQ-014” in his notes, so anyone could trace that exact piece of code straight back to the requirement that asked for it.

🧠 **Remember It Like This:** Every brick of code should trace back to the instruction that asked for it.

USER DOCUMENTATION PROCESS

In Simple Words: It’s like a game manual, written after the rules are final, explaining how to actually play.

Example — Homework Helper App: Once requirements were locked in, Priya wrote the “How to Use Homework Helper” one-page guide — explaining how to add an assignment and turn on reminders, taken straight from the requirements list.

🧠 **Remember It Like This:** The user guide is just the requirements, translated into “here’s how you use it.”

🎯 **Quick Recap:**

- **Relationship to Other Processes** — requirements are the hub of the wheel; everything else connects to them.
- **Project Planning** — requirements shape the plan; the plan shapes the requirements’ scope.
- **Project Tracking and Control** — tracks requirement status and handles requests for scope changes.
- **Change Control** — requirements are the locked-in baseline that changes get compared against.
- **Acceptance & System Testing** — requirements are the answer key that testing checks against.
- **Construction** — code should always trace back to the requirement that asked for it.
- **User Documentation** — the user guide is built directly from the requirements.

Who Cares About Requirements? Meet the Stakeholders

REQUIREMENTS AND VARIOUS STAKEHOLDER GROUPS

> 📖 **New Word!** A **stakeholder** is anyone who cares about the project or is affected by it — not just the people building it.

In Simple Words: It’s like planning a birthday party. It’s not only about what the birthday kid wants — parents, guests, and even the cake baker all have a say in how it turns out.

Example — Homework Helper App: Maya realized way more people had a stake in the app than just her team:

Stakeholder	What They Contribute
Management	establishes project constraints, resources, and commitments
Marketing or Product Management	specifies business or market requirements; requests changes
Technical Support	provides input from bug reports and enhancement requests
Users	describe user requirements and quality attributes; review requirements
Project Sponsor	provides business requirements
Systems Engineering	allocates system requirements to software; requests changes
Procurers	specify business and quality requirements; participate in vendor selection
Legal Department	handles licensing of tools and components

🖼️ **What this picture is telling you:** A whole circle of different people surrounds the software team — each one feeding requirements information in from a different direction.

🧠 **Remember It Like This:** A stakeholder doesn't have to build the app to care deeply about it.

MANAGEMENT

In Simple Words: It's like the adult who sets the rules for a lemonade stand — how much you can spend, and by when you need to open.

Example — Homework Helper App: Ms. Alvarez, the assistant principal, told Maya: "You can spend $0 — only free, school-approved tools — and it has to launch before Winter Break, December 19th." That's Management setting constraints, resources, and commitments.

🧠 **Remember It Like This:** Management draws the fence around the playground — you build inside the lines they set.

MARKETING OR PRODUCT MANAGEMENT

In Simple Words: It's like the friend who tells you what everyone at school actually wants at the party, so you don't guess wrong.

Example — Homework Helper App: Mrs. Diaz from the PTA said families wanted to check on their kids' assignments too, so she requested REQ-041: "The app shall let a parent view (but not edit) their child's assignment list." That single ask was both a new market requirement AND a request to change what V2.0 would cover.

🧠 **Remember It Like This:** Marketing listens to the crowd outside and brings their wishes to the table.

TECHNICAL SUPPORT

In Simple Words: It's like the kid who collects everyone's complaints about a broken swing set and reports them to the person in charge of fixing it.

Example — Homework Helper App: After launch, Jordan — who runs the school's tech help desk — collected 12 bug reports in the first week, including "the app crashes when I have more than 20 assignments," and forwarded them straight to Maya.

🧠 **Remember It Like This:** Support hears the complaints first — and turns them into gold for the next version.

USERS

In Simple Words: It's like the kids who actually play the board game — their opinion on whether it's fun matters more than anyone else's.

Example — Homework Helper App: 40 students across every grade filled out Maya's survey describing what they needed. Later, 10 of them reviewed the draft requirements and caught that REQ-014 never said *when* the reminder should arrive.

🧠 **Remember It Like This:** Users don't just use the app — they help write its wish list.

PROJECT SPONSOR

In Simple Words: It's like the person who says, "Someone needs to organize this fundraiser" — they don't run it themselves, but they're the reason it exists.

Example — Homework Helper App: Mr. Chen kicked off the whole project by telling Maya, "Too many kids are missing homework and their grades are dropping — we need something to help them remember." That single sentence became the business requirement the entire app grew from.

🧠 **Remember It Like This:** The sponsor plants the seed — the whole tree of requirements grows from their "why."

SYSTEMS ENGINEERING

In Simple Words: It's like deciding which chores belong to which sibling, so nothing gets done twice and nothing gets forgotten.

Example — Homework Helper App: The school already used a system called Grade Portal for official grades. Mr. Rivera, the district's IT systems coordinator, decided Homework Helper should only handle reminders and to-do lists — not grades — allocating which requirements belonged to which software. When the PTA later asked for the parent-view feature, it was Mr. Rivera who requested the change to add it to Homework Helper's side rather than Grade Portal's.

🌟 **Bonus Example:** Think about a smart robot vacuum. Someone has to decide: is "stop before hitting a wall" a job for a hardware sensor, or a software rule? Sorting which requirement belongs to which part of a system is exactly what systems engineering does.

🧠 **Remember It Like This:** Systems engineering decides which toy box each requirement belongs in.

PROCURERS

In Simple Words: It's like the family member who compares two stores before buying something, to make sure you get the best, safest deal.

Example — Homework Helper App: When it was time to pick a free service to store student data, Mrs. Kapoor, the school librarian, compared two options and checked which one followed the school's privacy rules before Maya's team signed up.

🌟 **Bonus Example:** Imagine a whole school district deciding to BUY a big learning app instead of building one. They'd invite two companies to show off their products, compare prices and features, and pick a winner — that's vendor selection at a much bigger scale.

🧠 **Remember It Like This:** Procurers go shopping for the project — and make sure you don't buy the wrong thing.

LEGAL DEPARTMENT

In Simple Words: It's like asking permission before you use your friend's cool sticker in your own drawing.

Example — Homework Helper App: Mrs. Kapoor — wearing a different hat this time — checked the license on a free icon pack Sam wanted to use for the app's buttons, to confirm it was really okay to use, even for a school project.

🌟 **Bonus Example:** A video game studio's legal team checks whether they're allowed to use a specific song in their game, or whether they need to pay the musician first.

🧠 **Remember It Like This:** Before you use someone else's stuff, Legal checks if you're actually allowed to.

🎯 **Quick Recap:**

- **Stakeholders overview** — a whole circle of people beyond the dev team have a say in requirements.
- **Management** — sets the constraints, budget, and deadlines.
- **Marketing/Product Management** — brings in business and market wishes, requests changes.
- **Technical Support** — turns bug reports and complaints into future requirements.
- **Users** — describe what they need and review whether it's right.
- **Project Sponsor** — supplies the core business reason the project exists.
- **Systems Engineering** — decides which requirements belong to which part of the system, and can request changes too.
- **Procurers** — compare options and help pick the right tools or vendors.
- **Legal Department** — checks licensing before you use someone else's stuff.

Getting People to Say Yes to Change

GAINING COMMITMENT TO CHANGE

In Simple Words: It's like getting your whole family to start recycling — everyone nods along, but old habits are hard to break, and someone always resists.

Example — Homework Helper App: When Maya suggested a requirements review checklist for V2.0, Sam groaned, "We don't have time for one more step." Maya knew she'd have to work through his resistance before the idea would actually stick.

🧠 **Remember It Like This:** A great idea only works if people actually pick it up and use it.

RESISTANCE FORM 1: "I DON'T HAVE TIME TO IMPROVE"

In Simple Words: It's like refusing to spend 5 minutes cleaning your room because you're "too busy" — then wasting 20 minutes tomorrow looking for your shoes.

Example — Homework Helper App: Sam said he was too busy coding to sit through requirements training. Maya reminded him, "Remember how you rewrote the reminders code three times? That training would've saved you way more time than it costs. If we skip investing in getting better now, V2.0 will hit the exact same bumps V1.0 did."

🧠 **Remember It Like This:** Skip the practice today, and you'll fumble the play tomorrow.

RESISTANCE FORM 2: "CHANGE CONTROL IS JUST A BARRIER"

In Simple Words: It's like thinking a crosswalk light is just "in your way" — until you realize it's actually keeping you safe from getting hit by a car.

Example — Homework Helper App: Sam complained that filling out a change request form felt like red tape. Maya explained, "It's not a wall — it's how well-informed people make good decisions and make sure everyone hears about them. If it's too slow, let's fix the FORM, not skip the step." Maya knew that if the process ever got too clunky, people would just start sneaking changes in around it — so it was her job to keep it simple enough that the team actually wanted to use it.

🧠 **Remember It Like This:** Change control isn't a locked gate — it's a crosswalk that keeps good decisions safe.

RESISTANCE FORM 3: "WRITING THIS DOWN IS A WASTE OF TIME"

In Simple Words: It's like refusing to write down a recipe before baking — then forgetting whether it was 2 cups of flour or 3, and ruining the whole cake.

Example — Homework Helper App: Sam once said, "Just tell me what to build, why write it down? It's an admin time-waster, it just delays the real work." Then a requirement got misremembered, and he built the wrong reminder timing — 9 wasted hours. Maya used that exact story to show why writing requirements

down actually saves “real work” time in the long run. She added, “And it’s not just about this launch — if a requirement stays wrong or missing, we’ll be quietly patching around it for YEARS after V2.0 ships, which costs way more than getting it right once, up front.”

🧠 **Remember It Like This:** Skip the notes now, pay in rewrites later — for years, not just for this launch.

🎯 **Quick Recap:**

- **Gaining Commitment** — even good ideas need people to actually buy in.
- **”No time” resistance** — skipping the investment now costs more time later.
- **”Just a barrier” resistance** — change control protects good decisions rather than blocking them, but only if the team keeps the process working smoothly enough that people don’t work around it.
- **”Waste of time” resistance** — documentation prevents expensive rewrites now AND years of costly patching down the road.

The Golden Rules of Getting Better

FUNDAMENTALS OF SOFTWARE PROCESS IMPROVEMENT

In Simple Words: It’s like the training rules every good sports coach follows, no matter which sport they coach.

Example — Homework Helper App: Before starting her improvement plan, Maya wrote four golden rules on a sticky note above her desk.

🧠 **Remember It Like This:** Four simple rules keep “getting better” from turning into a mess.

PRINCIPLE 1: EVOLUTIONARY AND CONTINUOUS

In Simple Words: It’s like leveling up in a video game — you don’t jump from Level 1 to Level 100 overnight. You improve a little bit at a time, forever.

Example — Homework Helper App: Instead of fixing everything at once for V2.0, Maya’s team tried ONE improvement first (a review checklist), then added a second one (a change request form) a month later.

🧠 **Remember It Like This:** Improvement isn’t a finish line — it’s a staircase you keep climbing.

PRINCIPLE 2: PEOPLE CHANGE ONLY WITH INCENTIVE

In Simple Words: It’s like a chore chart with a reward at the end — people try harder when there’s something real in it for them.

Example — Homework Helper App: Maya showed Sam a number: “The checklist would’ve caught REQ-014’s missing detail and saved you 9 hours.” Suddenly Sam wanted to use it — because now he could see exactly why it helped *him*.

🧠 **Remember It Like This:** Nobody changes for a rule — they change for a reason.

PRINCIPLE 3: GOAL-ORIENTED CHANGE

In Simple Words: It's like practicing free throws because you want to win the championship — not just practicing for no reason at all.

Example — Homework Helper App: The team didn't add a checklist just "because a book said so." They aimed it at a real target: "Reduce rework hours by 25%."

🧠 **Remember It Like This:** Don't change just to change — change to hit a target.

PRINCIPLE 4: TREAT IMPROVEMENT AS MINI-PROJECTS

In Simple Words: It's like planning a small bake sale with its own budget and deadline — not just a vague hope that "someday we'll sell cookies."

Example — Homework Helper App: Maya wrote a tiny plan just for the checklist: Priya creates it by next Friday, and they'll know it worked if rework hours drop.

🧠 **Remember It Like This:** Treat "getting better" like its own little project — with a plan, not just a wish.

🎯 **Quick Recap:**

- **Fundamentals overview** — four golden rules guide any real improvement effort.
- **Evolutionary & continuous** — improve step by step, forever, not all at once.
- **Incentive to change** — people adopt new habits when they see a clear personal benefit.
- **Goal-oriented** — aim every change at a specific, measurable target.
- **Mini-projects** — plan your improvement with the same care as any real project.

Finding the REAL Problem

ROOT CAUSE ANALYSIS

> 📖 **New Word!** A **root cause** is the REAL reason something went wrong — not just the obvious symptom sitting on the surface.

> 📖 **New Word!** A **fishbone diagram** (also called an **Ishikawa diagram**) is a picture shaped like a fish skeleton that sorts all the possible causes of a problem into neat groups.

> 📖 **New Word! Requirements elicitation** is the fancy word for gathering requirements by talking to people, asking questions, and watching how they work.

In Simple Words: It's like being a detective. If your ice cream keeps melting before you get home, you don't just say "hot weather" — you dig deeper: was it the route, the cup, or how long you waited in line? A fishbone diagram sorts your list of suspects into neat categories, all pointing at one big problem.

Example — Homework Helper App: Maya drew a fishbone diagram to figure out why they didn't finish V1.0 on time. Here's the real diagram, exactly as her team used it:

🖼️ **Fishbone Diagram — "Don't finish projects on time"**

- **Bone: Process**
  - requirements are missed during elicitation
    - analyst not trained
    - user class not represented
  - new requirements are added
    - no change control process
- **Bone: People**
  - developers are fixing problems caused by late requirements changes on previous project
    - developers aren't available
- **Bone: Project**
  - requirements change too frequently
    - didn't get input from right people
    - market is poorly defined
    - legislative changes

🖼️ **What this picture is telling you:** All three bones — Process, People, and Project — feed into one big problem at the fish's head ("Don't finish projects on time"), so you can see exactly which real causes are dragging the project down instead of just guessing.

Mapped to Homework Helper: "analyst not trained" was Maya's very first requirements interview ever. "User class not represented" meant she only interviewed 6th graders (her little brother's grade) and never asked 8th graders, who needed to filter homework by class period. "No change control process" was Mr. Chen's late-night texts asking for new features. "Developers aren't available" was Sam still fixing V1.0 crash bugs while V2.0 planning was supposed to start. "Legislative changes" became a new district rule banning phone notifications after 8pm, forcing a last-minute requirement change.

🧠 **Remember It Like This:** When something breaks, follow the bones back to the real fish — not just the fin you can see.

🎯 **Quick Recap:**

- **Root Cause Analysis** — use a fishbone diagram to trace symptoms back to their real causes across Process, People, and Project.

The Circle of Getting Better

THE PROCESS IMPROVEMENT CYCLE

> 📖 **New Word!** A **pilot** is a small, safe trial run of a new idea before you use it everywhere — like tasting one cookie before baking the whole batch.

> 📖 **New Word! Roll-out** means bringing a product to everyone — making it accessible and starting to promote it, like a store's grand opening.

In Simple Words: It's like a chore wheel that spins every week: check what's working, plan a fix, try it out, check the results — then spin around and do it again.

🖼️ **The Software Process Improvement Cycle**

1. **Assess current practices** → *(findings and recommendations)* →
2. **Plan improvement actions** → *(action plan)* →
3. **Create, pilot, and roll out processes** → *(new processes; results of pilots; rollout experience)* →
4. **Evaluate results** → loops back to Step 1

Feedback questions along the way: "Do the new processes achieve the desired results?", "How well did the action planning process work?", "How smoothly did the pilot and rollout go?", "What's the plan for the next improvement cycle?"

🖼️ **What this picture is telling you:** Getting better is a circle, not a straight line — you never really "finish," you just keep spinning around it.

Example — Homework Helper App:

1. **Assess** — they listed everything that went wrong with V1.0 (their fishbone results).
2. **Plan** — they decided: add a review checklist plus a change request form.
3. **Create, pilot, and roll out** — Priya tried the checklist on just ONE requirement first (a pilot!), it worked, so they rolled it out to all of V2.0's requirements.
4. **Evaluate** — three weeks later they checked: did rework hours actually drop? Yes — from 9 hours down to 2! Then the cycle spins around again for the next improvement.

🧠 **Remember It Like This:** Assess, plan, try, check — then spin the wheel again.

🎯 **Quick Recap:**

- **The Process Improvement Cycle** — four repeating stages (Assess → Plan → Create/Pilot/Roll out → Evaluate) that never really stop.

Building Blocks: Process Assets

TYPES OF PROCESS ASSETS

> 📖 **New Word!** A **process asset** is any reusable tool, document, or pattern that helps your team work better and more consistently — like a toolbox you build up over time.

In Simple Words: It's like a family recipe box passed down through the years — full of index cards, tips, and patterns that make cooking easier every single time.

🖼️ **Types of Process Assets**

Type	Description
Checklist	A list of activities, deliverables, or other items to be noted or verified. Checklists are memory joggers that help ensure busy people don't overlook important details.
Example	A representative of a specific type of work product. Accumulate and share good examples as project teams create them.
Plan	An outline of how an objective will be accomplished and what is needed to accomplish it.
Policy	A guiding principle that sets a management expectation of behaviors, actions, and deliverables. Processes should enable satisfaction of the policies.
Procedure	A step-by-step description of the sequence of tasks that accomplishes an activity. Describes the tasks performed and the project roles that perform them. Guidance documents can support a procedure with tutorial info and tips.
Template	A pattern used as a guide for producing a work product. Provides "slots" for capturing and organizing info, with guidance text to help the author use it well. Other templates define structure for specific info types too, like a functional requirement, quality attribute, business rule, or user story.

🖼️ **What this picture is telling you:** There isn't just one kind of helpful tool — there are six different types, each solving a different problem.

Example — Homework Helper App: Maya's team started building their own toolbox for V2.0 — one of each type, described below.

🧠 **Remember It Like This:** Process assets are your team's toolbox — built once, used forever.

CHECKLIST

Example — Homework Helper App: Priya made a "Requirements Review Checklist": □ Has a unique ID? □ Is it testable? □ Did a real student agree it's needed? □ Is it written as ONE clear sentence? No requirement counted as "done" until it checked every box.

🧠 **Remember It Like This:** A checklist is your memory's best friend — it never forgets, even when you do.

EXAMPLE (AS A PROCESS ASSET)

Example — Homework Helper App: Maya saved REQ-001 ("The app shall let a student add a new assignment with a title, subject, and due date, in under 10 seconds") in a folder called "Great Requirements," to show new teammates exactly what a clear requirement looks like.

🧠 **Remember It Like This:** One great example is worth a thousand vague instructions.

PLAN

Example — Homework Helper App: Maya wrote the "Homework Helper V2.0 Project Plan" — 8 weeks, 3 people, and a list of tools needed, like laptops, a shared doc, and testing phones.

🧠 **Remember It Like This:** A plan is your map before the trip — not a diary you write after you're already lost.

POLICY

Example — Homework Helper App: Mr. Chen and Maya set a policy: "No new feature gets built until it's written down and approved — no exceptions, not even for the teacher!"

🧠 **Remember It Like This:** A policy is the rule everyone agrees to follow — even the boss.

PROCEDURE

In Simple Words: It's like a recipe card with numbered steps — not just a list of ingredients, but exactly who does what, in what order.

Example — Homework Helper App: The team wrote a step-by-step "How to Submit a Change Request" procedure: (1) Fill out the change form, (2) Maya reviews it within 2 days, (3) The change board votes yes or no, (4) If yes, the requirements doc gets updated. Priya also taped a short "tips" note next to it, reminding everyone what "urgent" actually means — a little guidance to help people use the procedure correctly.

🧠 **Remember It Like This:** A procedure is a recipe for getting something done — step 1, step 2, step 3, done.

TEMPLATE

Example — Homework Helper App: Sam used a "Use Case Template" with blanks to fill in for "Student Marks Assignment Complete": Actor: ___, Trigger: ___, Steps: ___, Result: ___. He just filled in the blanks instead of starting from a blank page.

🌟 **Bonus Example:** Maya's team didn't stop at one template — they later made separate mini-templates for other info types too, like a short fill-in-the-blank pattern just for writing a "quality attribute" requirement (how FAST or how SAFE something must be), so nobody had to guess the right format from scratch.

🧠 **Remember It Like This:** A template is a coloring book page — the outline's already there, you just fill it in.

REQUIREMENTS ENGINEERING PROCESS ASSETS

> 📖 **New Word!** A **Change Control Board (CCB)** is a small group of people whose job is to say yes or no to requested changes.

> 📖 **New Word! Prioritization** means deciding which requirements matter most, so the most important ones get built first.

In Simple Words: It's like a two-drawer toolbox — one drawer full of tools that help you WRITE good requirements, and one drawer full of tools that help you MANAGE them once they exist.

🖼️ **Key Process Assets for Requirements Development and Requirements Management**

Requirements Development Process Assets

1. Requirements development process
2. Requirements allocation procedure
3. Requirements prioritization procedure
4. Vision and scope template
5. Use case template
6. Software requirements specification (SRS) template
7. Requirements review checklist

Requirements Management Process Assets

1. Requirements management process
2. Requirements status tracking procedure
3. Change control process
4. Change control board charter template
5. Requirements change impact analysis checklist
6. Requirements tracing procedure

🖼️ **What this picture is telling you:** All the toolbox items split neatly into two jobs — writing requirements well, and taking care of them after they're written.

Example — Homework Helper App: For Requirements Development, the team adopted a Use Case Template and an SRS template (SRS = the big document listing every requirement in one place). For Requirements Management, they created a Change Control Board charter — a one-page paper naming who's on the CCB (Mr. Chen, Maya, and Sam) and how they vote.

🧠 **Remember It Like This:** One drawer helps you WRITE requirements well, the other helps you TAKE CARE of them afterward.

🎯 **Quick Recap:**

- **Types of Process Assets** — six kinds of reusable tools: Checklist, Example, Plan, Policy, Procedure, Template.
- **Checklist** — memory jogger so nothing important gets skipped.
- **Example** — a saved sample of great work to copy the pattern from.
- **Plan** — an outline of how to reach a goal and what you'll need.
- **Policy** — a rule about expected behavior.
- **Procedure** — a step-by-step "how to do it" guide, sometimes backed by extra tips and guidance.
- **Template** — a fill-in-the-blanks pattern, with guidance built in, and sometimes a whole family of templates for different info types.
- **RE Process Assets** — a full toolbox split between developing requirements and managing them.

Measuring Whether You're Actually Getting Better

PERFORMANCE INDICATORS (KPIS)

> 📖 **New Word!** A **KPI (Key Performance Indicator)** is a number you track over time to see whether you're really improving — like a scoreboard for your process.

In Simple Words: It's like a fitness tracker's step count — an honest number that tells you whether your new habit is actually working, not just whether it *feels* like it's working.

Example — Homework Helper App: Maya picked a KPI: "hours of rework caused by unclear requirements." Before the checklist: 9 hours. Two months after: 2 hours. That number proved the checklist was really helping — not just making everyone feel busier.

If a KPI you picked doesn't show progress after a while, here's what to check:

🖼️ **What to Investigate When KPIs Don't Improve**

1. Were the problems correctly analyzed and root causes identified?
2. Did you select improvement actions that directly addressed those root causes?
3. Was the plan to implement those actions realistic? Was it executed as intended?
4. Has something changed since the original analysis that should redirect the team's efforts?
5. Have team members actually adopted the new ways of working and pushed through the learning curve?
6. Did you set realistic targets the team had a real chance of achieving?

🖼️ **What this picture is telling you:** If a number isn't moving, don't just push harder — check whether you fixed the right thing, the right way, with the right target.

🖼️ **Possible KPIs for Requirements Process Improvement Goals**

Improvement goal	Suggested indicators
Reduce rework performed because of requirements errors	Hours of rework at all life-cycle stages attributable to an erroneous, ambiguous, unnecessary, or missing requirement; percentage of requirements that have errors discovered following baselining
Reduce the negative impact of requirements changes	Number of new requirements presented after baselining that could have been known beforehand; percentage of requirements modified after baselining; hours per release or iteration needed to modify deliverables because of requirement changes; distribution of change requests by origin
Reduce the time needed to clarify requirements during development	Number of requirements questions/issues raised after baselining; average time needed to resolve each question or issue
Improve estimation accuracy for total requirements development effort	Estimated and actual labor hours spent on requirements development activities per release and for the total project
Reduce the number of unneeded features implemented	Percentage of committed features removed before implementation begins; percentage of committed features removed before delivering a release or iteration

🖼️ **What this picture is telling you:** Each improvement goal has its own honest scoreboard — you don't just "feel" better, you can point to a real number.

Homework Helper's numbers so far: Maya found that 15% of their 40 requirements (6 of them) had to change after being baselined — that's her KPI for "reduce the negative impact of requirements changes," and it's exactly the number Track C of their road map (below) is aiming to shrink.

🧠 **Remember It Like This:** If you can't measure it, you can't prove you actually got better.

🎯 **Quick Recap:**

- **Performance Indicators (KPIs)** — track real numbers over time, and if they're not improving, work back through the checklist of possible reasons before giving up.

Drawing the Map to a Better Future

CREATING A REQUIREMENTS PROCESS IMPROVEMENT ROAD MAP

> 📖 **New Word!** A **milestone** is a checkpoint on your journey that proves you've reached an important point — like a flag planted partway up a mountain.

> 📖 **New Word! Requirements volatility** means how often requirements keep changing — high volatility means they change A LOT.

In Simple Words: It's like a treasure map with three different paths that all lead toward better treasure, with flags (milestones) marking your progress along each one.

🗺️ **Sample Requirements Process Improvement Road Map**

- **Track A (testing/quality):** Train team in requirements engineering → Train team in software reviews → Review requirements documents → **M1** → *Outcome: Reduce system testing effort by 25% in 6 months*
- **Track B (use-case/prioritization):** Train team in requirements engineering → Adopt standard SRS template → Adopt use case approach and template → **M2** → Adopt requirements prioritization procedure → **M3** → *Outcome: Improve customer satisfaction by 1 rating point*
- **Track C (change-management):** Set up CCB and create CCB charter → Document requirements change procedure → **M4** → Adopt change impact analysis procedure → Track requirements status → **M5** → *Outcome: Reduce requirements volatility by 20%*

🗺️ **What this picture is telling you:** One road map, three paths, all running at the same time — Track A and Track B even share the same starting box ("Train team in requirements engineering"), and milestones like M1 and M3 can each unlock their own measurable payoff at the end. One road map, several real business outcomes.

Example — Homework Helper App: Maya turned this exact road map into Homework Helper's real V2.0 plan:

- **Track A:** Train Sam & Priya in requirements engineering → train them in reviews → review the V2.0 requirements doc → **M1** (January 30th) → Outcome: cut testing effort by 25% in 6 months.
- **Track B:** (shares the same training start) → adopt a standard SRS template → adopt a use case template → **M2** (February 15th) → adopt a prioritization procedure → **M3** (March 1st) → Outcome: raise student satisfaction ratings by 1 full point.
- **Track C:** Set up the Change Control Board and write its charter → document the change request procedure → **M4** (February 1st) → adopt a change-impact-analysis procedure → track requirements status weekly → **M5** (April 1st) → Outcome: cut requirements volatility by 20%.

🧠 **Remember It Like This:** A road map turns "we should get better" into "here's exactly how, and here's how we'll know we did."

🎯 **Quick Recap:**

- **Requirements Process Improvement Road Map** — lay out multiple improvement tracks with milestones so everyone can see the path to real, measurable outcomes.

🏆 Chapter Wrap-Up

Maya, Sam, and Priya didn't just ship Homework Helper V1.0 — they turned every bump along the way into a lesson. They found the hub-and-spoke web connecting requirements to every other process, met the whole circle of stakeholders who cared about the app, learned to push through resistance to change, followed the

fishbone back to real root causes, built a toolbox of reusable process assets, tracked honest KPIs, and finally drew a real road map with milestones to get to V2.0. That's what "improving requirements processes" really means: not being perfect the first time, but getting a little better, on purpose, every time.

If you remember ONLY these things…

- Requirements sit at the center of the whole project — every process connects back to them.
- A whole circle of stakeholders — not just the dev team — shapes what requirements say.
- Change needs real buy-in, so be ready to answer the "I don't have time," "this is just red tape," and "writing it down is a waste of time" objections.
- Good improvement follows four golden rules: evolutionary, incentive-driven, goal-oriented, and treated like its own mini-project.
- A fishbone diagram helps you trace a problem back to its real root cause, not just its symptom.
- Improvement moves in a repeating cycle: Assess → Plan → Create/Pilot/Roll out → Evaluate — then spin again.
- Process assets (checklists, examples, plans, policies, procedures, templates) are the reusable toolbox that makes every future project easier.
- KPIs are the honest scoreboard that proves whether you actually got better.
- A road map with clear milestones turns "we should improve" into an actual plan everyone can follow.

Chapter 10: Software Requirements & Risk Management

Maya, Sam, and Priya are hard at work building **Homework Helper**, a phone app that will help every student at Maple Street Middle School track assignments, remember deadlines, and get friendly study reminders. Their teacher and project sponsor, Mr. Chen, has one big goal: show off a working app at the school's Fall Tech Fair on September 30th. But building software — even a school project — means things can go wrong along the way, from confusing instructions to missed deadlines, so in this chapter you'll learn how smart teams spot trouble early, and exactly what kind of trouble tends to hide inside each step of figuring out what a piece of software needs to do.

> 📖 **New Word!** requirement means a specific thing a software product must do or must have — like "the app must remind students about homework due in 2 days."

What Is Risk Management?

RISK MANAGEMENT

In Simple Words: Imagine you're the captain of a ship sailing toward hidden treasure. A smart captain doesn't just sail straight ahead and hope for the best — they watch the sky for storms and keep a plan ready in case something goes wrong. Risk management is exactly that, but for a project instead of a ship.

> 📖 **New Word!** risk means something that *might* go wrong in the future and cause a problem, before it has actually happened.

Example — Homework Helper App: Every Monday afternoon, Maya's team holds a 10-minute "risk meeting" in the school library. They talk about anything that *could* go wrong before it actually does — like "What if the school Wi-Fi is too slow for the app to work during the Tech Fair demo?" Thinking about that in August, weeks before September 30th, means they still have time to fix it.

🖼️ **Figure 32-1 — Elements of Risk Management**

- **Risk Management**
  1. **Assessment**
     - Identification
     - Analysis
     - Prioritization
  2. **Avoidance**
  3. **Control**
     - Management Planning
     - Resolution
     - Monitoring

🖼️ What this picture is telling you: To handle a risk well, first you **assess** it (find it, understand it, and rank it against other risks). Then you either make sure it can **never happen at all** (avoid it), or you make a plan, take action, and keep watching it over time (control it).

🧠 **Remember It Like This:** Risk management means looking for storms before you set sail, not swimming through them after you're already soaked.

ASSESSMENT

In Simple Words: Assessment is like a doctor's checkup for your project — you can't fix a problem until you actually check for it, understand it, and decide how urgent it is.

Example — Homework Helper App: Maya's team keeps a shared "Risk List." Every risk anyone spots gets added, then discussed: is it likely to happen, how bad would it be, and how important is it compared to all the other risks on the list?

🧠 **Remember It Like This:** Assessment is checking risks the way a doctor checks you — look, understand, then decide what matters most.

IDENTIFICATION

In Simple Words: Identification is like playing "I Spy" for problems. The very first step is simply noticing something *could* go wrong, before you do anything else about it.

Example — Homework Helper App: During a test run, Priya notices something: "Hey — if a student's phone runs out of storage, the app might not save their homework list!" She writes this down as a brand-new item: "Risk #7: Low phone storage could cause lost data."

🧠 **Remember It Like This:** You can't dodge a problem you never spotted.

ANALYSIS

In Simple Words: Analysis is like deciding, if you forgot your umbrella, how likely it is to rain today, and how wet you'd actually get. Two questions, every single time: how likely, and how bad?

Example — Homework Helper App: For Risk #4 ("the app might crash with no internet"), the team analyzes it: Likelihood = High, because lots of students lose signal in the school's basement hallway. Impact = High, because they can't see tonight's homework right when they need it. That combination makes it a serious risk.

🧠 **Remember It Like This:** Every risk gets two questions: How likely? How bad?

PRIORITIZATION

> 📖 **New Word!** prioritization means deciding what order to deal with things in, based on what matters most.

In Simple Words: Prioritization is like sorting your Halloween candy so you eat the best pieces first — you handle the scariest risks before the small, annoying ones.

Example — Homework Helper App: On the Risk List, “app crashes with no internet” (High likelihood, High impact) jumps straight to the top. “The app icon could look a little nicer” sinks to the very bottom — nice to fix, but not scary.

🧠 **Remember It Like This:** The biggest, scariest risks jump to the front of the line.

AVOIDANCE

In Simple Words: Avoidance is like wearing a bike helmet — you change what you do so a risk can never turn into a real injury in the first place.

Example — Homework Helper App: Instead of just hoping the Wi-Fi is fine, Sam redesigns the app to save all homework data right on the phone first, before it ever needs the internet. Now bad Wi-Fi simply *can’t* crash the app — the risk isn’t handled, it’s erased.

🧠 **Remember It Like This:** The best-fixed problem is the one that never got the chance to happen.

CONTROL

In Simple Words: Control is like a fire drill — you can’t make it impossible for a fire to ever start, but you’ve got an exact plan ready for if one does.

Example — Homework Helper App: The team can’t stop Maple Street from changing its homework-schedule format mid-project. So instead, they build a control plan: watch for the announcement, assign Sam to update the code fast if it happens, and check in every month.

🧠 **Remember It Like This:** If you can’t avoid it, plan it, fix it, and keep watching it.

MANAGEMENT PLANNING

In Simple Words: Management Planning is like deciding, before your camping trip, exactly what you’ll do “if it rains” — you decide the plan *before* you actually need it.

Example — Homework Helper App: The team writes down: “IF the Wi-Fi risk becomes real, THEN Sam will build an offline-saving feature within 3 days.” That’s the management plan, ready and waiting, before anything even goes wrong.

🧠 **Remember It Like This:** Decide your move before the game gets scary — not during.

RESOLUTION

In Simple Words: Resolution is like actually putting up the tent’s rain cover once dark clouds show up — it’s the real action, not just talking about it.

Example — Homework Helper App: When Mr. Chen announces on August 25th that the school is switching to a new homework-schedule format, Sam actually rewrites the import code that same weekend. The risk isn't just planned for anymore — it's resolved, for real.

🧠 **Remember It Like This:** A plan is just talk. Resolution is doing.

MONITORING

In Simple Words: Monitoring is like checking the weather forecast every single day before a picnic, not just once — because risks can grow, shrink, or disappear over time.

Example — Homework Helper App: Every Friday, Priya rechecks the Risk List: Is the Wi-Fi problem still bad? Did Sam's offline-saving fix actually work? Are there any brand-new risks to add?

🧠 **Remember It Like This:** Risks change — keep watching, don't look away.

🎯 **Quick Recap:**

- **Risk Management** — spotting trouble before it happens, like a lookout on a ship.
- **Assessment** — check every risk like a doctor's checkup.
- **Identification** — spot the risk first; you can't fix what you can't see.
- **Analysis** — ask how likely and how bad.
- **Prioritization** — handle the scariest risks first.
- **Avoidance** — change your plan so the risk can never happen.
- **Control** — can't avoid it? Plan for it, act on it, and watch it.
- **Management Planning** — decide your move ahead of time.
- **Resolution** — actually take the action.
- **Monitoring** — keep checking; risks change over time.

Where Do Requirements Risks Hide?

REQUIREMENTS-RELATED RISKS BY ACTIVITY

In Simple Words: Building software requirements is like planning a class party in five stages: first you ask everyone what they want (gathering), then you sort through those wishes and think them through (studying), then you write it all down clearly on a plan (writing it down), then you double-check it with everyone before the big day (checking), and finally you keep the plan updated if anything changes, right up until party day (keeping track). Each of those five stages can hide its own kind of trouble.

> 📖 **New Word!** stakeholder means anyone who cares about or is affected by a project — like the people who will use it, pay for it, or are in charge of it.

Example — Homework Helper App: Maya's team sorts their risks the same five-stage way: risks from *talking to* students and Mr. Chen (elicitation), risks from *studying* what those students said (analysis), risks from *writing it down* (specification), risks from *double-checking* it (validation), and risks from *keeping it*

updated all the way to the Tech Fair (management). The stakeholders here are Mr. Chen, the students, Maya, Sam, and Priya — basically everyone connected to Homework Helper.

🧠 **Remember It Like This:** Different stages hide different traps — a risk hiding in the writing-down stage isn't the same risk hiding in the double-checking stage.

🎯 **Quick Recap:**

- **Requirements-Related Risks by Activity** — requirement risks show up in five different stages: gathering, studying, writing down, checking, and keeping track, just like five stages of planning a class party.

Risks When Gathering Requirements (Elicitation)

> 📖 **New Word!** elicitation means the step where you talk to people and ask questions to figure out what they need — like Maya interviewing students and teachers.

REQUIREMENTS ELICITATION RISKS

In Simple Words: This whole category is about the risky part of planning a birthday party where you ask, "What do you actually want?" If you ask badly, or don't ask enough people, you can get the whole gift wrong.

Example — Homework Helper App: Maya's very first job on Homework Helper is asking Mr. Chen and a group of students what the app should do. If she rushes this step or asks the wrong questions, everything built afterward could be wrong too.

🧠 **Remember It Like This:** Ask badly at the start, and you'll build the wrong thing at the end.

PRODUCT VISION AND PROJECT SCOPE

> 📖 **New Word!** scope means everything a project includes — and just as importantly, everything it does NOT include.

In Simple Words: This is like deciding, before you build a LEGO castle, whether you're building just one tower or an entire kingdom with a moat and a dragon. Without deciding that first, nobody knows when they're "done."

Example — Homework Helper App: Maya writes a one-page Vision Statement: "Homework Helper will let students at Maple Street Middle School see their assignments, due dates, and get reminders. It will NOT include a chat feature or grades." Without this, Sam might waste weeks building a chat feature nobody actually asked for.

🧠 **Remember It Like This:** No vision, no finish line.

TIME SPENT ON REQUIREMENTS DEVELOPMENT

In Simple Words: This is like trying to plan an entire birthday party in five minutes — rushing the planning almost always means forgetting something important.

Example — Homework Helper App: Mr. Chen wants the team to start coding “right away” to save time. Maya insists on two full weeks just for figuring out the requirements first, warning: “If we skip this, we’ll be fixing mistakes in October instead of finishing in September.”

🧠 **Remember It Like This:** Rushing the planning means redoing the building.

CUSTOMER ENGAGEMENT

In Simple Words: This is like trying to guess your friend’s favorite pizza topping instead of just asking them — guessing instead of checking with the real customer is risky.

Example — Homework Helper App: Mr. Chen is very busy grading papers and often cancels meetings with Maya. With low customer engagement, Maya has to guess what teachers need instead of confirming it, which is risky.

🧠 **Remember It Like This:** A quiet customer means a guessing developer.

COMPLETENESS AND CORRECTNESS OF REQUIREMENTS SPECIFICATIONS

In Simple Words: This is like a recipe missing a step, like “add sugar!” If a requirement is incomplete or wrong, the app built from it will be incomplete or wrong too.

Example — Homework Helper App: Maya’s document says “the app shows homework due dates,” but forgets to mention “…and sorts them soonest-first.” Sam builds it exactly as written — technically correct, but missing something students actually needed.

🧠 **Remember It Like This:** A recipe missing a step bakes a broken cake.

REQUIREMENTS FOR INNOVATIVE PRODUCTS

In Simple Words: This is like trying to describe a game nobody has ever played before — it’s much harder to know what you want when nothing similar exists yet to compare it to.

Example — Homework Helper App: No app like Homework Helper exists yet at Maple Street. Students can imagine “something like a to-do list,” but they can’t picture exact details, since there’s nothing similar around school to point at and say, “like that, but different.”

🧠 **Remember It Like This:** It’s hard to describe a treasure you’ve never seen before.

DEFINING NON-FUNCTIONAL REQUIREMENTS

> 📖 **New Word!** non-functional requirement means a requirement about HOW WELL something works, like speed or safety — not WHAT it does.

In Simple Words: This is like caring not just about what a bike does (it rides!) but how well it does it — how fast, how safe, how comfortable. People often forget to ask about these.

Example — Homework Helper App: Everyone talks about WHAT Homework Helper should do (show assignments), but almost nobody mentions HOW WELL it should do it — until Priya asks, “How fast should the app open? Two seconds? Ten?” Nobody had thought about that.

🧠 **Remember It Like This:** It’s not just WHAT it does — it’s HOW WELL it does it.

CUSTOMER AGREEMENT ON REQUIREMENTS

In Simple Words: This is like planning a group pizza order where half the group wants pepperoni and half wants none at all — if stakeholders disagree, someone will be unhappy no matter what gets built.

Example — Homework Helper App: Mr. Chen wants reminders sent by email. A group of students insists they never check school email and want push notifications instead. If Maya doesn’t resolve this disagreement, one side will hate the final app.

🧠 **Remember It Like This:** Unsettled disagreements today become angry customers tomorrow.

UNSTATED REQUIREMENTS

In Simple Words: This is like assuming everyone knows a birthday cake should NOT have peanuts in it, without ever actually saying so out loud — some needs feel “obvious,” so nobody writes them down.

Example — Homework Helper App: No one ever tells Maya “the app must work on both iPhone AND Android” — everyone just assumes it, since half the students have each. It’s never written down, until Sam almost builds an iPhone-only app by accident.

🧠 **Remember It Like This:** “Obvious” to you might be invisible on paper.

EXISTING PRODUCT USED AS THE REQUIREMENTS REFERENCE

In Simple Words: This is like copying your older sibling’s science project exactly, instead of thinking about what YOUR project actually needs — relying too much on “just make it like the old thing” can hide changes that are actually needed.

Example — Homework Helper App: Someone says, “Just make Homework Helper like that other to-do app on my phone!” But that app doesn’t have due-date reminders tied to class periods, which is exactly what Maple Street students need. Copying it blindly would miss the real requirement.

🧠 **Remember It Like This:** Copying the old thing can copy its old mistakes too.

DISTRUST BETWEEN THE BUSINESS AND THE DEVELOPMENT TEAM

In Simple Words: This is like a soccer team where the forwards don’t trust the defenders to pass the ball — when two sides of a project don’t trust each other, teamwork falls apart.

Example — Homework Helper App: After Sam misses one small deadline, Mr. Chen starts double-checking everything Maya’s team says, and Maya starts feeling like he’s watching over their shoulder instead of working WITH them. That distrust slows everyone down.

🧠 **Remember It Like This:** A team that doesn't trust each other can't build anything great together.

🎯 **Quick Recap:**

- **Requirements Elicitation Risks** — trouble hiding in the very first step: asking people what they need.
- **Product vision and project scope** — no clear goal means no clear finish line.
- **Time spent on requirements development** — rushing the asking means redoing the building.
- **Customer engagement** — a quiet customer forces developers to guess.
- **Completeness and correctness** — a missing detail builds a broken feature.
- **Requirements for innovative products** — hard to describe something that's never existed before.
- **Non-functional requirements** — don't forget HOW WELL, not just WHAT.
- **Customer agreement** — unresolved disagreements make someone unhappy later.
- **Unstated requirements** — "obvious" needs can vanish if nobody writes them down.
- **Existing product as reference** — copying the old thing can copy its old mistakes.
- **Distrust** — no trust between business and dev team wrecks teamwork.

Risks When Studying Requirements (Analysis)

> 📖 **New Word!** In this step, "analysis" means carefully studying and thinking through the requirements that were gathered — sorting them, comparing them, and figuring out how hard each one will be to build. (It's a different job than the risk analysis you learned earlier — same word, different task!)

REQUIREMENTS ANALYSIS RISKS

In Simple Words: This is like, after collecting everyone's party wish list, sitting down to actually figure out which wishes are realistic and which ones might be way too hard.

Example — Homework Helper App: After Maya collects all the requirements, Sam has to sit down and study them: which ones are easy to build, which are hard, and which need skills the team doesn't have yet.

🧠 **Remember It Like This:** Gathering wishes is step one. Studying them is step two.

REQUIREMENTS PRIORITIZATION

In Simple Words: This is like deciding which chores to do first when you have too many — if you don't rank your requirements, you might waste time on small stuff while the important stuff waits.

Example — Homework Helper App: The team has 20 requirements but only 6 weeks before the Tech Fair. Maya ranks them: "Show assignments and due dates" is Must-Have. "Custom app themes with sparkly colors" is Nice-to-Have, for later. Without this ranking, Sam might spend a week on sparkly colors and run out of time for due dates.

🧠 **Remember It Like This:** If everything is "important," nothing actually is.

TECHNICALLY DIFFICULT FEATURES

In Simple Words: This is like a recipe that calls for a fancy technique you've never tried, like flipping a pancake blindfolded — some features are simply hard to build, and that's a risk in itself.

Example — Homework Helper App: A student asks for the app to automatically read handwritten homework notes from photos. Sam realizes that's incredibly hard to build well — a huge risk to the September 30th deadline if the team promises it.

🧠 **Remember It Like This:** The harder the feature, the bigger the risk of running out of time.

UNFAMILIAR TECHNOLOGIES, METHODS, LANGUAGES, TOOLS, OR HARDWARE

In Simple Words: This is like being asked to cook a dish using a brand-new kitchen gadget you've never touched before — if the team doesn't know its tools well, things are more likely to go wrong.

Example — Homework Helper App: Sam has only ever built apps for Android before. Building Homework Helper for iPhones too means learning a brand-new programming language he's never used. That unfamiliarity is a real risk to the deadline.

🧠 **Remember It Like This:** New tools in untrained hands make risky projects.

🎯 **Quick Recap:**

- **Requirements Analysis Risks** — trouble hiding while studying and sorting what was gathered.
- **Requirements prioritization** — rank the requirements, or risk wasting time on the wrong ones.
- **Technically difficult features** — hard-to-build features carry hidden risk.
- **Unfamiliar technologies** — new, untested tools and skills raise the risk.

Risks When Writing Requirements Down (Specification)

> 📖 **New Word!** specification means writing the requirements down clearly, in a document everyone can read and agree on.

REQUIREMENTS SPECIFICATION RISKS

In Simple Words: This is like writing party plans in a shared notebook so no one forgets or misunderstands — but writing things down carelessly creates its own new problems.

Example — Homework Helper App: Once Maya has studied and sorted the requirements, she has to actually write them down in a clear document that both Sam and Priya can understand and build from.

🧠 **Remember It Like This:** A badly written recipe ruins even a great idea.

REQUIREMENTS UNDERSTANDING

In Simple Words: This is like a game of “telephone,” where a message gets twisted as it passes from person to person — the person who WROTE a requirement and the person who READS it can understand it completely differently.

Example — Homework Helper App: Maya writes, “the app should remind students soon before homework is due.” Sam reads “soon” as 1 hour before. Maya actually meant 1 day before. Same words, two very different apps.

🧠 **Remember It Like This:** What you meant and what they understood aren’t always the same thing.

TIME PRESSURE TO PROCEED DESPITE OPEN ISSUES

In Simple Words: This is like starting to bake a cake even though you’re still not sure if it needs 2 eggs or 3 — moving forward while questions are still unanswered is risky.

Example — Homework Helper App: With the Tech Fair deadline looming, Sam says, “We still don’t know if reminders should be by text OR app notification — but let’s just start coding and figure it out later.” Starting before that question is answered risks building the wrong thing.

🧠 **Remember It Like This:** An open question today is a bigger problem tomorrow.

AMBIGUOUS TERMINOLOGY

> 📖 **New Word!** ambiguous means a word or sentence that can be understood in more than one way.

In Simple Words: This is like your friend saying “let’s meet soon” — does that mean in 5 minutes, or next week? Vague words let people imagine different things.

Example — Homework Helper App: A requirement says “the app should be fast.” Fast compared to what? Priya thinks it means “opens in 1 second.” Sam thinks “loads homework within 5 seconds” counts as fast too. The word “fast” alone is ambiguous.

🧠 **Remember It Like This:** Vague words leave room for everyone to imagine something different.

DESIGN INCLUDED IN REQUIREMENTS

In Simple Words: This is like a party invitation that already tells the DJ exactly which song to play at 7:03pm, instead of just saying “we want music” — mixing up WHAT you need with HOW to build it too early can limit better solutions.

Example — Homework Helper App: A requirement says “use a red button in the top-left corner to submit homework,” instead of simply “students must be able to submit homework easily.” That locks Sam into one exact design before anyone has even discussed what actually works best.

🧠 **Remember It Like This:** Say WHAT you need, not HOW to build it — not yet.

🎯 **Quick Recap:**

- **Requirements Specification Risks** — trouble hiding while writing requirements down clearly.
- **Requirements understanding** — writer and reader can understand the same words differently.
- **Time pressure** — rushing past open questions creates bigger problems later.
- **Ambiguous terminology** — vague words can mean different things to different people.
- **Design included in requirements** — describe WHAT you need, not HOW to build it, too soon.

Risks When Checking Requirements (Validation)

> 📖 **New Word!** validation means checking that the requirements are actually correct and complete before building starts — like proofreading before you turn in an essay.

REQUIREMENTS VALIDATION RISKS

In Simple Words: This is like showing your party plan to a parent before the big day, to catch mistakes while there's still time to fix them.

Example — Homework Helper App: Before Sam starts building, Priya and Mr. Chen are supposed to review the written requirements together and confirm they're right.

🧠 **Remember It Like This:** Check it before you build it — not after.

INVALIDATED REQUIREMENTS

In Simple Words: This is like sending out party invitations without ever double-checking the date is correct — if nobody ever confirms a requirement is actually correct, mistakes sail straight through.

Example — Homework Helper App: The requirements document says the app should support "up to 8 classes per student," but nobody at Maple Street actually confirmed that number with Mr. Chen. It turns out some students take 9 classes. Because it was never validated, the mistake wasn't caught in time.

🧠 **Remember It Like This:** An unchecked requirement is a mistake waiting to happen.

INSPECTION PROFICIENCY

> 📖 **New Word!** inspection means a careful, organized review of a document by other people, specifically to find mistakes in it.

In Simple Words: This is like asking a friend who's terrible at spelling to proofread your essay — if the reviewer isn't skilled at spotting problems, they'll miss them, even while trying hard.

Example — Homework Helper App: Priya is new to reviewing requirements documents and doesn't yet know what to look for. She reads through Maya's document and says, "Looks good to me!" — but misses that two requirements actually contradict each other, because she hasn't learned how to inspect carefully yet.

🧠 **Remember It Like This:** A weak inspector lets weak requirements sail right through.

🎯 **Quick Recap:**

- **Requirements Validation Risks** — trouble hiding while checking that requirements are correct.
- **Invalidated requirements** — never double-checking a requirement lets mistakes through.
- **Inspection proficiency** — reviewers need real skill to actually catch problems.

Risks When Managing Requirements Over Time (Management)

REQUIREMENTS MANAGEMENT RISKS

In Simple Words: This is like keeping your party plan updated when the guest list keeps changing, right up until the big day — things change, and requirements need to be tracked and controlled as they do.

Example — Homework Helper App: Even after Maya's requirements document is written and checked, the project keeps changing all the way to September 30th — and someone has to keep track of every change.

🧠 **Remember It Like This:** Requirements aren't "done" just because they're written down — they need to be watched.

CHANGING REQUIREMENTS

In Simple Words: This is like a friend changing their pizza order after you've already ordered — some change is normal, but if it happens without any control, chaos follows.

Example — Homework Helper App: In week 4, a group of 8th graders suddenly asks for a totally new "group project tracker" feature. If Maya just says "sure, add it!" without checking the impact on the deadline, the whole schedule could fall apart.

🧠 **Remember It Like This:** Change isn't the enemy — uncontrolled change is.

REQUIREMENTS CHANGE PROCESS

In Simple Words: This is like having a rule that any change to the party plan has to be written down and approved by the birthday kid first — a clear process for handling changes keeps things from spiraling out of control.

Example — Homework Helper App: Maya sets up a simple rule: any new request goes on a "Change Request Form," gets reviewed at Monday's meeting, and Mr. Chen must approve it before Sam builds it. That process protects the deadline.

🧠 **Remember It Like This:** No process for change means every change becomes an emergency.

UNIMPLEMENTED REQUIREMENTS

In Simple Words: This is like promising a friend a party favor bag and then simply forgetting to make one — a requirement everyone agreed on, but that never actually gets built.

Example — Homework Helper App: Everyone agreed weeks ago that the app needed a "mark homework as done" checkbox. But in the rush before the Tech Fair, it slips through the cracks, and Sam simply forgets to build it. Nobody notices until demo day.

🧠 **Remember It Like This:** An agreed requirement that's never built is a promise that's never kept.

EXPANDING PROJECT SCOPE

> 📖 **New Word!** scope creep means a project slowly growing bigger and bigger with extra features, without anyone officially deciding it should.

In Simple Words: This is like planning a small backyard birthday party that quietly turns into a giant carnival with a bounce house, a clown, AND fireworks — nobody decided that on purpose, it just crept in bit by bit.

Example — Homework Helper App: Homework Helper started as "track assignments and reminders." By week 5, it's grown to also include a chat feature, a leaderboard, and a homework-sharing tool nobody planned for — with the same September 30th deadline. That's scope creep, and it puts the whole project at risk.

🧠 **Remember It Like This:** Scope creep is a snowball — small at first, but it grows if nobody stops it.

🎯 **Quick Recap:**

- **Requirements Management Risks** — trouble hiding in keeping requirements up to date over time.
- **Changing requirements** — some change is normal, but it needs control.
- **Requirements change process** — a clear process keeps changes from causing chaos.
- **Unimplemented requirements** — an agreed requirement that's never built is a broken promise.
- **Expanding project scope (scope creep)** — a project quietly growing bigger without anyone deciding it should.

Our Chapter's Guidebook

REFERENCE: KARL WIEGERS AND JOY BEATTY, *SOFTWARE REQUIREMENTS*

In Simple Words: This is like how Maya's team has one trusted encyclopedia they always check when they're not sure about something — every field of study has a few "big books" that experts trust, and this whole course is built on one of them.

Example — Homework Helper App: All of the ideas in this chapter — risk management, elicitation, analysis, specification, validation — come from a real book that requirements engineers actually use: *Software Requirements* (Third Edition) by Karl Wiegers and Joy Beatty. It's like the ultimate rulebook for figuring out what software should do.

🧠 **Remember It Like This:** Every great class has a trusted book behind it — this is ours.

🎯 **Quick Recap:**

- **Reference** — this whole chapter's ideas come from the real textbook *Software Requirements* by Karl Wiegers and Joy Beatty.

🏆 Chapter Wrap-Up

Maya, Sam, and Priya still have weeks of work ahead before the Fall Tech Fair, but now they have a superpower: they know how to spot trouble before it becomes a disaster. Every time Priya spots a risk early, every time Maya writes a clear vision statement, and every time the team double-checks a requirement before building it, they're making Homework Helper stronger and their September 30th deadline safer. Great software isn't built by hoping nothing goes wrong — it's built by teams who go looking for trouble on purpose, so they can beat it to the punch.

If you remember ONLY these things…

- Risk management means finding, ranking, avoiding, and controlling problems before they happen — like a lookout watching for storms.
- Elicitation risks hide in the very first step: asking people what they need.
- Analysis risks hide while studying and ranking what was gathered.
- Specification risks hide while writing requirements down clearly.
- Validation risks hide while double-checking requirements are correct.
- Management risks hide in keeping requirements under control as things change, all the way to the finish line.
- Every good idea in this chapter comes from a real textbook — *Software Requirements* by Karl Wiegers and Joy Beatty.

🧠 Total Recall Cheat Sheet

Every topic from every chapter is listed here next to just its memory hook. Try covering the hook with your hand and saying it back from memory before peeking — that's the fastest way to make it permanent.

CHAPTER 5: FINALIZING REQUIREMENTS DEVELOPMENT

- **Purposes of Prototypes:** A prototype is a "practice castle" — build it small and rough first so the real one doesn't collapse.
- **Paper Prototypes:** Paper prototypes are "sketch it, don't code it" — if a crayon can fix the problem, don't call a programmer yet.
- **Mock-ups:** A mock-up is a movie set — it looks completely real from the front, but there's nothing behind the door.
- **Wireframe:** A wireframe is the house blueprint — it shows where the rooms connect, not what color the walls are.
- **Proof of Concept:** Proof of concept means "does the plumbing actually work?" — never mind how the bathroom looks yet.
- **Typical Applications of Software Prototypes (Table 15-1):** Two questions, four jobs — throw it away or grow it, looks or works — that's the whole table.
- **Classes of Prototype Attributes:** Every prototype has three tags, like a luggage tag: what it's testing, whether it's keeping its job, and what it's made of.
- **Ways to Fit Prototyping into the SDLC (Figure 15-1):** Prototyping is a boomerang, not an arrow — it always throws information back before the real building begins.
- **Working with Prototypes (Figure 15-2):** Four steps forward, but the eraser always works backward too.
- **Dialog Map:** A dialog map is a subway map for buttons — every arrow is a tap that takes you somewhere new.
- **Risks of Prototyping:** A shiny prototype is a movie trailer, not the movie — don't sell tickets yet.
- **Prototyping Success Factors:** Say WHY you're building it before you build it — a prototype without a purpose is just doodling.
- **Why Prioritize Requirements?:** You can't put every topping on the pizza — prioritizing is choosing the toppings that matter most.
- **Prioritization Pragmatics:** Before you rank the runners, you need to know the race rules — needs, importance, timing, dependencies, grouping, and cost.
- **In or Out:** In or Out is a light switch, not a dimmer — every requirement gets flipped one way or the other.
- **Pairwise Comparison and Rank Ordering:** Pairwise comparison is a tournament bracket — two requirements enter, one priority order comes out.
- **MoSCoW:** MoSCoW: Must happens, Should tries, Could waits, Won't waits longer.
- **Three-Level Scale (Importance × Urgency):** Important AND urgent wins the race — the further from that corner, the longer it can wait.
- **Multipass Prioritization:** Rank once, then check twice — priorities and dependencies have to agree with each other.

- **The $100 Game (Resource-Allocation Framing):** Pretend money reveals real priorities — people spend the most on what they truly want.
- **Value/Cost/Risk-Based Prioritization Model:** Priority = Value ÷ (Cost + Risk) — the best deal wins, not just the fanciest feature.
- **Validation and Verification:** Verification checks the recipe was followed. Validation checks the food actually tastes good.
- **Reviewing Requirements:** Catching a typo before you print 100 copies is way cheaper than after.
- **Peer Desk Check:** One friend, one careful read — that's a peer desk check.
- **Pass-Around:** Pass-around is a group chat for reviewers — everyone reads it, nobody has to meet.
- **Walkthrough:** A walkthrough is a guided tour — the author drives, everyone else asks questions along the way.
- **Formal Peer Reviews:** Formal peer review means it's on the record, not just a hallway chat.
- **Inspection:** Inspection is the "gold standard" review — when it really matters, this is the one you run.
- **The Inspection Process:** Four roles, seven steps, and two loops back — an inspection is a courtroom for your document, not a casual read.
- **Defect Checklist — Completeness:** Completeness asks: "Did we forget anything?" — like checking your backpack twice before the trip.
- **Defect Checklist — Correctness:** Correctness asks: "Is what's written actually TRUE and clear?" — no contradictions allowed.
- **Defect Checklist — Quality Attributes:** "Fast" isn't testable. "Under 2 seconds" is — quality needs numbers, not just nice words.
- **Requirements Review Tips:** A review with no plan is just skimming — plan it like you'd plan a study session.
- **Requirements Review Challenges:** Big documents, big teams, far-apart people, and unprepared readers — the four things that make reviews go sideways.
- **Prototyping in Requirements Validation:** A sentence can hide a problem. A prototype can't — seeing it makes it real.
- **Testing the Requirements:** Same starting line, two separate racers, then compare notes at the finish — that's how testing catches what one person alone would miss.
- **Acceptance Criteria:** Acceptance criteria are the finish-line checklist — cross every item off, or the race isn't officially over.
- **Why Reuse Requirements?:** Don't rewrite the book report from scratch — borrow the good parts and make them your own.
- **Dimensions of Requirements Reuse:** Reuse has three dials — how much you borrow, how much you change, and how you grab it.
- **Requirement Evolution:** One requirement, many family branches — reuse doesn't clone it, it grows separate versions.
- **Requirement Patterns:** A requirement pattern is Mad Libs for requirement-writing — just fill in your blanks.
- **Reuse Barriers:** Reuse fails when pride (NIH), mismatch (NAH), or messy differences get in the way.
- **Reuse Success Factors:** Good reuse needs a library, a map of connections, shared words, and a culture that says "yes, borrow it!"

- **Use of Requirements:** Baselined requirements are a recipe three cooks share at once — the planner, the builder, and the tester all read the same page.

CHAPTER 6: REQUIREMENTS FOR SPECIFIC PROJECT CLASSES

- **What Is a Packaged Solution (COTS)?:** Why bake it from zero when you can buy the mix and just add sprinkles?
- **Software as a Service (SaaS) & Cloud Solutions:** SaaS means you rent the software, and the cloud keeps it running while you sleep.
- **Why Requirements Still Matter for Packaged Solutions:** Buying doesn't mean skipping the wish list — the wish list is what picks the right toy AND makes it fit.
- **Configuring, Integrating & Extending a Package:** Configure it, connect it, or create new pieces for it — that's how you make a package truly fit.
- **Requirements for Selecting a Packaged Solution:** Before you say yes, make the list, then check the box.
- **Developing User Requirements (for Selection):** Know what you need before you go shopping for it.
- **Considering Business Rules:** The app has to follow the school's rules — the school doesn't follow the app's rules.
- **Defining Quality Requirements:** Good software should be fast, easy, flexible, friendly with others, honest with your data, and locked up tight.
- **Evaluating Solutions:** Score every option like a judge — then let the numbers help you choose.
- **The Spectrum of Implementation Effort:** Out-of-the-box is the easiest slice; Extended is a whole custom pizza — know where on the spectrum you're standing.
- **Common Challenges With Packaged Solutions:** Buying software sounds easy — until you hit these five bumps.
- **Too Many Candidates:** Too many choices — all looking good on the surface — can freeze you like a game of statues.
- **Too Many Evaluation Criteria:** Pick your top few must-haves based on what you actually need — don't drown in a checklist ocean.
- **Vendor Misrepresents Package Capabilities:** Trust, but verify — even a friendly salesperson can be wrong.
- **Incorrect Solution Expectations:** Demos are movie trailers — always watch the whole movie before you clap.
- **Users Reject the Solution:** Buying it isn't the finish line — people actually using it is.
- **Appropriate Levels of Requirements Detail:** Strangers need more directions than friends do.
- **Acquirer–Supplier Interactions:** Ask, agree, then deliver — that's the outsourcing dance.
- **Plan Time for Multiple Review Cycles:** One look is never enough — plan for round two AND round three.
- **Hold Requirements Workshops Immediately Followed by Implementation Tasks:** Strike while the whiteboard's still warm.
- **Peer Reviews and Prototypes Reveal Supplier Interpretation:** A rough sketch now saves a rebuild later.

- **Risk: Contractors Lacking Domain Knowledge:** An outside team may be expert coders but rookie hallway-walkers — spell out what only insiders know.
- **Business Process Analysis (BPA):** Watch it first, fix it second.
- **Business Process Reengineering (BPR):** Sometimes you don't fix the old wheel — you build a whole new cart.
- **Business Process Improvement (BPI):** Little fixes, again and again, keep things running smooth.
- **Business Process Management (BPM):** Managing every process, not just one — that's the big-picture job.
- **Business Process Model and Notation (BPMN):** BPMN is a process's own comic strip — shapes and arrows telling the story.
- **What Is Big Data?:** Big data means lots, fast, and messy — like a school hallway at the bell, times a million.
- **Managing Big Data:** Big data needs big organizing — find it, keep it, use it, fast.
- **Modeling Related Data With ERDs:** An ERD is a map of how your data pieces are best friends with each other.
- **Examples of Big-Data-Generating Organizations:** When millions of people use something every day, the data becomes a mountain, not a molehill.
- **Data vs. Information:** Data is the pile of puzzle pieces. Information is the finished picture.
- **The Simple Business Analytics Framework:** Data, Analysis, and Usage chase each other in a circle forever.
- **Information Usage by People:** Great information is useless if nobody can actually get it or use it.
- **Delivery Mechanism:** It's not just what you say — it's how it gets delivered to the door.
- **Format:** Same information, different outfits, depending on who's looking.
- **Flexibility:** Some people want it served exactly as-is; others want to mix it themselves.
- **Big Data Structure:** Some data lines up in rows. Some data is a wild, tangled mess.
- **Unstructured Data:** Unstructured data is a junk drawer — everything's in there, but good luck finding the tape.
- **Semi-Structured Data:** Semi-structured data has labels on the junk drawer, even if the stuff inside is still loose.
- **Data-Based Requirements:** Four big data questions to answer before you build anything.
- **Data Sources:** Know your ingredients' grocery store before you start cooking — and check if you're also bringing in leftovers from before.
- **Data Storage:** A growing pile of data needs a bigger and safer shelf every year.
- **Data Management & Governance:** Governance is the rulebook for who touches the data, how it's kept safe, and when it gets thrown away.
- **Data Extraction:** Extraction asks: do you need the answer right now, or is overnight okay?
- **Defining Analyses That Transform the Data:** Ask about yesterday, today, and tomorrow — each needs a different kind of answer.
- **System Requirements, Architecture, and Allocation:** Big requirement first, then figure out who or what handles which piece.

- **A System's Architecture Has Three Elements:** Every system is built from parts, what they show the world, and how they connect.
- **Components:** A component can be code, a gadget, OR a person — anything doing a job in the system.
- **Externally Visible Properties:** You don't need to see the engine — just know what the part promises to do.
- **Connections:** Connections are the wires (or messages) letting parts talk to each other.
- **System Requirements Decomposition & Allocation:** One big requirement branches into software, hardware, and people jobs, each given to the right part.
- **Timing Requirements for Real-Time Systems:** Real-time means on time, every time — late is a fail.
- **Predictability:** Predictable means you could set your watch by it.
- **Execution Time:** Execution time is the stopwatch time from start to finish of a task.
- **Latency:** Latency is the little pause between "something happened" and "the system notices."
- **Additional Timing & Scheduling Issues to Explore:** Real-time systems need a whole extra checklist that regular apps can skip.
- **Periodicity (Frequency) of Task Execution:** Periodicity is how often the alarm clock rings.
- **Deadlines and Tolerances (Float):** A deadline is the line; tolerance is how far you can wiggle before crossing it.
- **Typical and Worst-Case Execution Time per Task:** Plan for the normal day AND the messiest possible day.
- **Consequences of Missing a Deadline:** Ask what breaks (or falls down the stairs) if the deadline is missed.
- **Minimum, Average, and Maximum Arrival Rate of Data:** Data can trickle in, flow steadily, or flood in — be ready for all three.
- **Maximum Time Before First Input/Output Is Expected:** Don't leave people staring at silence for too long.
- **Timeout — What To Do If Data Isn't Received in Time:** Always have a backup plan for when the expected data never shows up.
- **The Sequence in Which Tasks Must Run:** Some tasks are picky — they need their turn, in the right order.
- **Tasks That Must Begin/End Before Other Tasks Begin:** Socks before shoes — some tasks are simply first.
- **Task Prioritization & Pre-emption:** The most urgent task gets to cut in line.
- **Functions That Depend on System Mode:** Same machine, different rules, depending on the mode it's in.

CHAPTER 7: REQUIREMENTS MANAGEMENT

- **Definition of Requirements Management:** Requirements management is like watering a garden — you don't plant the seeds once and walk away, you tend to it the whole time it's growing.
- **Requirements Baseline:** A baseline is a "line in the sand" — everyone agreed to it, so you don't cross it without a good reason and everyone's okay with it.

- **Major Requirements Management Activities (The Four Pillars):** Four pillars, one steady table: Version, Change, Status, and Tracing hold up the whole project.
- **Version Control:** Version control is like a video game's save files — you can always go back to an earlier save if the newest one breaks something.
- **Change Control:** Change control is the "raise your hand and ask first" rule for requirements — no surprise changes allowed!
- **Requirements Status Tracking:** Status tracking is a chore chart for requirements — everyone can see what's done, what's doing, and what's still waiting.
- **Requirements Tracing:** Tracing is the dotted line on the treasure map — follow it, and you always find what connects to what.
- **Four Ways to Accommodate New or Changed Requirements:** When the suitcase won't zip, you either take something out, get help, get a bigger bag, or you can risk breaking the zipper — but breaking the zipper is never a good plan!
- **Requirement Attributes:** Every requirement gets its own trading card, packed with stats that tell its whole story.
- **Tracking Requirements Status:** Every requirement has a passport, and each status is a stamp on its journey.
- **Resolving Requirements Issues:** An issue tracker is a group project's "parking lot" list — every stray question gets parked somewhere safe instead of driving off and getting lost.
- **Why Manage Changes:** Ten kids randomly adding LEGO pieces makes a mess — one plan, checked and shared with everyone, keeps the castle standing strong.
- **Managing Scope Creep:** Every "just one more topping" adds up — a good project lead knows how to say "not now" before the pizza gets too big to lift.
- **Change Control Policy:** No official rulebook, no game — the Change Control Policy is the rulebook that keeps every change fair and orderly.
- **Change Control Process Description:** A process description is the instruction manual for making changes — follow the steps in order, and nothing gets skipped.
- **The Change Control Board (CCB):** The CCB is the judges' panel — nothing gets served (or shipped) without their official thumbs-up.
- **CCB Composition:** A CCB needs a seat at the table for everyone the decision touches — not just the loudest voice in the room.
- **CCB Decision-Making:** Agree on the voting rules BEFORE the vote — not in the middle of an argument about who wins.
- **Change Control Tools:** A good change control tool is a digital ballot box — it counts every vote correctly and shows you exactly what happened and when.
- **Measuring Change Activity:** Measuring change activity is the project's thermometer — the numbers tell you if things are running smoothly or running a fever.
- **Impact Analysis Procedure:** Think it through before you move the hamster cage — one small move can ripple out further than you'd expect.
- **Simple Evaluation Procedure:** Follow the recipe card in order — skip a step, and the cookies (or the analysis) don't turn out right.

- **Figure 28-5 — Checklist: Understanding the Implications:** A good detective never closes the case without checking every question on the list first.
- **Figure 28-6 — Checklist: What Work Products Are Affected:** Before you repaint, check the curtains AND the trim — a change always touches more than just the one wall you were thinking of.
- **Figure 28-7 — Worksheet: Estimating the Effort:** Add up the minutes for every chore on the worksheet, and suddenly "quick little change" turns into an honest, real number.
- **Figure 28-8 — Impact Analysis Template:** The impact analysis template is the "incident report" — one clean page that tells decision-makers everything they need to know, fast.
- **Change Management on Agile Projects:** The backlog is a lunch tray with three sections — what's on your fork right now can change, and snacks can move up front if you get hungry for them sooner.

CHAPTER 8: REQUIREMENTS TRACING AND TOOLS

- **Finding Missing Requirements:** No broken links — every big wish needs a small task, or the wish falls through the cracks.
- **Finding Unnecessary Requirements:** If it doesn't trace back, cut it back.
- **Certification and Compliance:** Traces are your receipts — proof you didn't skip the rules.
- **Change Impact Analysis:** Pull one thread, see the whole sweater.
- **Maintenance:** Good traces make future fixes fast, not scary.
- **Project Tracking:** Traces turn "I think we're close" into "we're 72% done, for real."
- **Reengineering:** Before you replace something old, map out everything it used to do.
- **Reuse:** Trace it once, reuse the whole bundle forever.
- **Testing:** Trace links turn "good luck finding it" into "look right here."
- **What Is a Requirements Traceability Matrix (RTM)?:** The RTM is the map that shows how every requirement connects, backward and forward.
- **Table 29-1 Style: One Combined Matrix:** One row, whole story — from wish to working code.
- **Table 29-2 Style: Multiple Separate Matrices:** Separate simple grids beat one giant messy table when you only need one kind of link.
- **Suspect Links:** A question-mark arrow means "look again," not "something's broken."
- **Step 1: Educate the Team and Management:** Teach the rules before you start the game.
- **Step 2: Select Which Link Relationships to Define:** Don't try to trace everything at once — you'll drown.
- **Step 3: Choose the Matrix Style and Storage Mechanism:** Pick your table style and your storage box before you start collecting cards.
- **Step 4: Identify Which Parts of the Product Need Traceability:** Trace the risky, important stuff first — not the wallpaper.
- **Step 5: Identify Who Supplies Each Type of Link, and the Coordinator:** Everyone links their own piece; one person makes sure it all fits.
- **Step 6: Modify Development Procedures to Update Links Promptly:** Update the link the moment the work is done, not "someday."
- **Step 7: Define Labeling Conventions:** Give everything a name tag so nobody gets confused at the party.

- **Step 8: Provide Trace Info as Small Units of Work Are Completed:** A little bit of tracing every day beats a giant scramble at the end.
- **Step 9: Audit the Trace Information Periodically:** Check your traces regularly, or “done” might really mean “not really done.”
- **Difficult to Keep Documents Current and Synchronized:** Paper documents drift apart like boats without anchors.
- **Communicating Changes to All Affected Team Members Is a Manual Process:** Documents don’t tap people on the shoulder — you have to do it yourself, every single time.
- **Not Easy to Store Supplementary Attributes About Each Requirement:** A plain document is a name tag; a good tool is a full trading card.
- **Hard to Define Links Between Requirements and Other System Elements:** Words on a page can’t hold hands with other words on other pages.
- **Tracking Status of Individual Requirements and the Whole Set Is Cumbersome:** Counting by hand is slow; a real tool counts for you.
- **Concurrently Managing Requirement Sets Planned for Different Releases Is Tricky:** Moving a requirement between versions by hand is like moving furniture without a dolly.
- **Reusing a Requirement Means Manually Copying Text From One Document Into Another:** Copy-paste reuse means two copies that can quietly grow apart.
- **Difficult for Multiple, Especially Geographically Separated, Participants to Modify Requirements Together:** One document, many hands, equals many accidents.
- **No Convenient Place to Store Rejected/Deleted Requirements:** A document has no memory bin for the ideas you said “not now” to.
- **Identifying Missing, Duplicate, and Unnecessary Requirements Is Difficult:** Duplicates and gaps hide easily in a giant wall of text.
- **Elicitation Tools:** Capture it as it happens — memory fades, notes don’t.
- **Prototyping Tools:** Try the cardboard treehouse before you build the real one.
- **Modeling Tools (e.g., UML):** Blueprints before bricks — draw the plan before you build the app.
- **Manage Versions and Changes:** The tool remembers every version, so you don’t have to.
- **Store Requirements Attributes:** Real trading cards, not just name tags — the tool holds all the stats.
- **Facilitate Impact Analysis:** One click shows the whole ripple, not just the splash.
- **Identify Missing and Extraneous Requirements:** The tool shines a flashlight into the gaps and the extras.
- **Track Requirements Status:** Status at a glance, not status by guesswork.
- **Control Access:** Not everyone needs the keys to every room.
- **Communicate With Stakeholders:** The tool taps stakeholders on the shoulder automatically.
- **Reuse Requirements:** Reuse the whole traced bundle, not just the words.
- **Track Issue Status:** Every issue gets a visible tag, from “uh-oh” to “all fixed.”
- **Generate Tailored Subsets:** Ask the tool a smart question, get exactly the pile you need.
- **RM Tool Feature Families:** One tool, six toolboxes — managing, capturing, help, tracing, outputting, and reporting/integrating — from capturing an idea to tracing it to printing the final report.

- **RM Tools Integrate With Other Kinds of Tools (Figure 30-2):** No tool is an island — the RM tool is the central hub that keeps everyone's tools talking.
- **Assign an Experienced BA to Own Tool Setup and Process Adaptation:** Let the person who understands the "why" set up the "how."
- **Think Carefully About the Requirement Types You Define:** Not 30 tiny folders, not 1 giant junk drawer — find the middle.
- **Use the Tool to Facilitate Communication With Stakeholders in Various Locations:** Open the doors wide enough to let ideas in, not so wide anyone can wreck the place.
- **Don't Try to Capture Requirements Directly in the RM Tool During Early Elicitation Workshops:** Brainstorm on scrap paper first; move to the real filing cabinet once ideas calm down.
- **Use Requirements-Development (RD) Tools During Elicitation Only If Confident They Won't Slow Discovery Down:** A fancy tool that slows down the conversation isn't helping — it's hurting.
- **Don't Define Trace Links Until Requirements Stabilize:** Don't tie the ribbon until the present stops changing shape.
- **Set a Cutover Date:** Pick a moving day, and actually move on that day.
- **Identify a Tool Advocate:** Every new tool needs a biggest fan who helps everyone else fall in love with it too.
- **Overcome the Belief That "The Tool Won't Add Value":** Trade "the tool is pointless" for a real story about the pain of not having one.
- **Train Users Rather Than Expecting Them to Self-Discover the Tool:** Left alone, people learn the basics; taught properly, they learn the superpowers.
- **Don't Expect Instant Results:** Test the water in the shallow end before you dive into the deep end.
- **Where These Ideas Come From (Wiegers & Beatty):** Even superheroes have training manuals — this chapter's ideas come from requirements engineering's favorite one.

CHAPTER 9: IMPROVING REQUIREMENTS PROCESSES

- **Objective of Process Improvement:** Watch the game film, fix the plays — that's how every team (and every requirements process) levels up.
- **How Requirements Relate to Other Project Processes:** Requirements sit in the middle of the wheel — every other process spins around them.
- **Project Planning Process:** Requirements tell the plan what's needed; the plan tells requirements what actually fits.
- **Project Tracking and Control Process:** Tracking watches where requirements stand; changes ask requirements to grow.
- **Change Control Process:** No baseline, no fair fight — lock in the rules before you change the game.
- **Acceptance & System Testing Processes:** Requirements write the test; testing proves the promise was kept.
- **Construction Process:** Every brick of code should trace back to the instruction that asked for it.
- **User Documentation Process:** The user guide is just the requirements, translated into "here's how you use it."
- **Requirements and Various Stakeholder Groups:** A stakeholder doesn't have to build the app to care deeply about it.

- **Management:** Management draws the fence around the playground — you build inside the lines they set.
- **Marketing or Product Management:** Marketing listens to the crowd outside and brings their wishes to the table.
- **Technical Support:** Support hears the complaints first — and turns them into gold for the next version.
- **Users:** Users don't just use the app — they help write its wish list.
- **Project Sponsor:** The sponsor plants the seed — the whole tree of requirements grows from their "why."
- **Systems Engineering:** Systems engineering decides which toy box each requirement belongs in.
- **Procurers:** Procurers go shopping for the project — and make sure you don't buy the wrong thing.
- **Legal Department:** Before you use someone else's stuff, Legal checks if you're actually allowed to.
- **Gaining Commitment to Change:** A great idea only works if people actually pick it up and use it.
- **Resistance Form 1: "I Don't Have Time to Improve":** Skip the practice today, and you'll fumble the play tomorrow.
- **Resistance Form 2: "Change Control Is Just a Barrier":** Change control isn't a locked gate — it's a crosswalk that keeps good decisions safe.
- **Resistance Form 3: "Writing This Down Is a Waste of Time":** Skip the notes now, pay in rewrites later — for years, not just for this launch.
- **Fundamentals of Software Process Improvement:** Four simple rules keep "getting better" from turning into a mess.
- **Principle 1: Evolutionary and Continuous:** Improvement isn't a finish line — it's a staircase you keep climbing.
- **Principle 2: People Change Only With Incentive:** Nobody changes for a rule — they change for a reason.
- **Principle 3: Goal-Oriented Change:** Don't change just to change — change to hit a target.
- **Principle 4: Treat Improvement as Mini-Projects:** Treat "getting better" like its own little project — with a plan, not just a wish.
- **Root Cause Analysis:** When something breaks, follow the bones back to the real fish — not just the fin you can see.
- **The Process Improvement Cycle:** Assess, plan, try, check — then spin the wheel again.
- **Types of Process Assets:** Process assets are your team's toolbox — built once, used forever.
- **Checklist:** A checklist is your memory's best friend — it never forgets, even when you do.
- **Example (as a process asset):** One great example is worth a thousand vague instructions.
- **Plan:** A plan is your map before the trip — not a diary you write after you're already lost.
- **Policy:** A policy is the rule everyone agrees to follow — even the boss.
- **Procedure:** A procedure is a recipe for getting something done — step 1, step 2, step 3, done.
- **Template:** A template is a coloring book page — the outline's already there, you just fill it in.
- **Requirements Engineering Process Assets:** One drawer helps you WRITE requirements well, the other helps you TAKE CARE of them afterward.
- **Performance Indicators (KPIs):** If you can't measure it, you can't prove you actually got better.
- **Creating a Requirements Process Improvement Road Map:** A road map turns "we should get better" into "here's exactly how, and here's how we'll know we did."

CHAPTER 10: SOFTWARE REQUIREMENTS & RISK MANAGEMENT

- **Risk Management:** Risk management means looking for storms before you set sail, not swimming through them after you're already soaked.
- **Assessment:** Assessment is checking risks the way a doctor checks you — look, understand, then decide what matters most.
- **Identification:** You can't dodge a problem you never spotted.
- **Analysis:** Every risk gets two questions: How likely? How bad?
- **Prioritization:** The biggest, scariest risks jump to the front of the line.
- **Avoidance:** The best-fixed problem is the one that never got the chance to happen.
- **Control:** If you can't avoid it, plan it, fix it, and keep watching it.
- **Management Planning:** Decide your move before the game gets scary — not during.
- **Resolution:** A plan is just talk. Resolution is doing.
- **Monitoring:** Risks change — keep watching, don't look away.
- **Requirements-Related Risks by Activity:** Different stages hide different traps — a risk hiding in the writing-down stage isn't the same risk hiding in the double-checking stage.
- **Requirements Elicitation Risks:** Ask badly at the start, and you'll build the wrong thing at the end.
- **Product vision and project scope:** No vision, no finish line.
- **Time spent on requirements development:** Rushing the planning means redoing the building.
- **Customer engagement:** A quiet customer means a guessing developer.
- **Completeness and correctness of requirements specifications:** A recipe missing a step bakes a broken cake.
- **Requirements for innovative products:** It's hard to describe a treasure you've never seen before.
- **Defining non-functional requirements:** It's not just WHAT it does — it's HOW WELL it does it.
- **Customer agreement on requirements:** Unsettled disagreements today become angry customers tomorrow.
- **Unstated requirements:** "Obvious" to you might be invisible on paper.
- **Existing product used as the requirements reference:** Copying the old thing can copy its old mistakes too.
- **Distrust between the business and the development team:** A team that doesn't trust each other can't build anything great together.
- **Requirements Analysis Risks:** Gathering wishes is step one. Studying them is step two.
- **Requirements prioritization:** If everything is "important," nothing actually is.
- **Technically difficult features:** The harder the feature, the bigger the risk of running out of time.
- **Unfamiliar technologies, methods, languages, tools, or hardware:** New tools in untrained hands make risky projects.
- **Requirements Specification Risks:** A badly written recipe ruins even a great idea.
- **Requirements understanding:** What you meant and what they understood aren't always the same thing.
- **Time pressure to proceed despite open issues:** An open question today is a bigger problem tomorrow.

- **Ambiguous terminology:** Vague words leave room for everyone to imagine something different.
- **Design included in requirements:** Say WHAT you need, not HOW to build it — not yet.
- **Requirements Validation Risks:** Check it before you build it — not after.
- **Invalidated requirements:** An unchecked requirement is a mistake waiting to happen.
- **Inspection proficiency:** A weak inspector lets weak requirements sail right through.
- **Requirements Management Risks:** Requirements aren't "done" just because they're written down — they need to be watched.
- **Changing requirements:** Change isn't the enemy — uncontrolled change is.
- **Requirements change process:** No process for change means every change becomes an emergency.
- **Unimplemented requirements:** An agreed requirement that's never built is a promise that's never kept.
- **Expanding project scope:** Scope creep is a snowball — small at first, but it grows if nobody stops it.
- **Reference: Karl Wiegers and Joy Beatty, *Software Requirements*:** Every great class has a trusted book behind it — this is ours.